

问题背景：子数组



	1	2	3	4	5	6	7	8	9	10
X	-1	-3	3	5	-4	3	2	-2	3	6

问题背景：子数组



	1	2	3	4	5	6	7	8	9	10
X	-1	-3	3	5	-4	3	2	-2	3	6

- 子数组为数组 X 中**连续**的一段序列

问题背景：子数组



	1	2	3	4	5	6	7	8	9	10
X	-1	-3	3	5	-4	3	2	-2	3	6

- 子数组 $X[3..7]$

问题背景：子数组



	1	2	3	4	5	6	7	8	9	10
X	-1	-3	3	5	-4	3	2	-2	3	6

- 子数组 $X[3..7]$
 - 求和为： $3 + 5 - 4 + 3 + 2 = 9$

问题背景：子数组



	1	2	3	4	5	6	7	8	9	10
X	-1	-3	3	5	-4	3	2	-2	3	6

- 子数组 $X[3..7]$
 - 求和为： $3 + 5 - 4 + 3 + 2 = 9$
- 子数组 $X[1..10]$

问题背景：子数组



	1	2	3	4	5	6	7	8	9	10
X	-1	-3	3	5	-4	3	2	-2	3	6

- 子数组 $X[3..7]$
 - 求和为： $3 + 5 - 4 + 3 + 2 = 9$
- 子数组 $X[1..10]$
 - 求和为： $-1 - 3 + 3 + 5 - 4 + 3 + 2 - 2 + 3 + 6 = 12$

问题背景：子数组



	1	2	3	4	5	6	7	8	9	10
<i>X</i>	-1	-3	3	5	-4	3	2	-2	3	6

- 子数组 $X[3..7]$
 - 求和为： $3 + 5 - 4 + 3 + 2 = 9$
- 子数组 $X[1..10]$
 - 求和为： $-1 - 3 + 3 + 5 - 4 + 3 + 2 - 2 + 3 + 6 = 12$
- 子数组 $X[3..10]$

问题背景：子数组



	1	2	3	4	5	6	7	8	9	10
<i>X</i>	-1	-3	3	5	-4	3	2	-2	3	6

- 子数组 $X[3..7]$
 - 求和为： $3 + 5 - 4 + 3 + 2 = 9$
- 子数组 $X[1..10]$
 - 求和为： $-1 - 3 + 3 + 5 - 4 + 3 + 2 - 2 + 3 + 6 = 12$
- 子数组 $X[3..10]$
 - 求和为： $3 + 5 - 4 + 3 + 2 - 2 + 3 + 6 = 16$

问题背景：子数组



	1	2	3	4	5	6	7	8	9	10
X	-1	-3	3	5	-4	3	2	-2	3	6

- 子数组 $X[3..7]$
 - 求和为： $3 + 5 - 4 + 3 + 2 = 9$
- 子数组 $X[1..10]$
 - 求和为： $-1 - 3 + 3 + 5 - 4 + 3 + 2 - 2 + 3 + 6 = 12$
- 子数组 $X[3..10]$
 - 求和为： $3 + 5 - 4 + 3 + 2 - 2 + 3 + 6 = 16$

问题：寻找数组 X 最大的非空子数组？

问题背景：子数组



	1	2	3	4	5	6	7	8	9	10
X	-1	-3	3	5	-4	3	2	-2	3	6

- 子数组 $X[3..7]$
 - 求和为： $3 + 5 - 4 + 3 + 2 = 9$
- 子数组 $X[1..10]$
 - 求和为： $-1 - 3 + 3 + 5 - 4 + 3 + 2 - 2 + 3 + 6 = 12$
- 子数组 $X[3..10]$
 - 求和为： $3 + 5 - 4 + 3 + 2 - 2 + 3 + 6 = 16$

问题：寻找数组 X 最大的非空子数组？

答案： $X[3..10] = 16$

- 形式化定义

最大子数组问题

Max Continuous Subarray, MCS

输入

- 给定一个数组 $X[1..n]$, 对于任意一对数组下标为 l, r ($l \leq r$)的**非空子数组**, 其和记为

$$S(l, r) = \sum_{i=l}^r X[i]$$

输出

- 求出 $S(l, r)$ 的**最大值**, 记为 S_{max}

蛮力枚举



	1	2	3	4	5	6	7	8	9	10
X	-1	-3	3	5	-4	3	2	-2	3	6

	1	2	3	4	5	6	7	8	9	10
X	-1	-3	3	5	-4	3	2	-2	3	6

- 数组 $X[1..n]$, 其所有的下标 $l, r (l \leq r)$ 组合分为以下两种情况

	1	2	3	4	5	6	7	8	9	10
X	-1	-3	3	5	-4	3	2	-2	3	6

- 数组 $X[1..n]$, 其所有的下标 $l, r (l \leq r)$ 组合分为以下两种情况
 - 当 $l = r$ 时, 一共 $C_n^1 = n$ 种组合

	1	2	3	4	5	6	7	8	9	10
X	-1	-3	3	5	-4	3	2	-2	3	6

- 数组 $X[1..n]$, 其所有的下标 $l, r (l \leq r)$ 组合分为以下两种情况
 - 当 $l = r$ 时, 一共 $C_n^1 = n$ 种组合
 - 当 $l < r$ 时, 一共 C_n^2 种组合

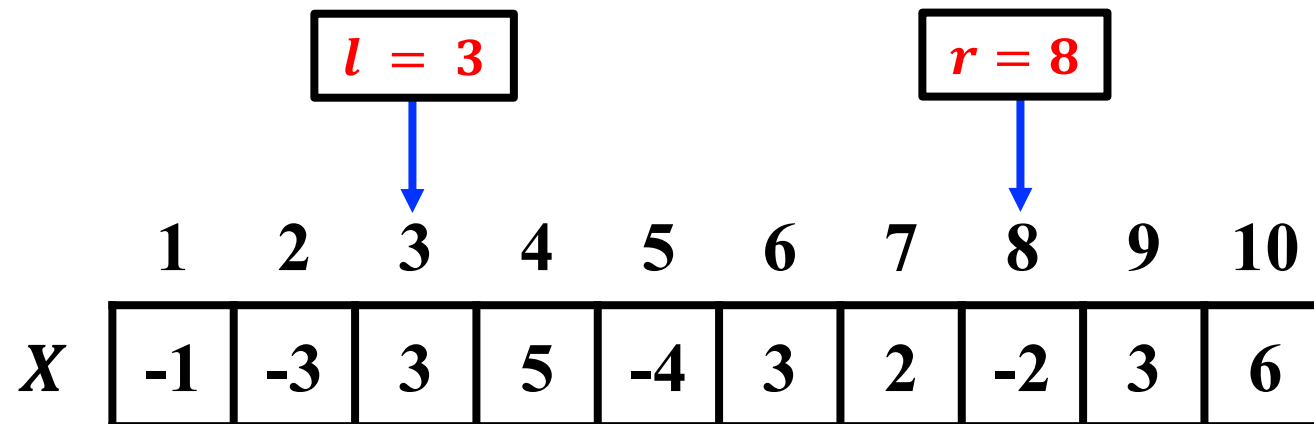
	1	2	3	4	5	6	7	8	9	10
X	-1	-3	3	5	-4	3	2	-2	3	6

- 数组 $X[1..n]$ ，其所有的下标 $l, r (l \leq r)$ 组合分为以下两种情况
 - 当 $l = r$ 时，一共 $C_n^1 = n$ 种组合
 - 当 $l < r$ 时，一共 C_n^2 种组合
- 枚举 $n + C_n^2$ 种下标 l, r 组合，求出最大子数组之和

蛮力枚举：算法实例



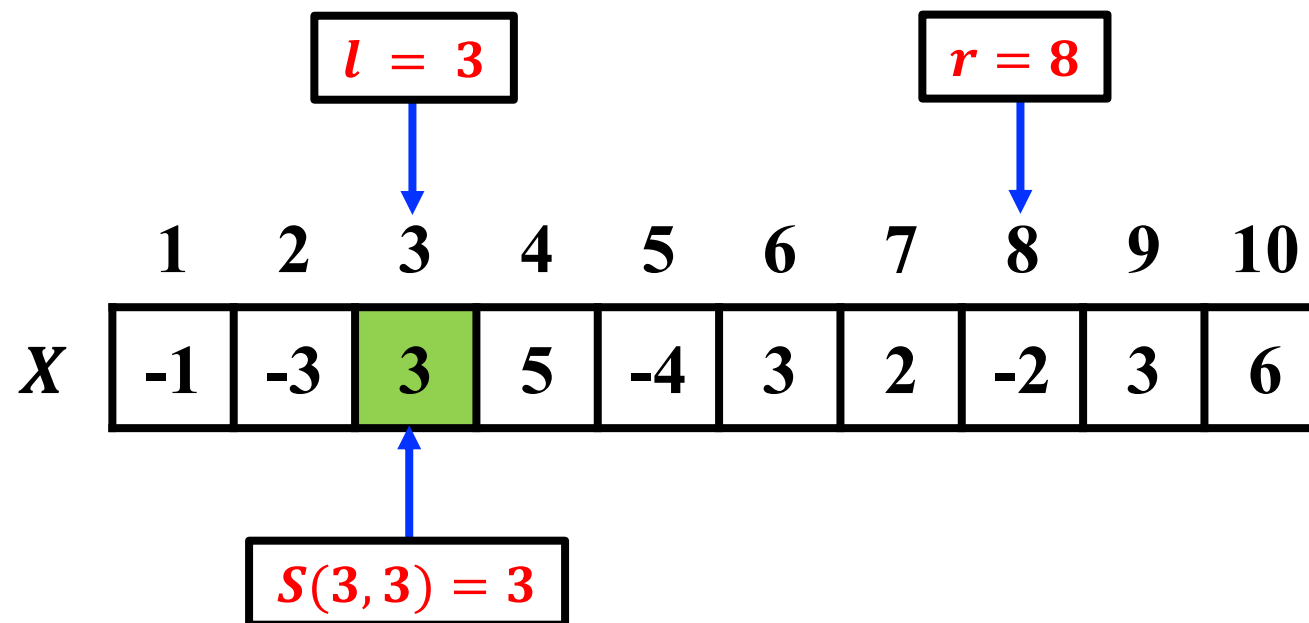
- $l = 3, r = 8$
- 计算 $S(3, 8)$:



蛮力枚举：算法实例



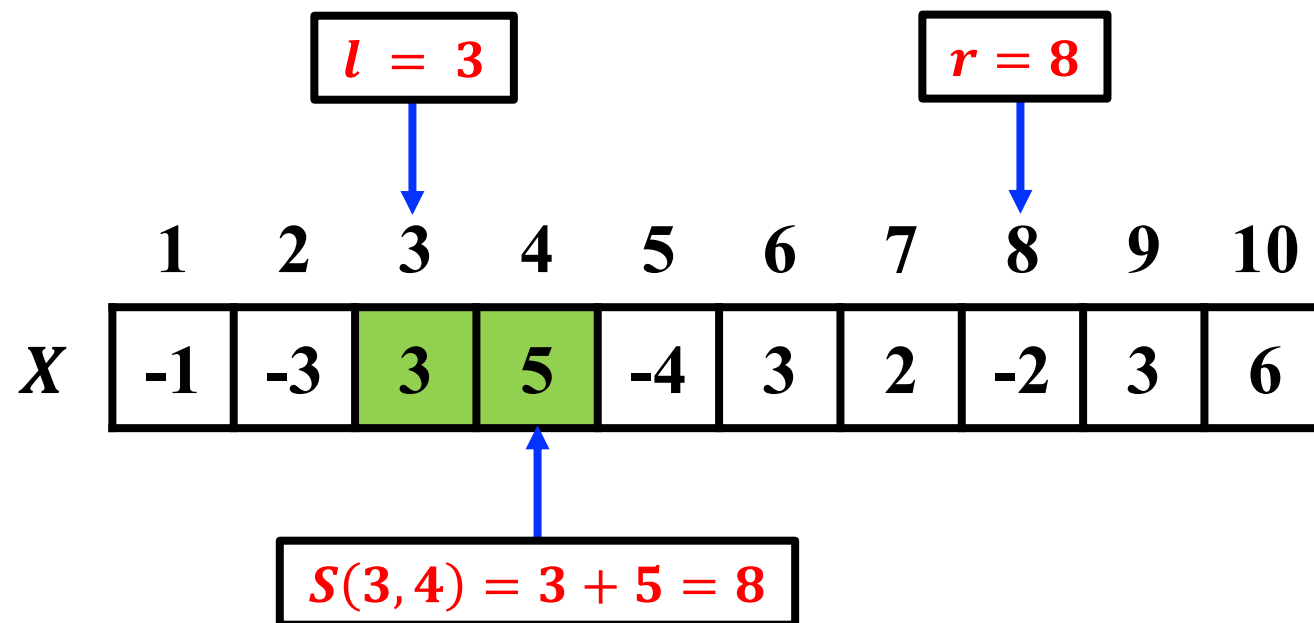
- $l = 3, r = 8$
- 计算 $S(3, 8)$:



蛮力枚举：算法实例



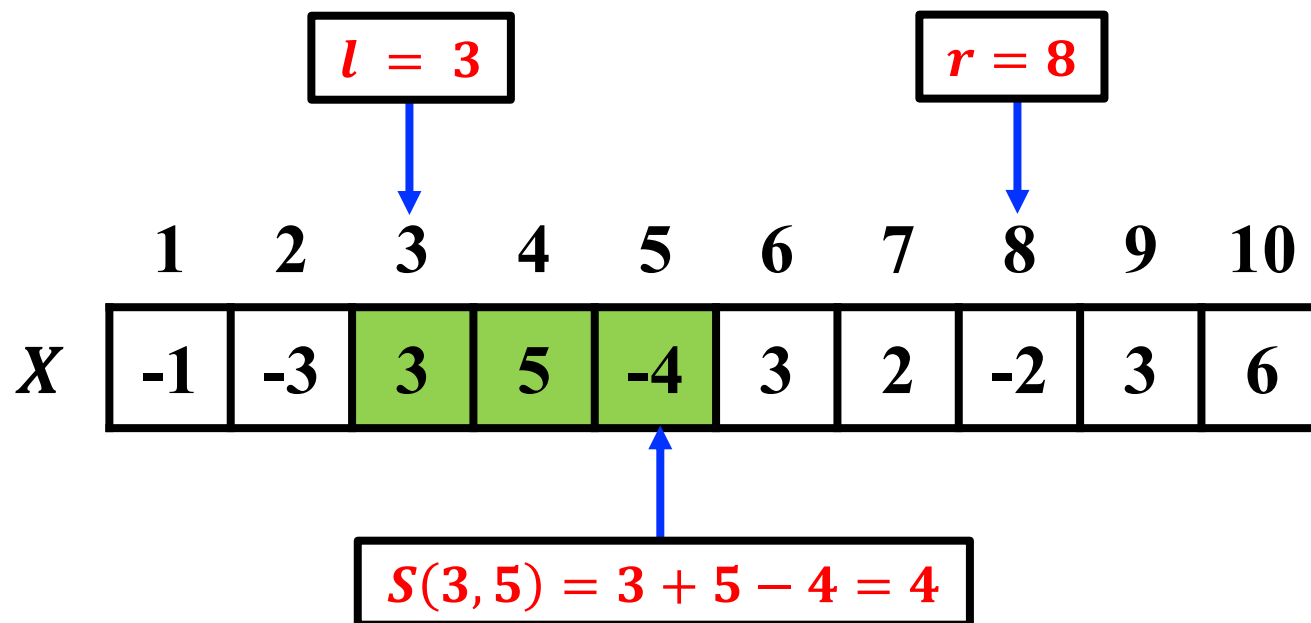
- $l = 3, r = 8$
- 计算 $S(3, 8)$:



蛮力枚举：算法实例



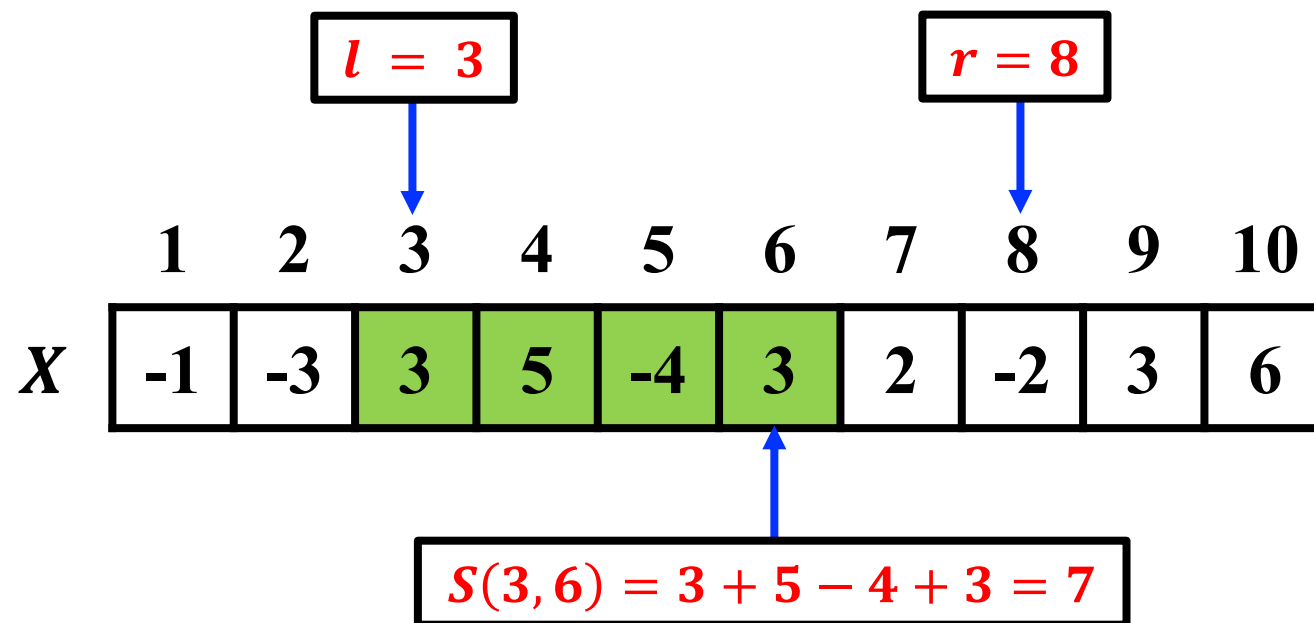
- $l = 3, r = 8$
- 计算 $S(3, 8)$:



蛮力枚举：算法实例



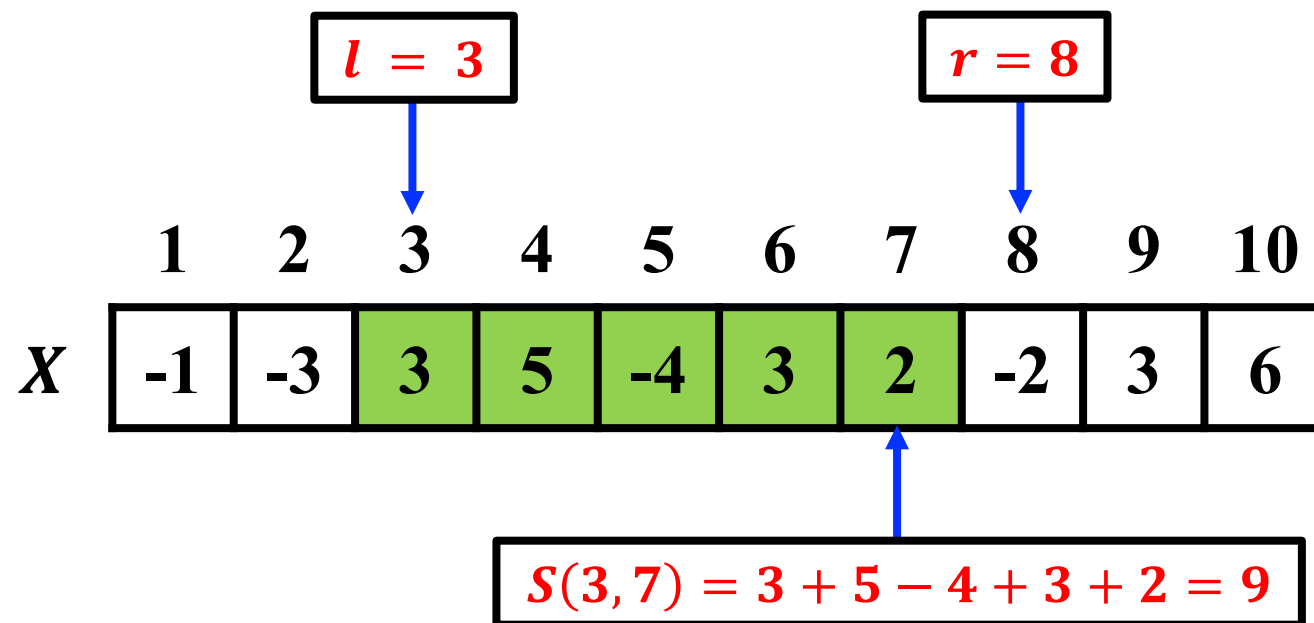
- $l = 3, r = 8$
- 计算 $S(3, 8)$:



蛮力枚举：算法实例



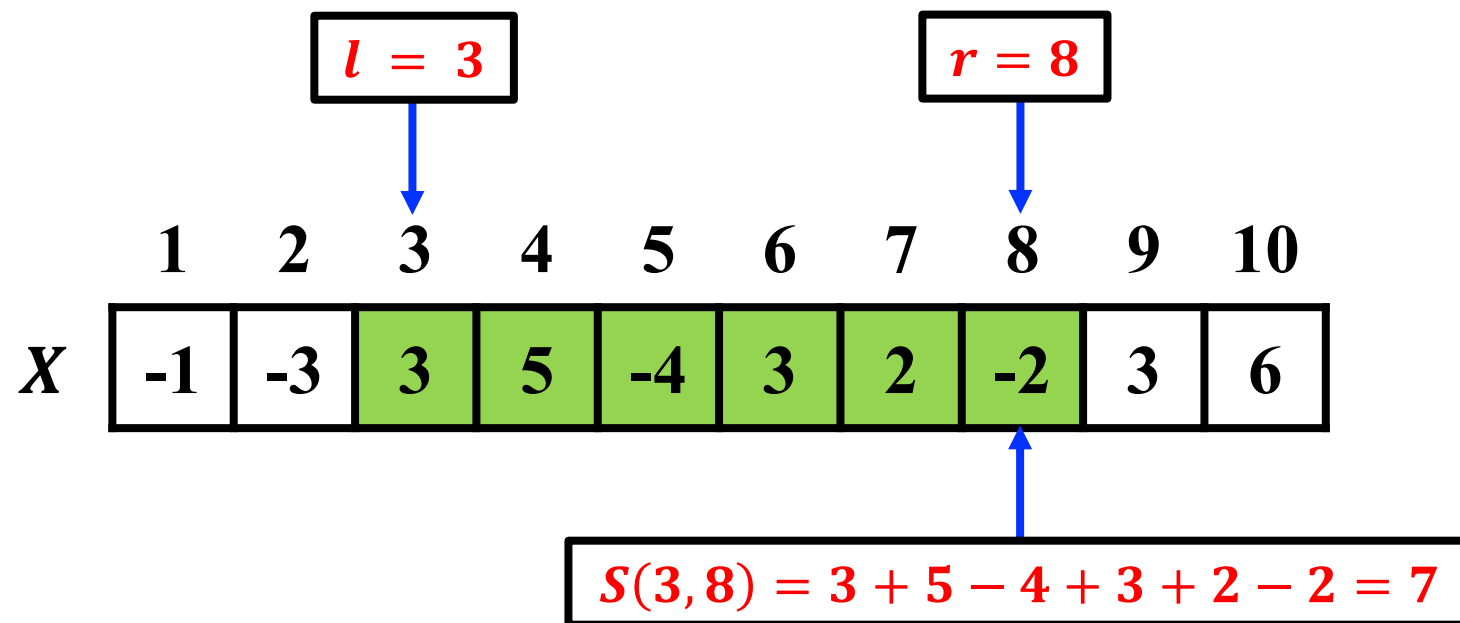
- $l = 3, r = 8$
- 计算 $S(3, 8)$:



蛮力枚举：算法实例



- $l = 3, r = 8$
- 计算 $S(3, 8)$:



蛮力枚举：算法实例



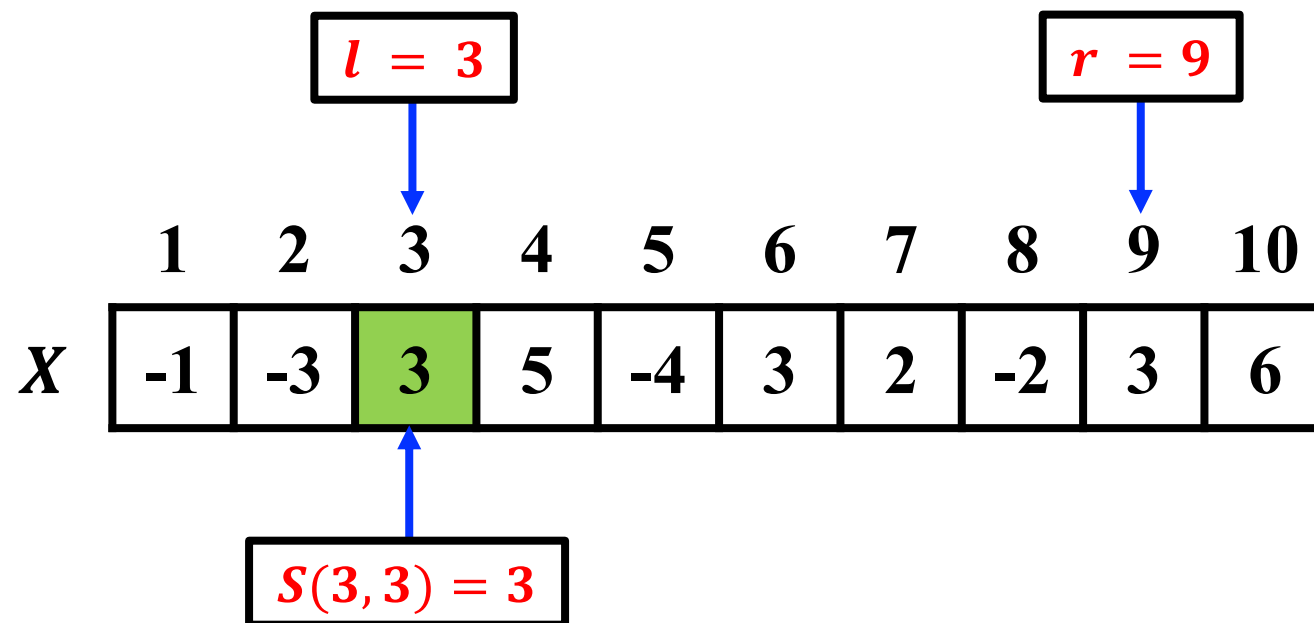
- $l = 3, r = 9$
- 计算 $S(3, 9)$:

		$l = 3$						$r = 9$		
	1	2	3	4	5	6	7	8	9	10
X	-1	-3	3	5	-4	3	2	-2	3	6

蛮力枚举：算法实例



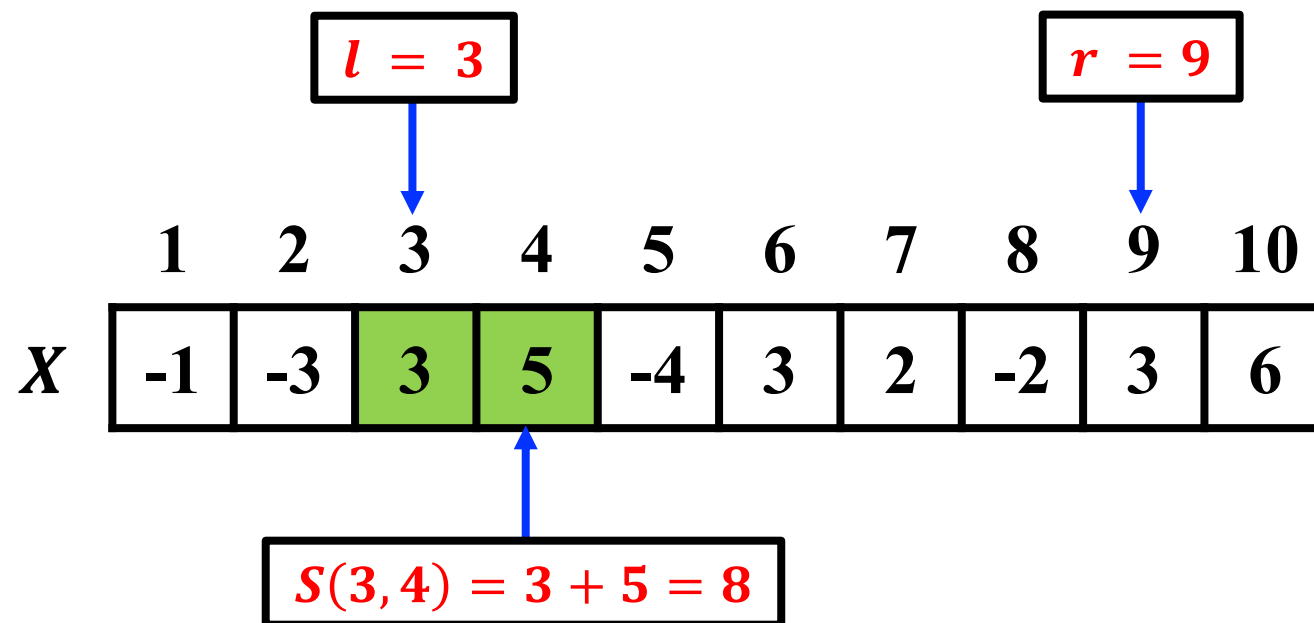
- $l = 3, r = 9$
- 计算 $S(3, 9)$:



蛮力枚举：算法实例



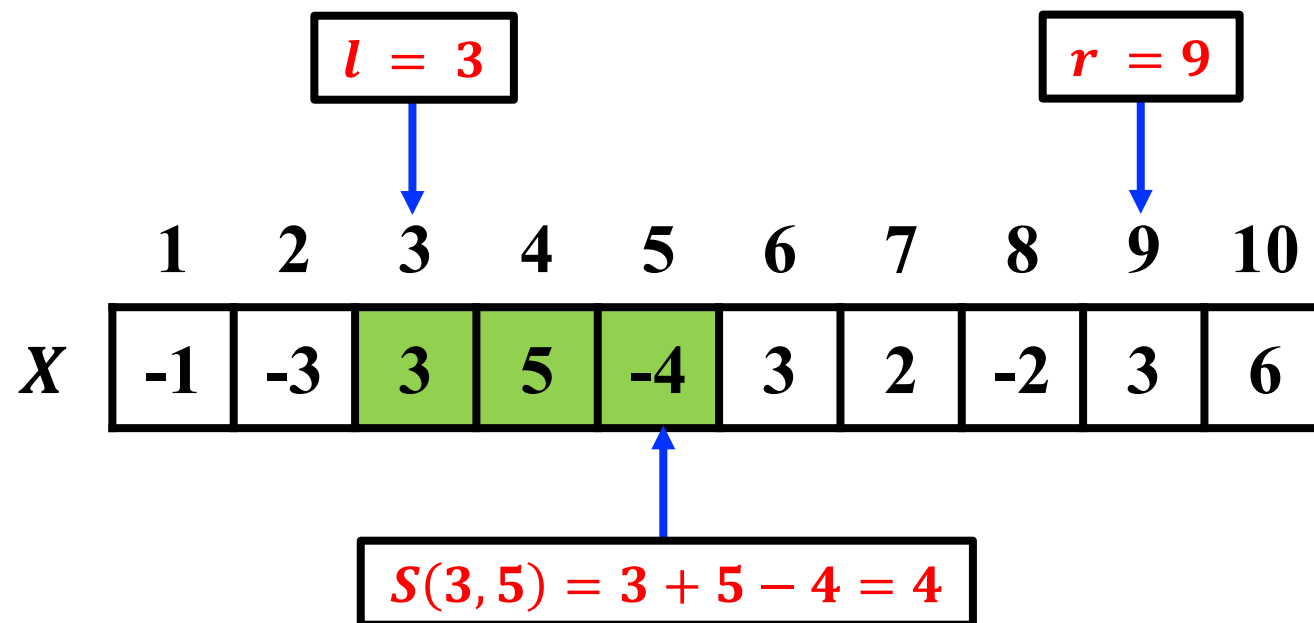
- $l = 3, r = 9$
- 计算 $S(3, 9)$:



蛮力枚举：算法实例



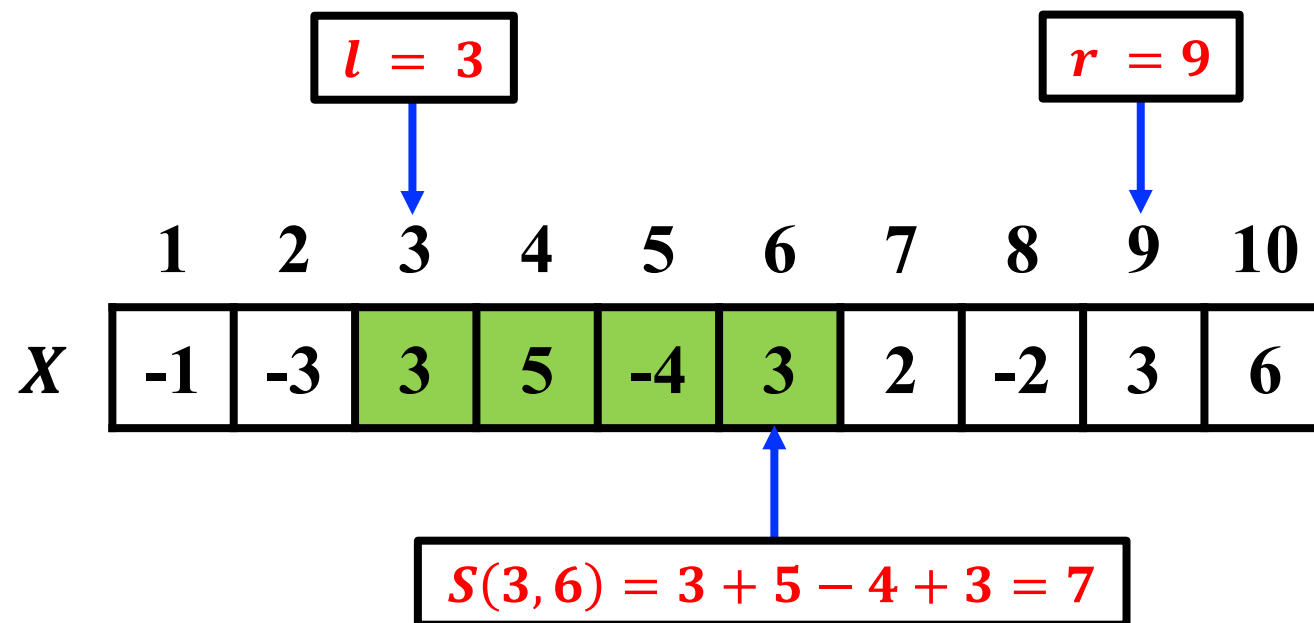
- $l = 3, r = 9$
- 计算 $S(3, 9)$:



蛮力枚举：算法实例



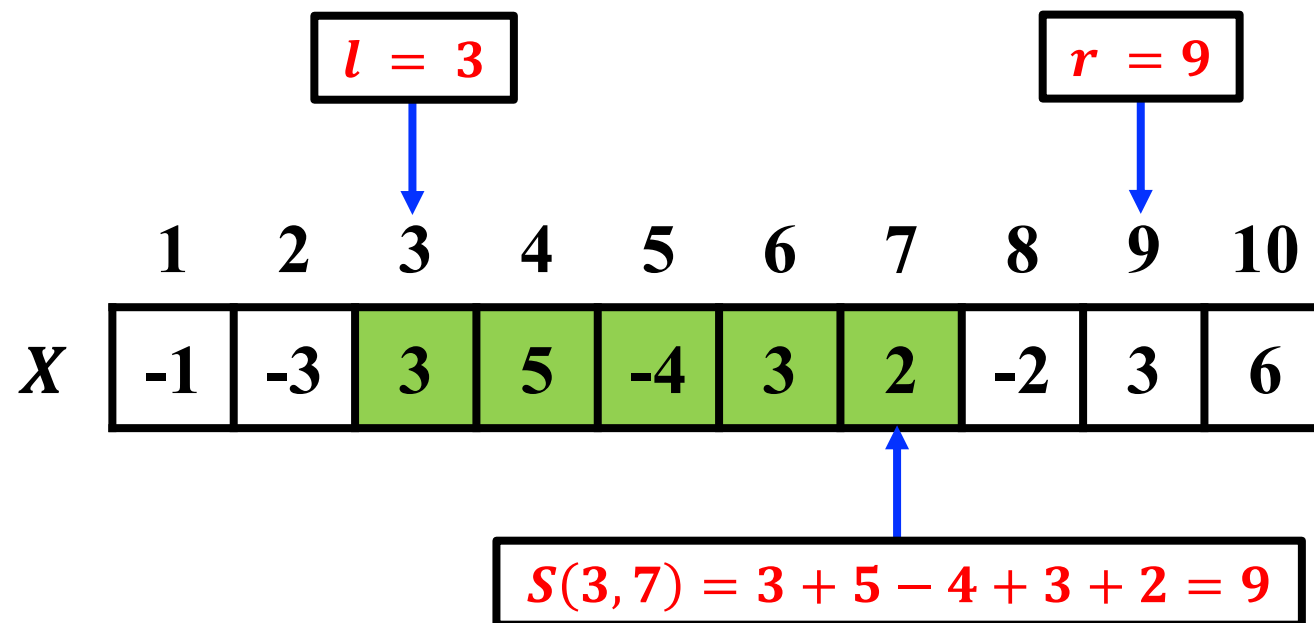
- $l = 3, r = 9$
- 计算 $S(3, 9)$:



蛮力枚举：算法实例



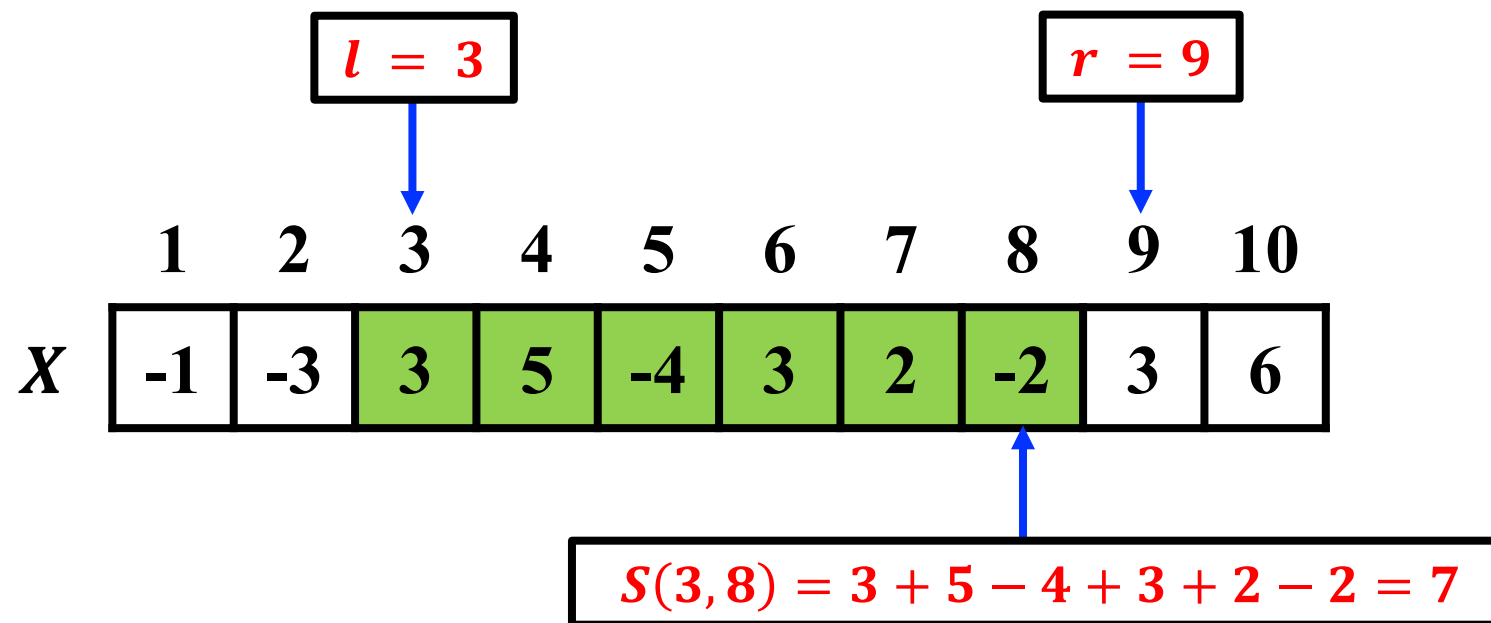
- $l = 3, r = 9$
- 计算 $S(3, 9)$:



蛮力枚举：算法实例



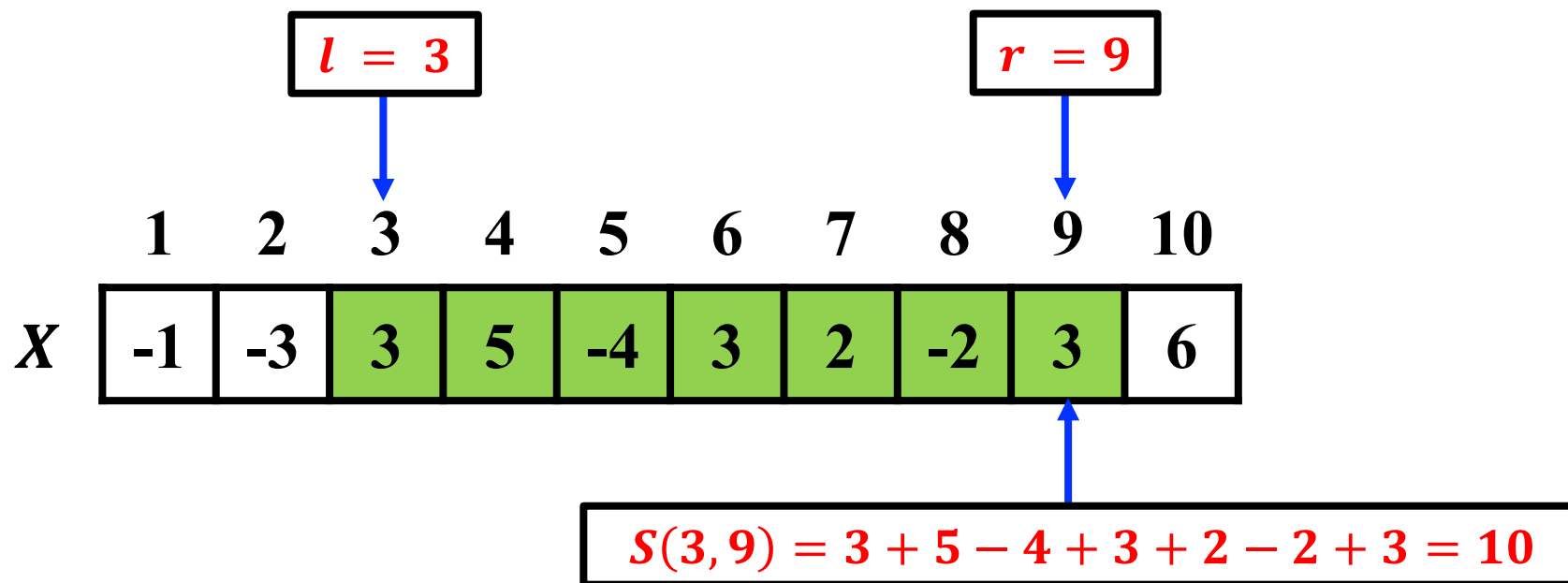
- $l = 3, r = 9$
- 计算 $S(3, 9)$:



蛮力枚举：算法实例



- $l = 3, r = 9$
- 计算 $S(3, 9)$:



蛮力枚举：伪代码



- 枚举 $n + C_n^2$ 种可能的区间 $[l, r]$ ($l \leq r$), 求解最大子数组之和 S_{max}

输入: 数组 $X[1..n]$

输出: 最大子数组之和 S_{max}

```
 $S_{max} \leftarrow -\infty$   
for  $l \leftarrow 1$  to  $n$  do  
    for  $r \leftarrow l$  to  $n$  do  
         $S(l, r) \leftarrow 0$   
        for  $i \leftarrow l$  to  $r$  do  
             $S(l, r) \leftarrow S(l, r) + X[i]$   
        end  
         $S_{max} \leftarrow \max\{S_{max}, S(l, r)\}$   
    end  
end  
return  $S_{max}$ 
```

枚举所有的起始下标 l

蛮力枚举：伪代码



- 枚举 $n + C_n^2$ 种可能的区间 $[l, r]$ ($l \leq r$), 求解最大子数组之和 S_{max}

输入: 数组 $X[1..n]$

输出: 最大子数组之和 S_{max}

$S_{max} \leftarrow -\infty$

for $l \leftarrow 1$ to n do

for $r \leftarrow l$ to n do

$S(l, r) \leftarrow 0$

 for $i \leftarrow l$ to r do

$S(l, r) \leftarrow S(l, r) + X[i]$

 end

$S_{max} \leftarrow \max\{S_{max}, S(l, r)\}$

end

end

return S_{max}

枚举所有的终止下标 r

蛮力枚举：伪代码

- 枚举 $n + C_n^2$ 种可能的区间 $[l, r]$ ($l \leq r$), 求解最大子数组之和 S_{max}

输入: 数组 $X[1..n]$

输出: 最大子数组之和 S_{max}

$S_{max} \leftarrow -\infty$

for $l \leftarrow 1$ to n do

 for $r \leftarrow l$ to n do

$S(l, r) \leftarrow 0$

 for $i \leftarrow l$ to r do

$S(l, r) \leftarrow S(l, r) + X[i]$

 end

$S_{max} \leftarrow \max\{S_{max}, S(l, r)\}$

 end

end

return S_{max}

迭代计算 $S(l, r)$ 并更新 S_{max}

- ```

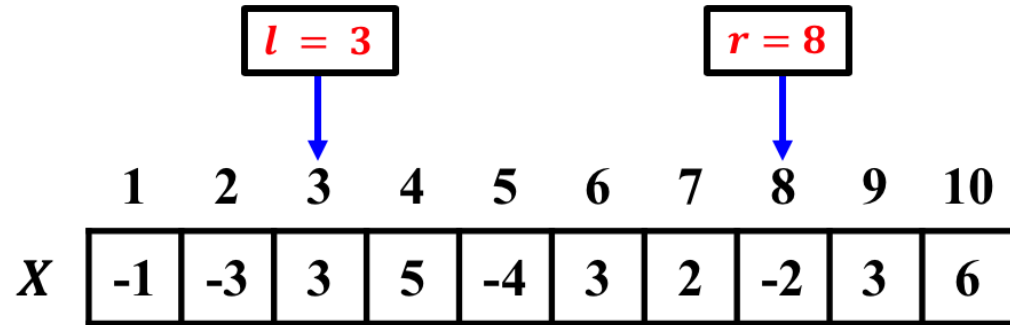
输入: 数组 $X[1..n]$
输出: 最大子数组之和 S_{max}
 $S_{max} \leftarrow -\infty$
for $l \leftarrow 1$ to n do
 | for $r \leftarrow l$ to n do
 | | $S(l, r) \leftarrow 0$
 | | for $i \leftarrow l$ to r do
 | | | $S(l, r) \leftarrow S(l, r) + X[i]$
 | | end
 | | $S_{max} \leftarrow \max\{S_{max}, S(l, r)\}$
 | end
end
return S_{max}

```

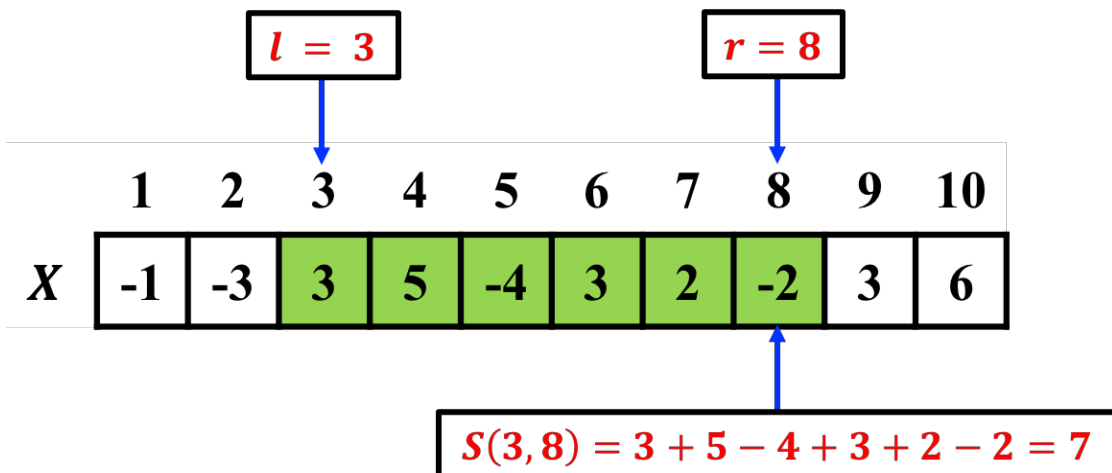
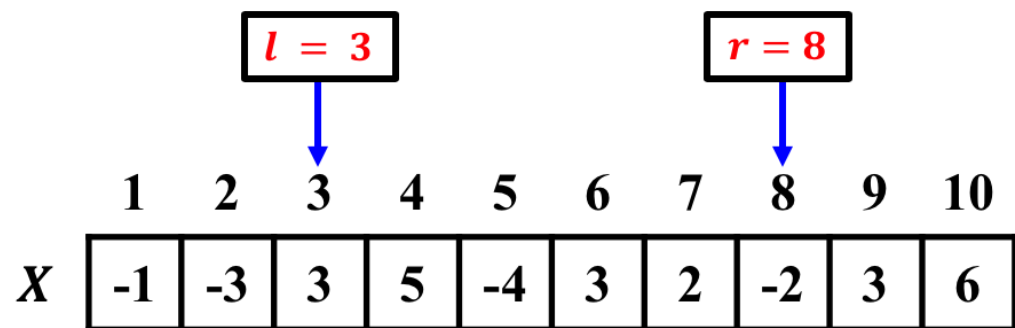
## 时间复杂度: $O(n^3)$

$$\left[ \left[ \left[ O(n) \right] O(n^2) \right] O(n^3) \right]$$

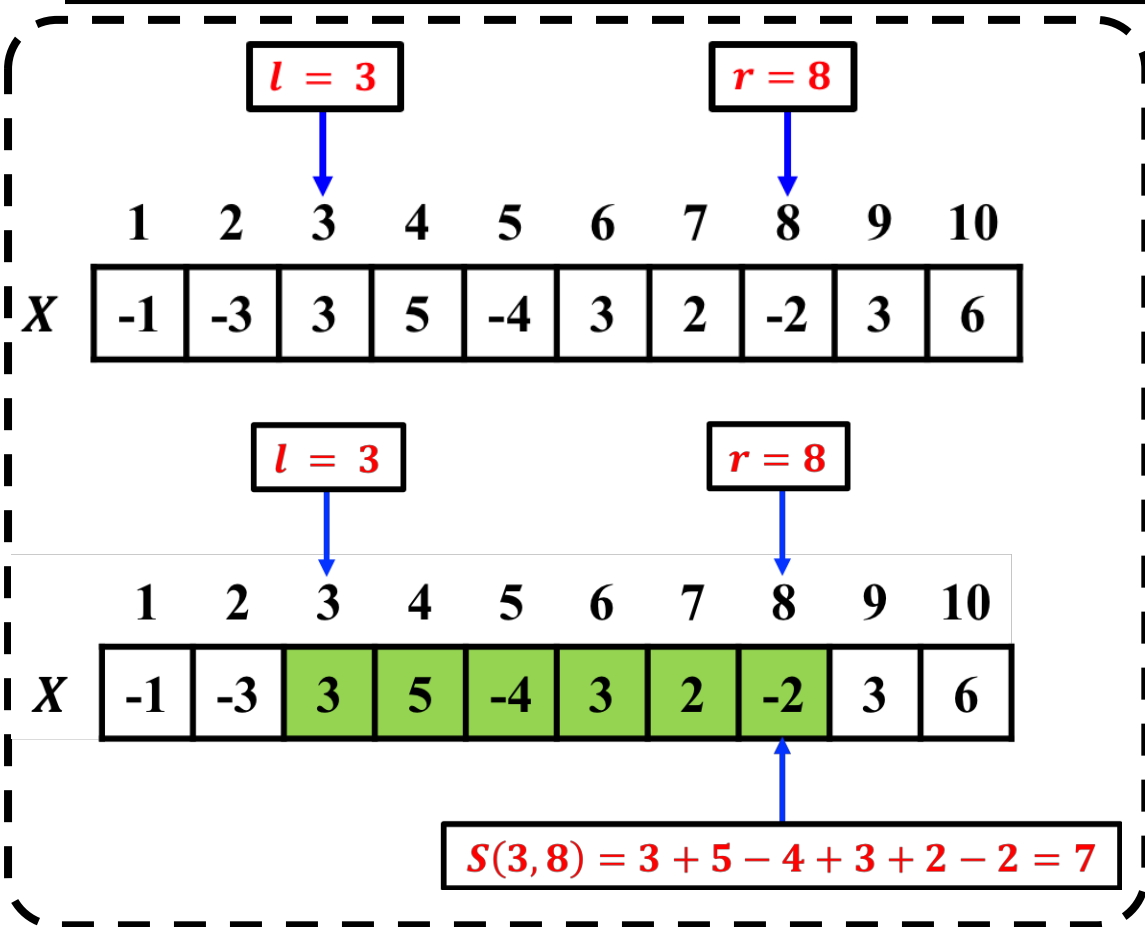
# 蛮力枚举：实例分析



# 蛮力枚举：实例分析

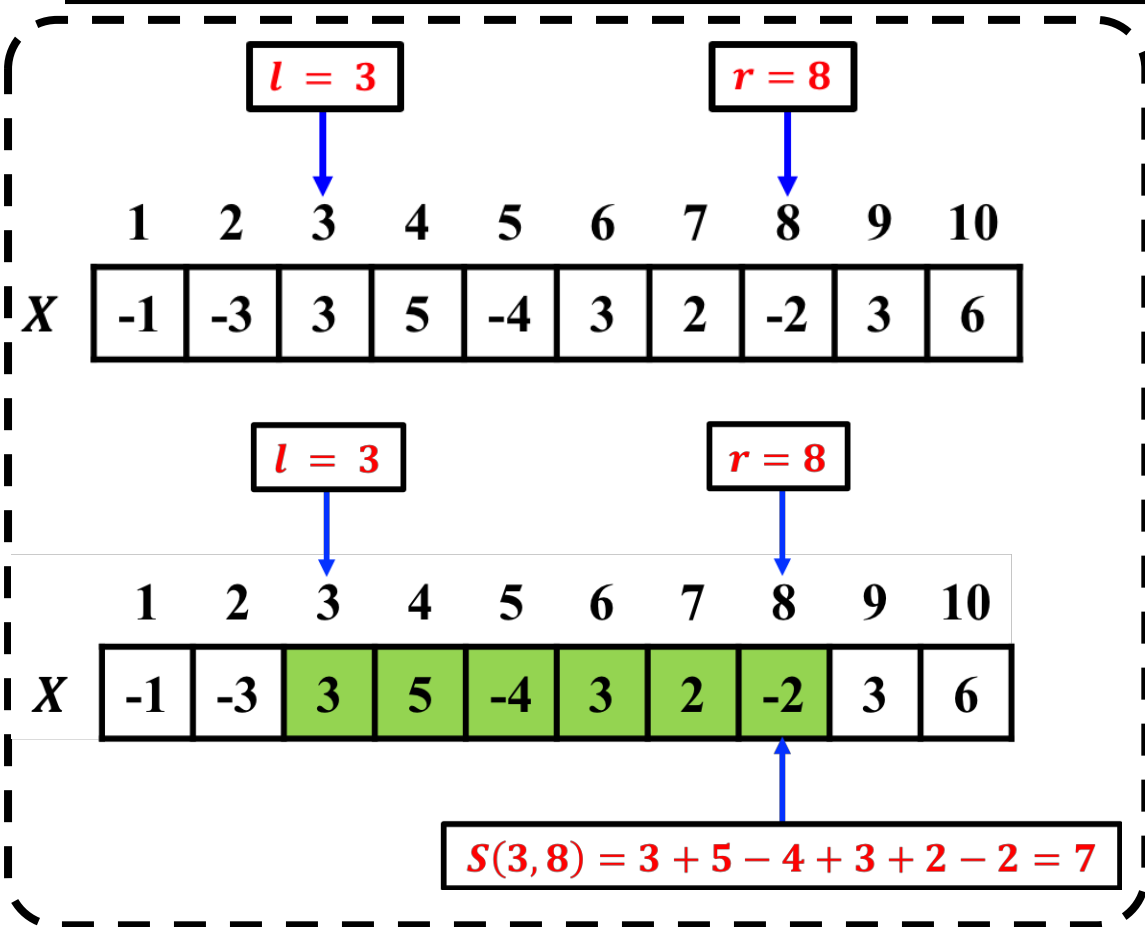


# 蛮力枚举：实例分析

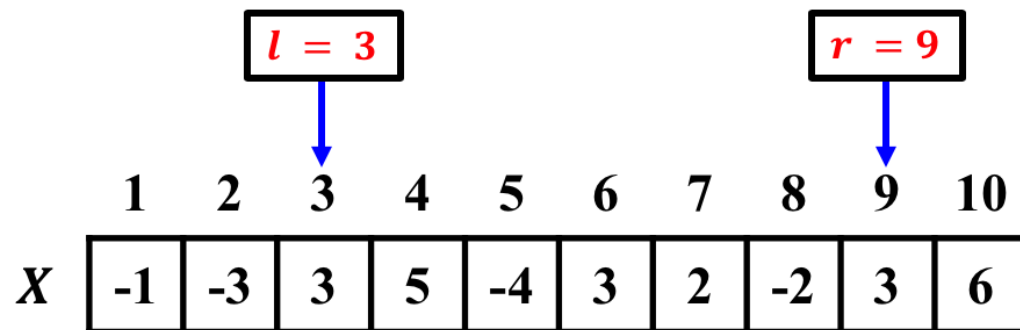


计算 $S(3, 8)$ 循环6次

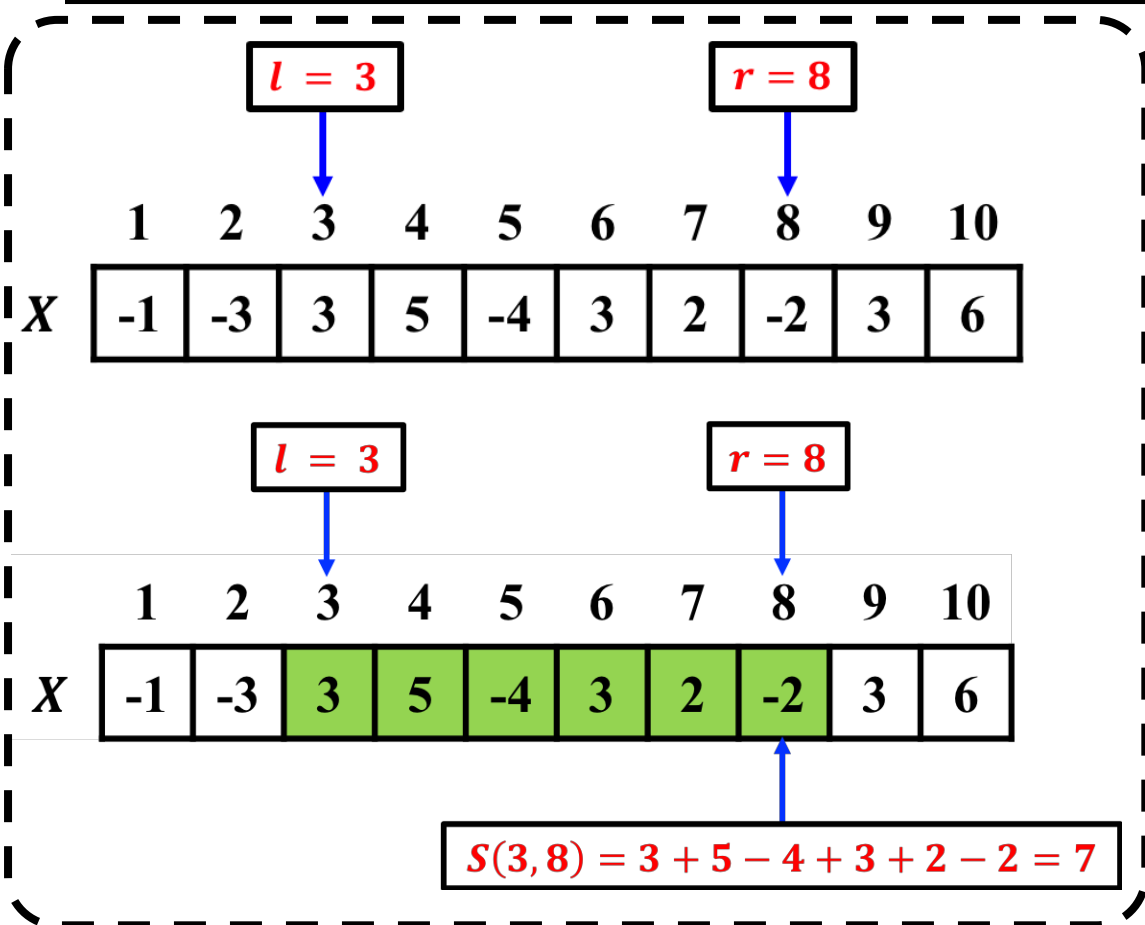
# 蛮力枚举：实例分析



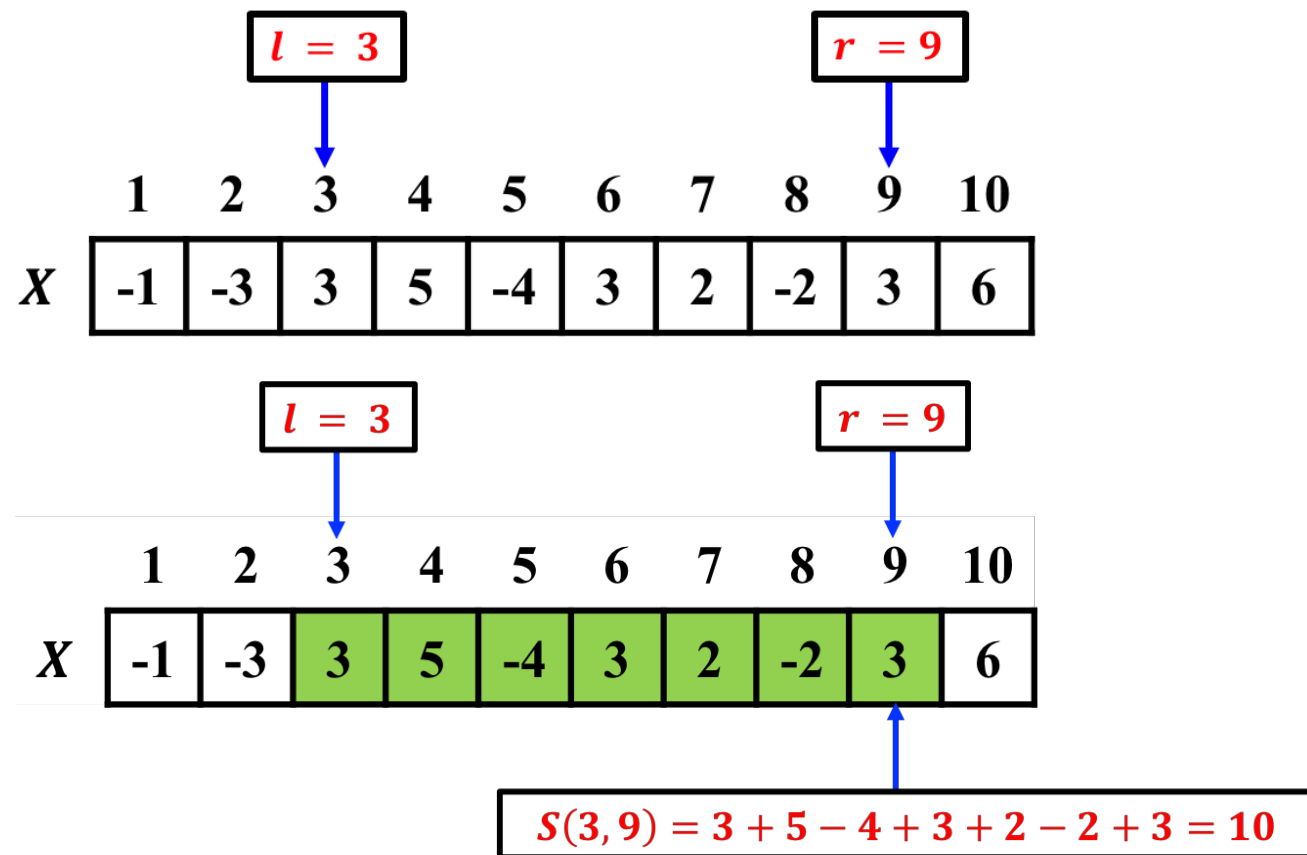
计算 $S(3, 8)$ 循环6次



# 蛮力枚举：实例分析

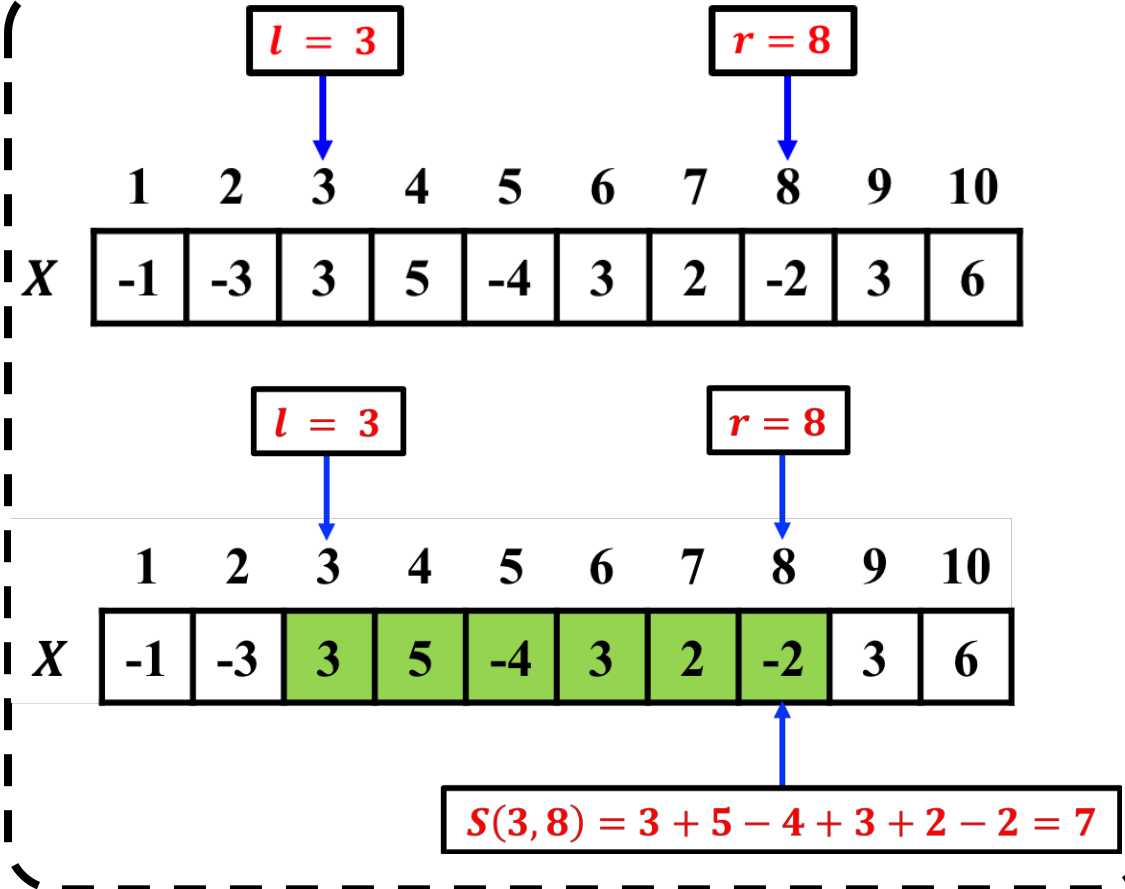


计算 $S(3, 8)$ 循环6次

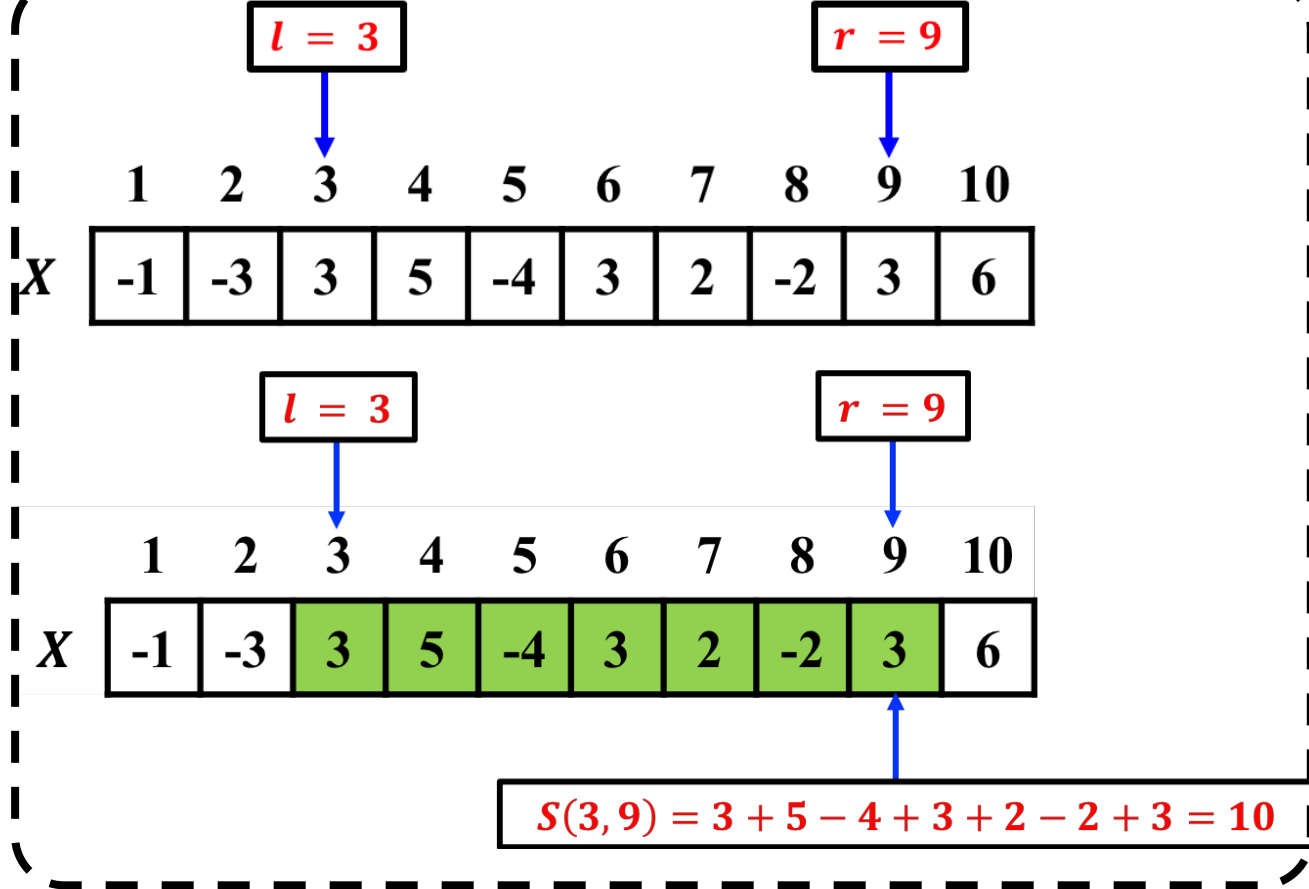




# 蛮力枚举：实例分析

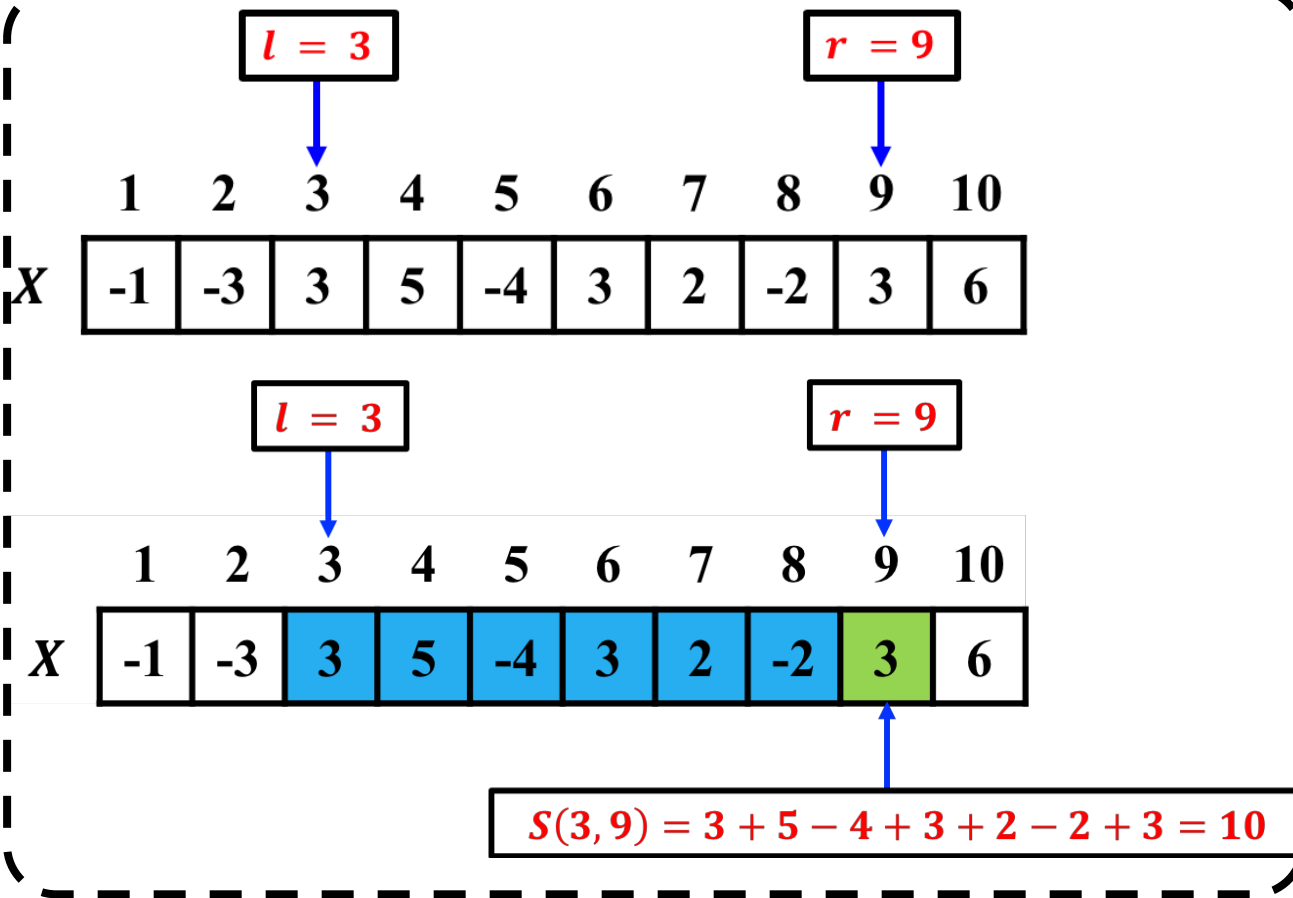
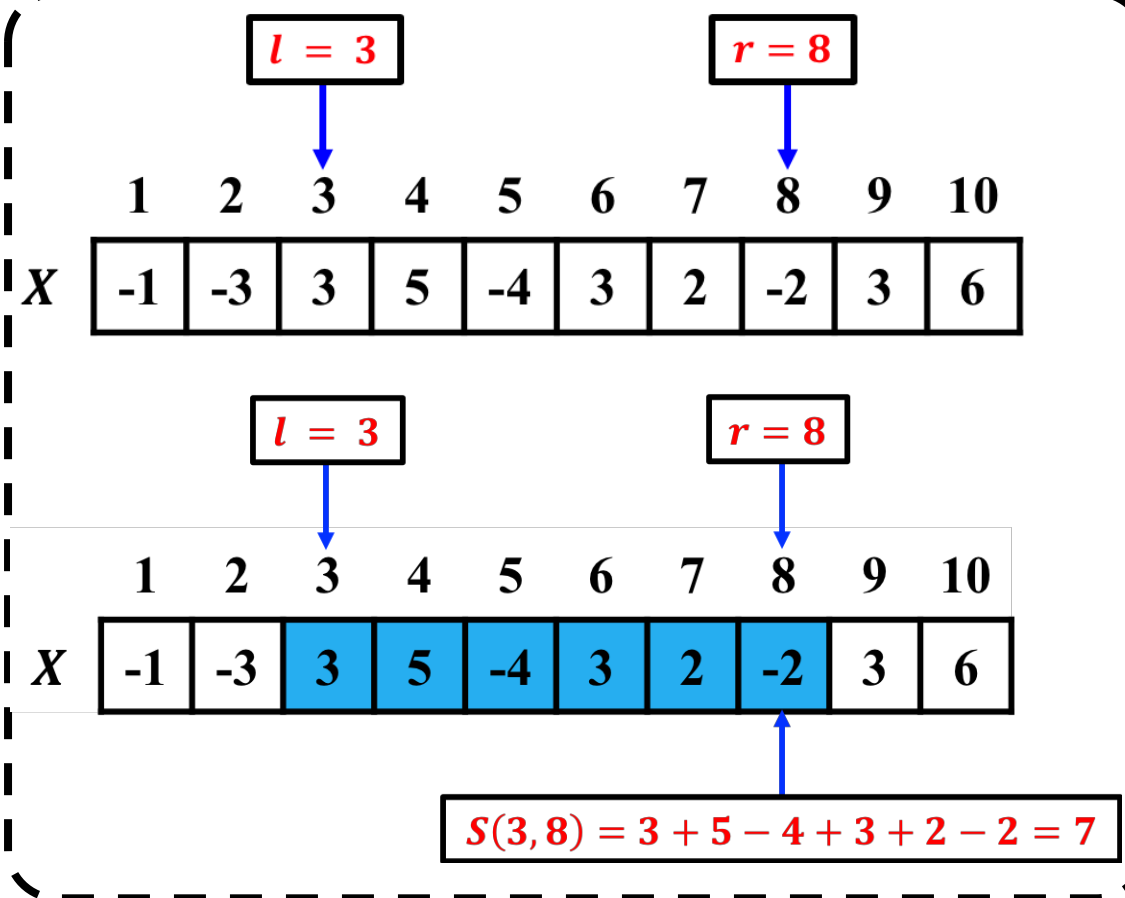


计算 $S(3, 8)$ 循环6次



计算 $S(3, 9)$ 循环7次

# 蛮力枚举：实例分析

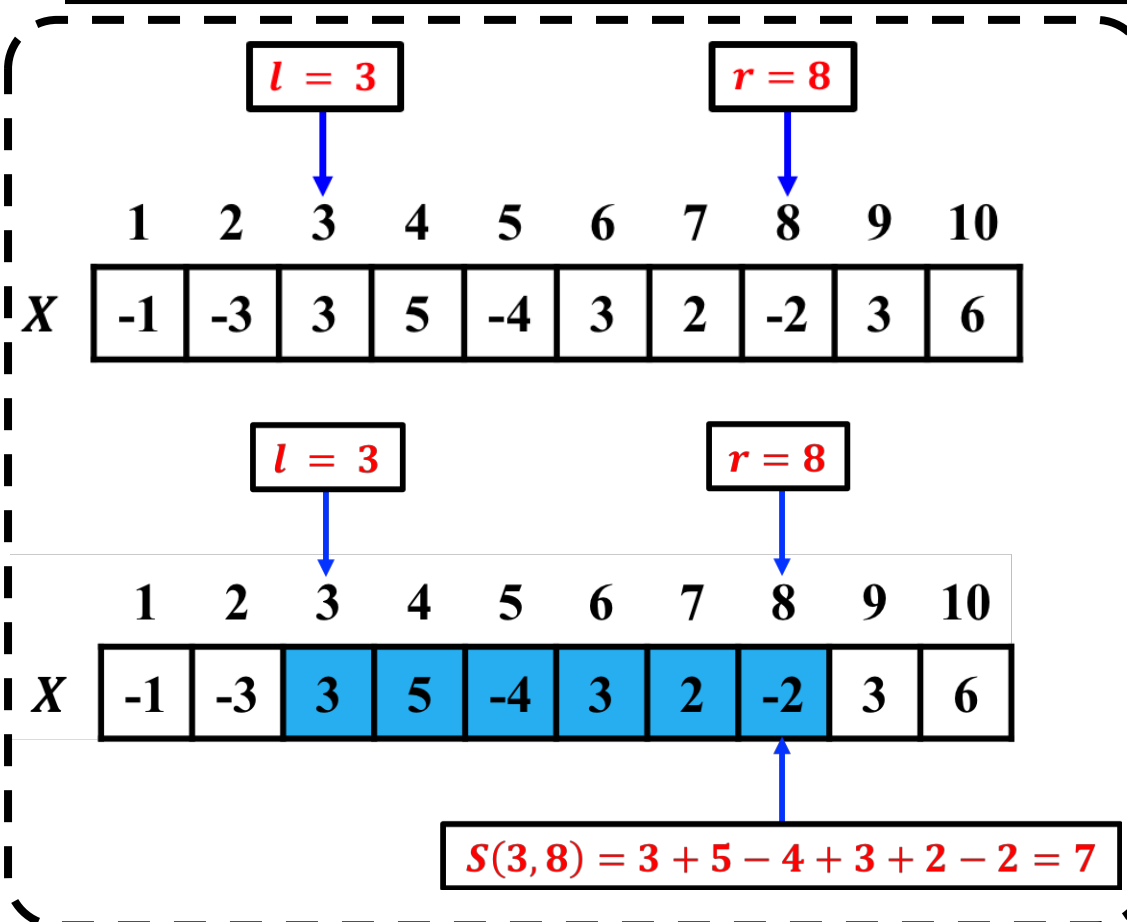


计算 $S(3, 8)$ 循环6次

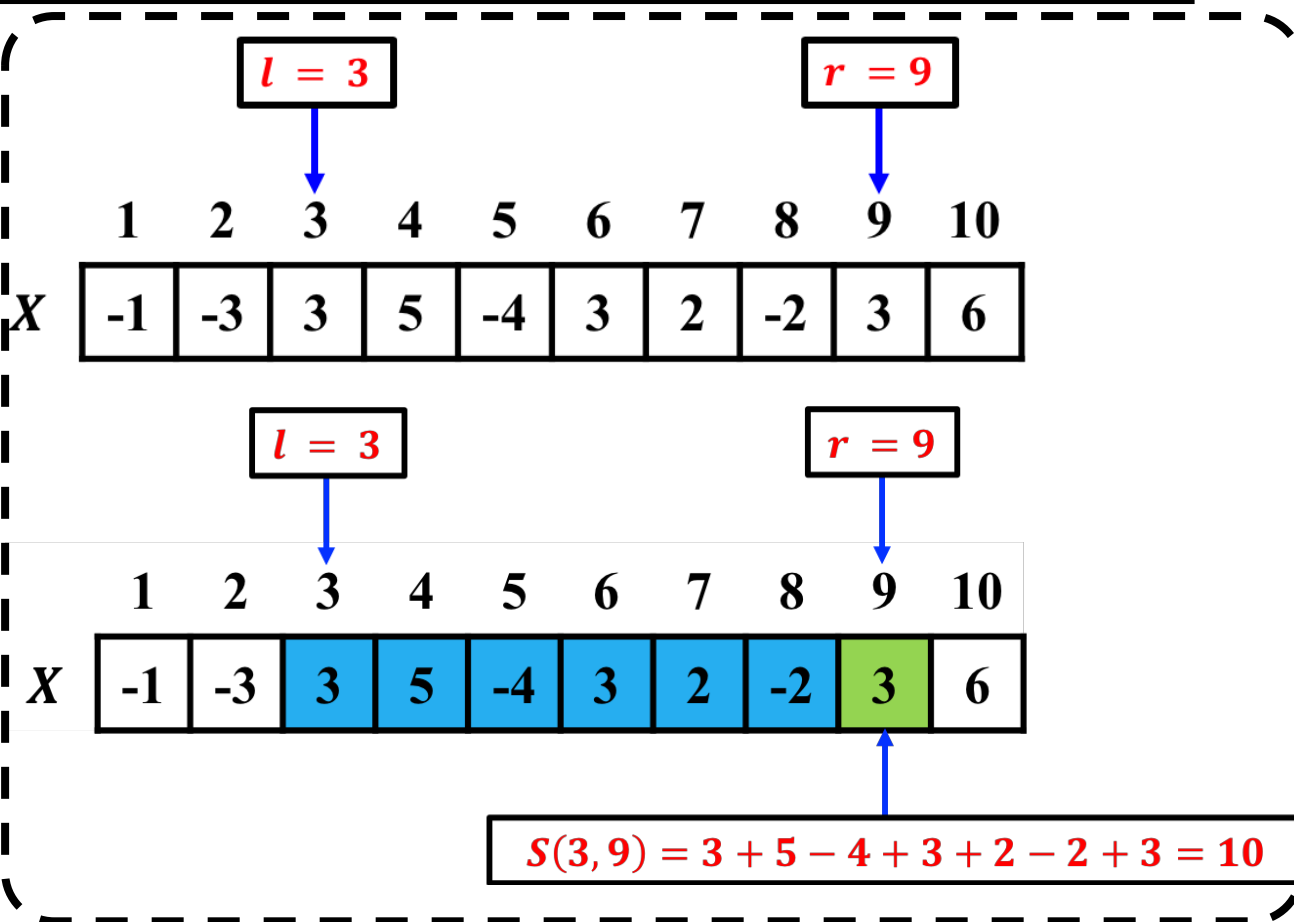
计算 $S(3, 9)$ 循环7次

存在重复计算

# 蛮力枚举：实例分析



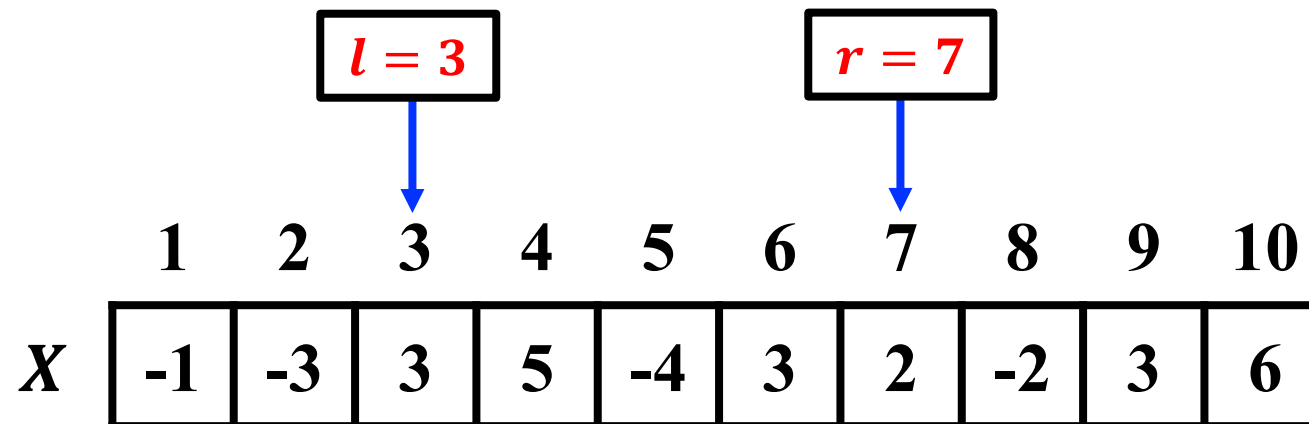
计算 $S(3, 8)$ 循环6次



计算 $S(3, 9)$ 循环7次

问题：如何减少重复计算？

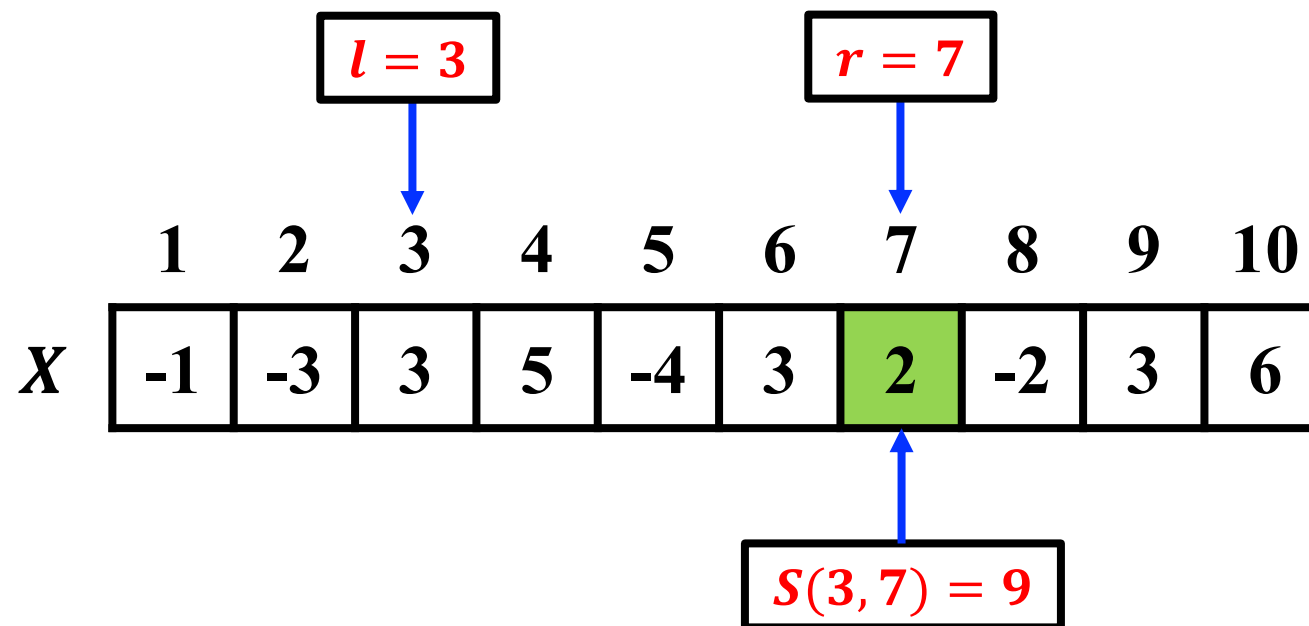
- $l = 3, r = 7$



# 优化枚举：算法实例



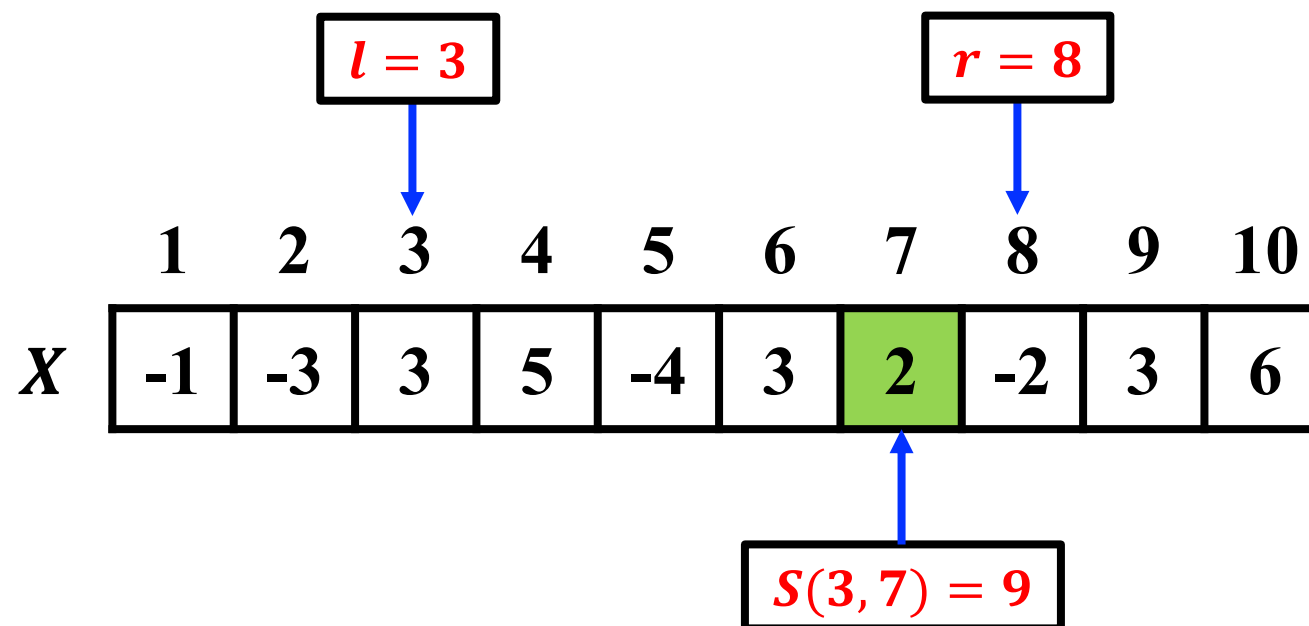
- $l = 3, r = 7, S(3, 7) = 9$



# 优化枚举：算法实例



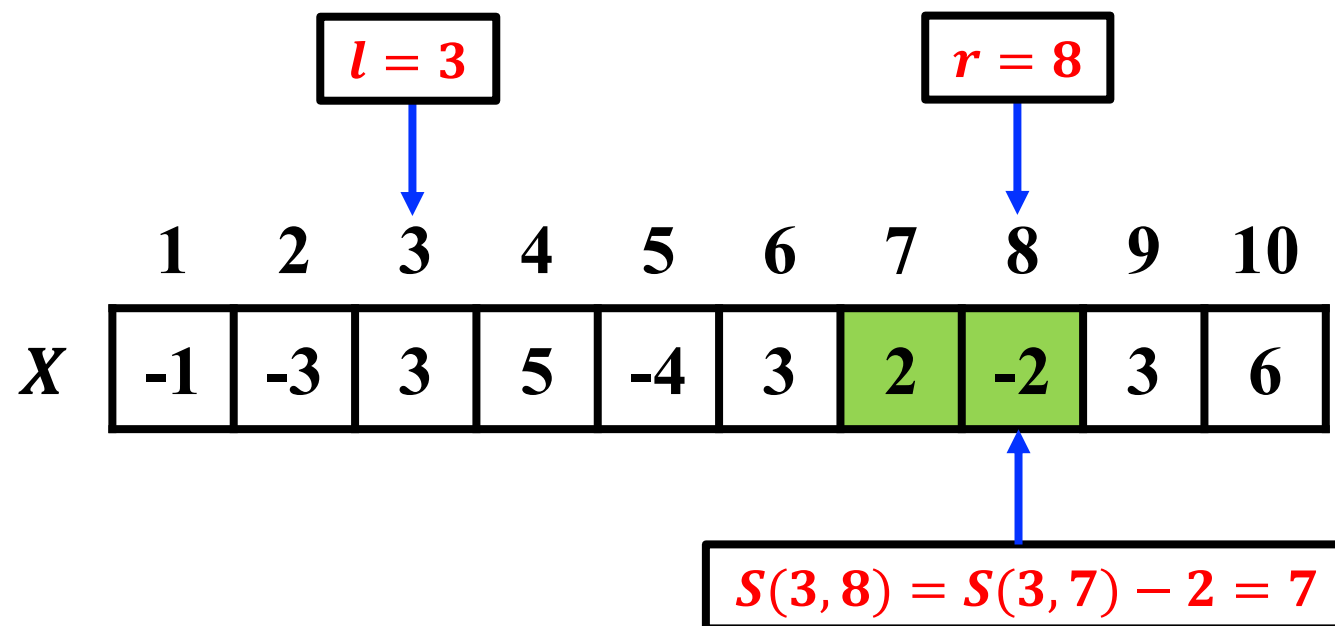
- $l = 3, r = 8, S(3, 8) = ?$



# 优化枚举：算法实例



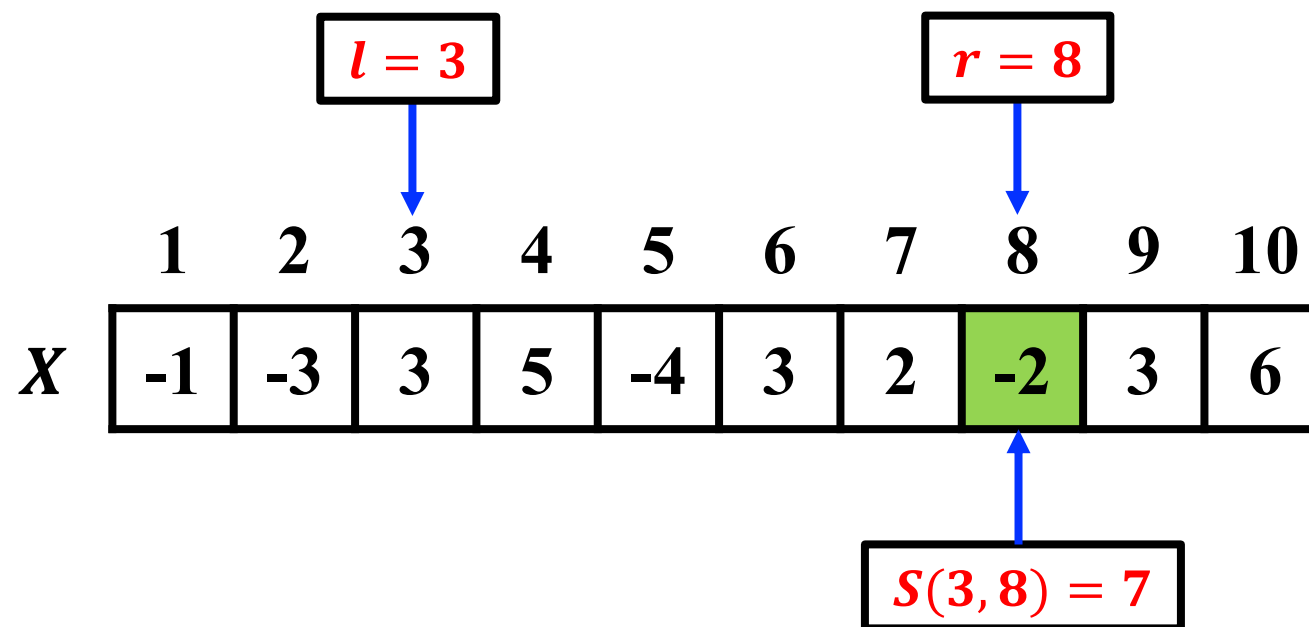
- $l = 3, r = 8, S(3, 8) = 7$



# 优化枚举：算法实例



- $l = 3, r = 8, S(3, 8) = 7$

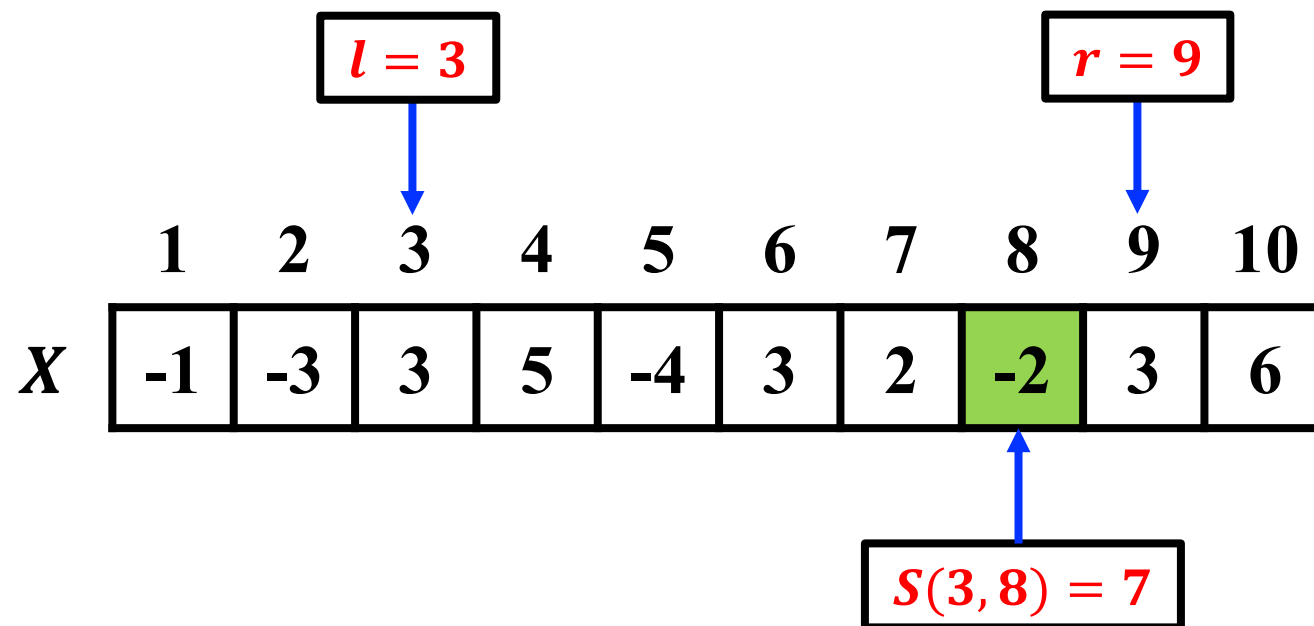




# 优化枚举：算法实例



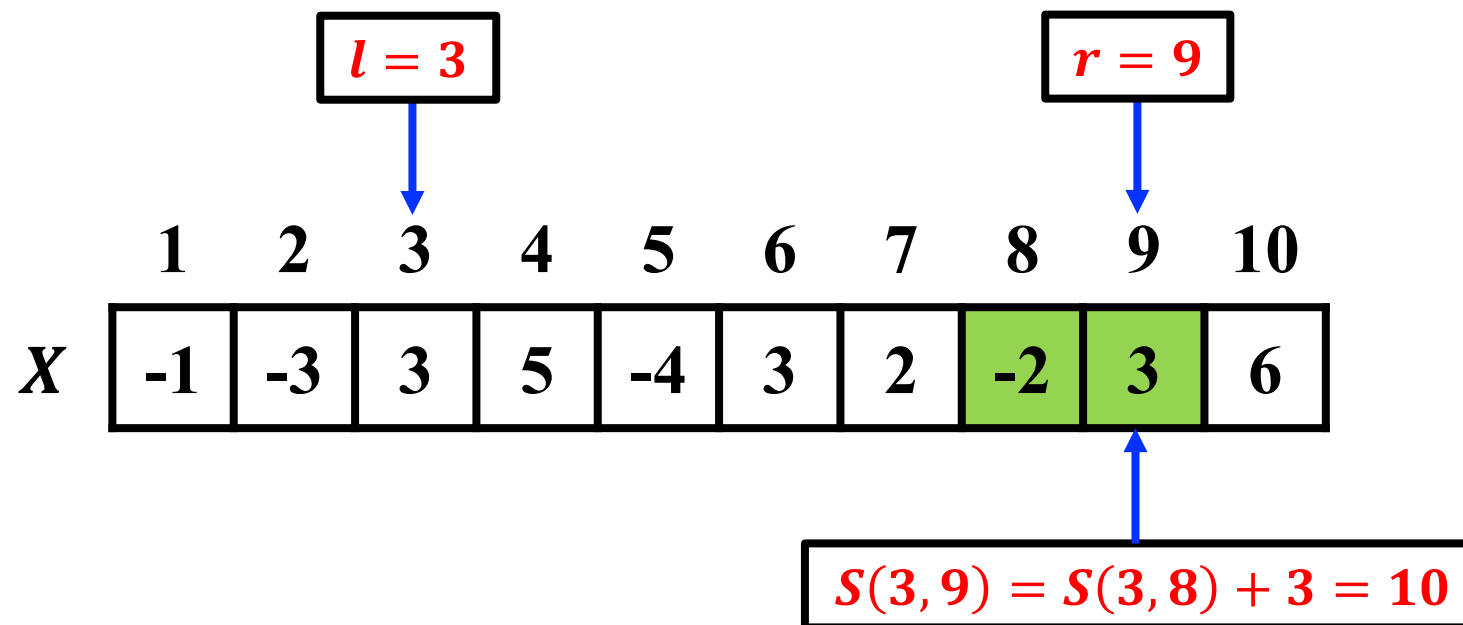
- $l = 3, r = 9, S(3, 9) = ?$



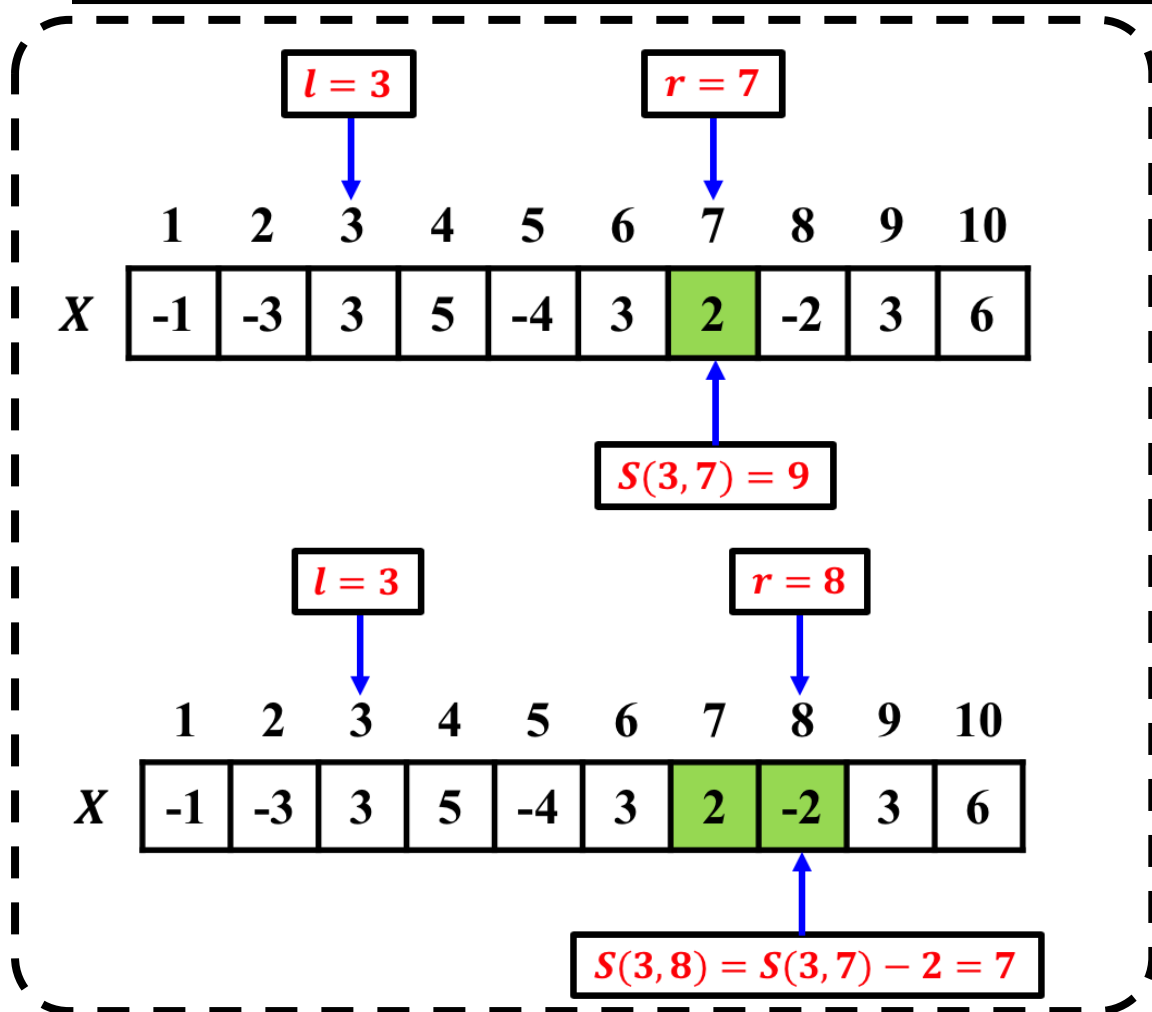
# 优化枚举：算法实例



- $l = 3, r = 9, S(3, 9) = ?$

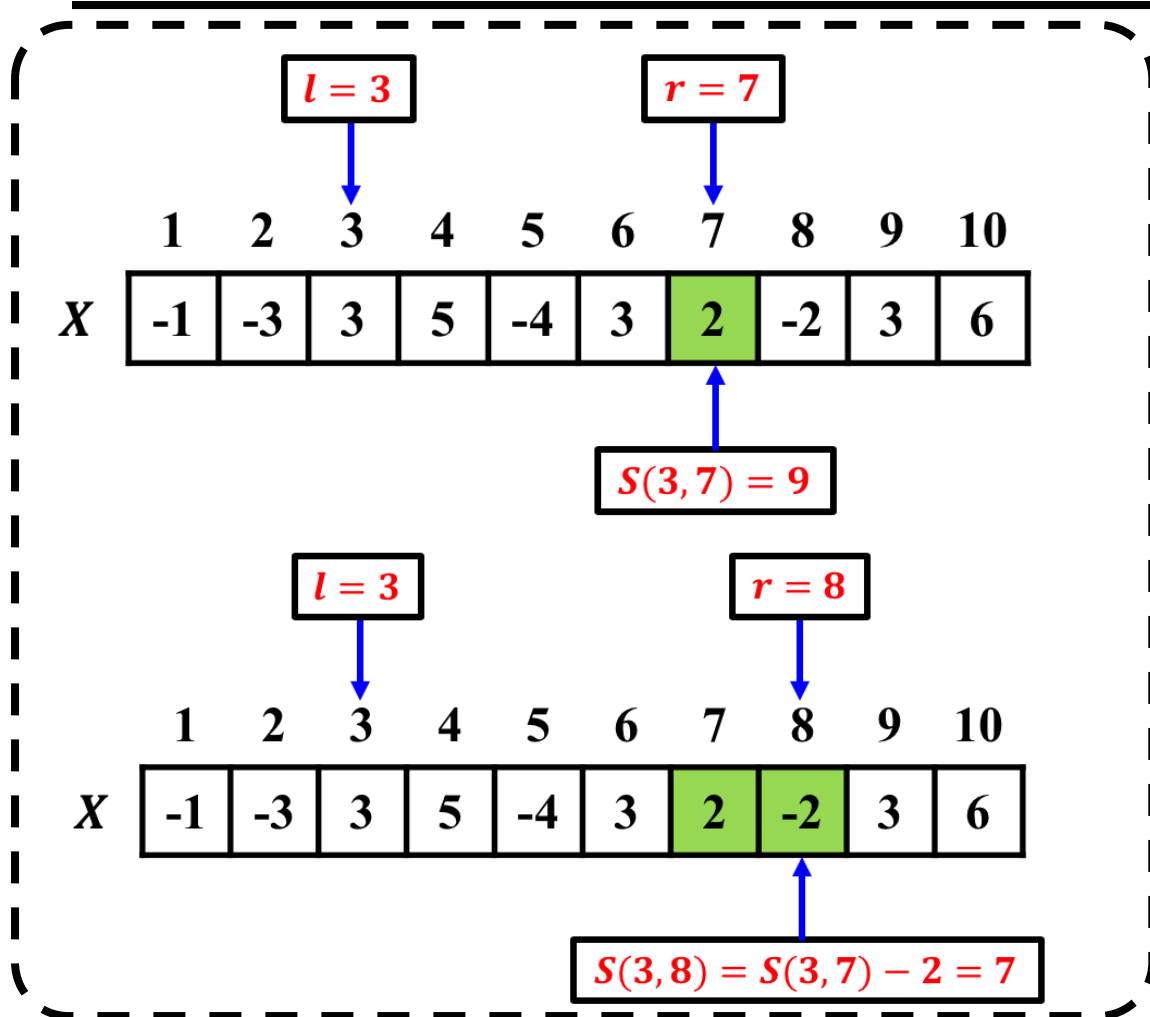


# 优化枚举：实例分析

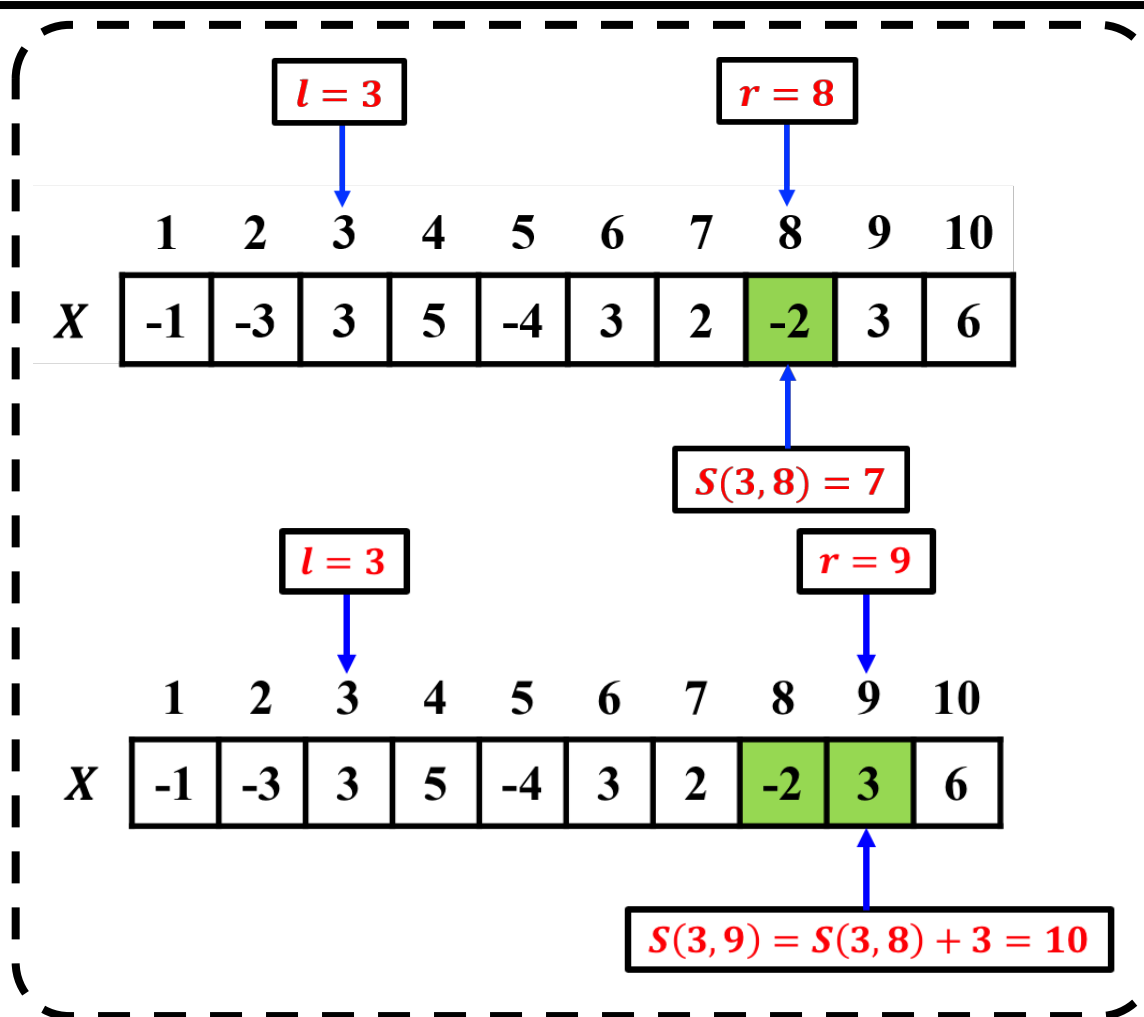


计算 $S(3, 8)$ 操作1次

# 优化枚举：实例分析

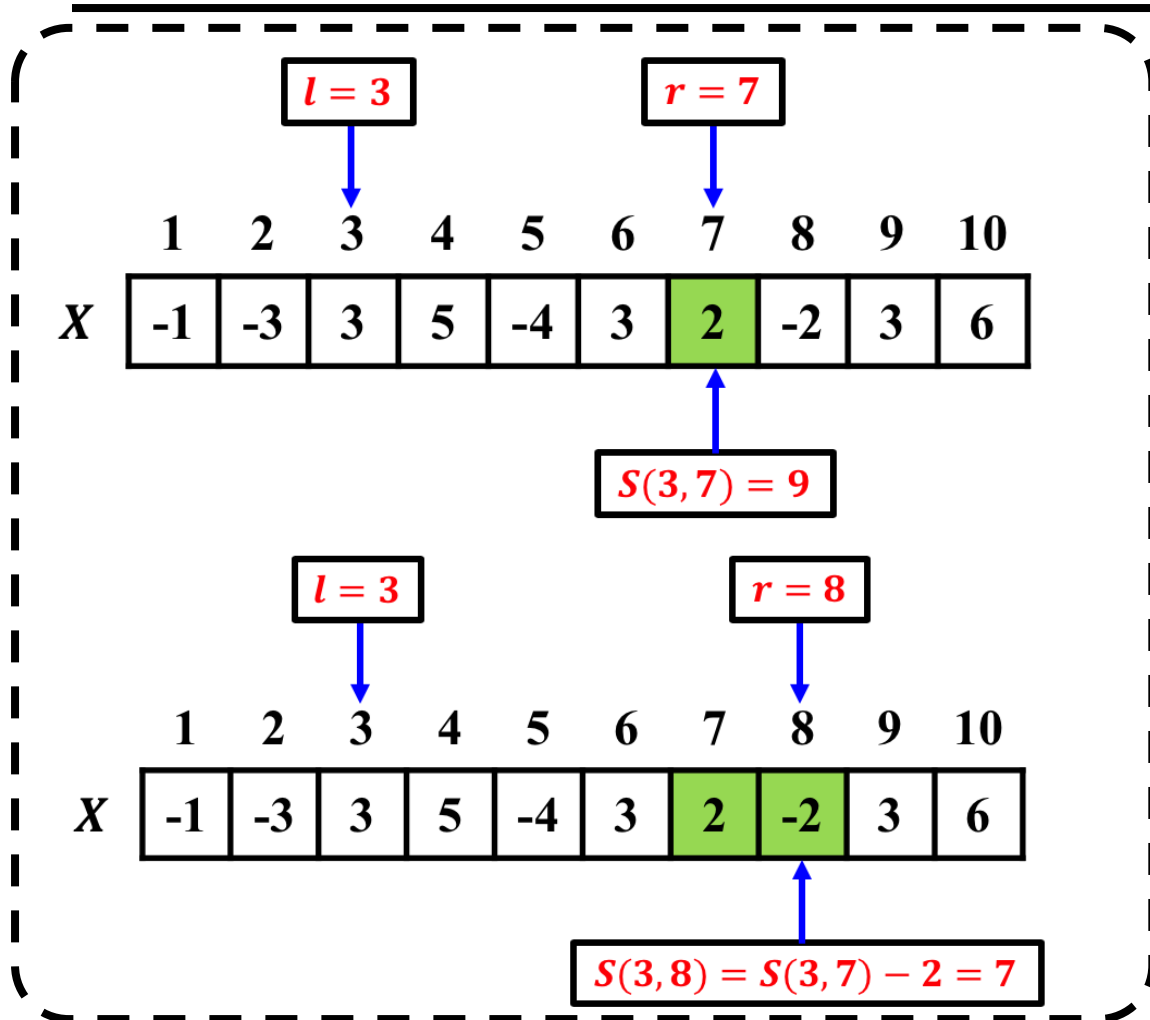


计算 $S(3, 8)$ 操作1次

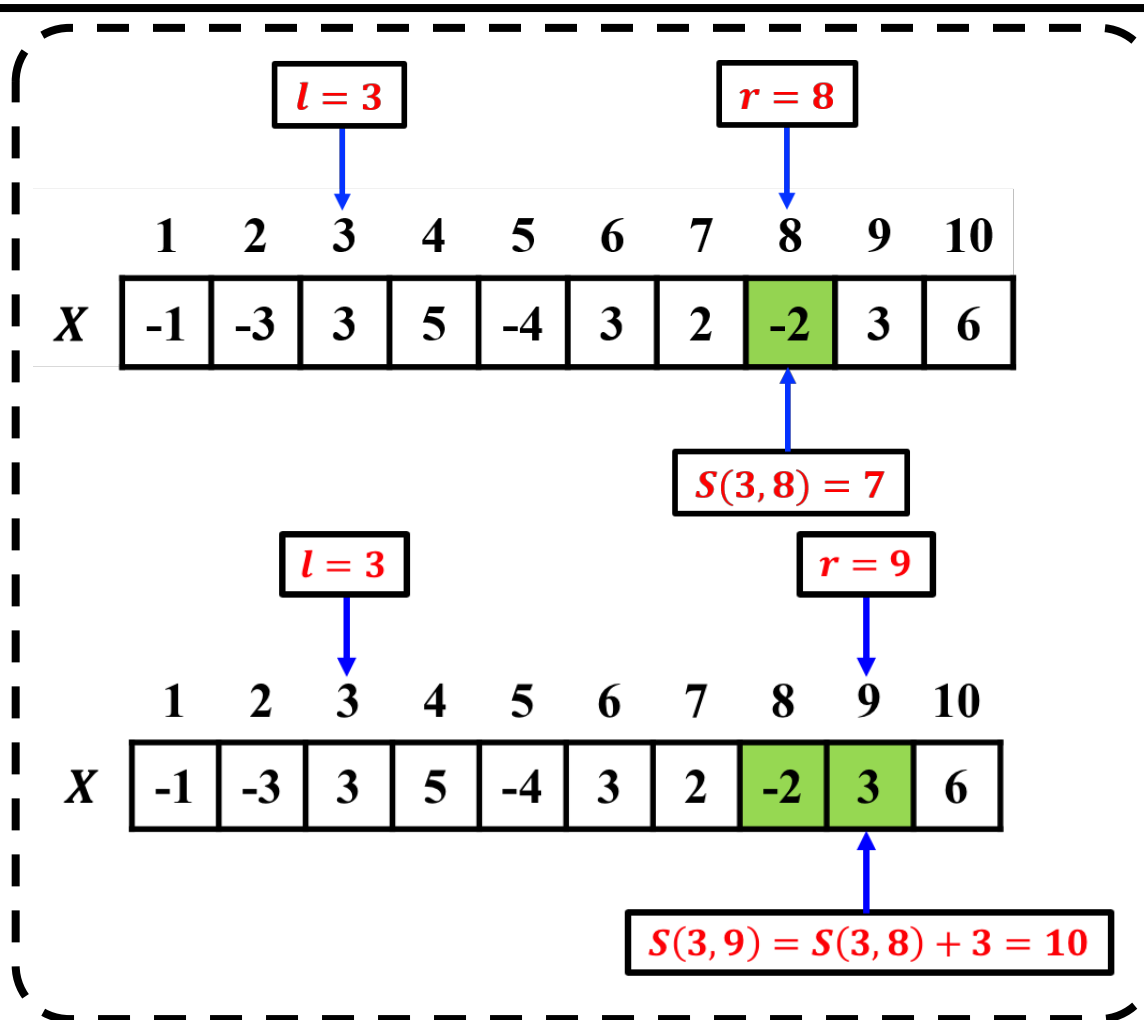


计算 $S(3, 9)$ 操作1次

# 优化枚举：实例分析



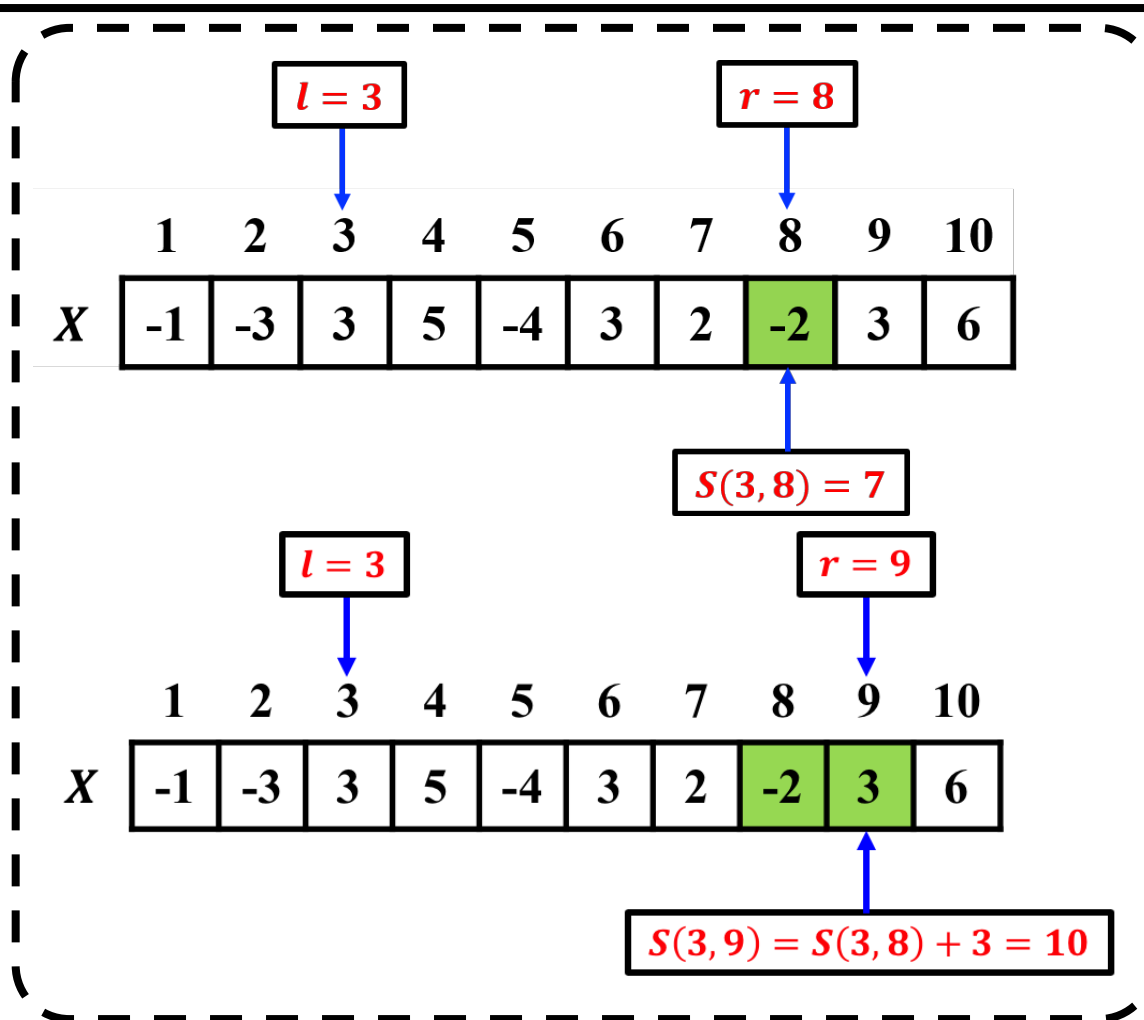
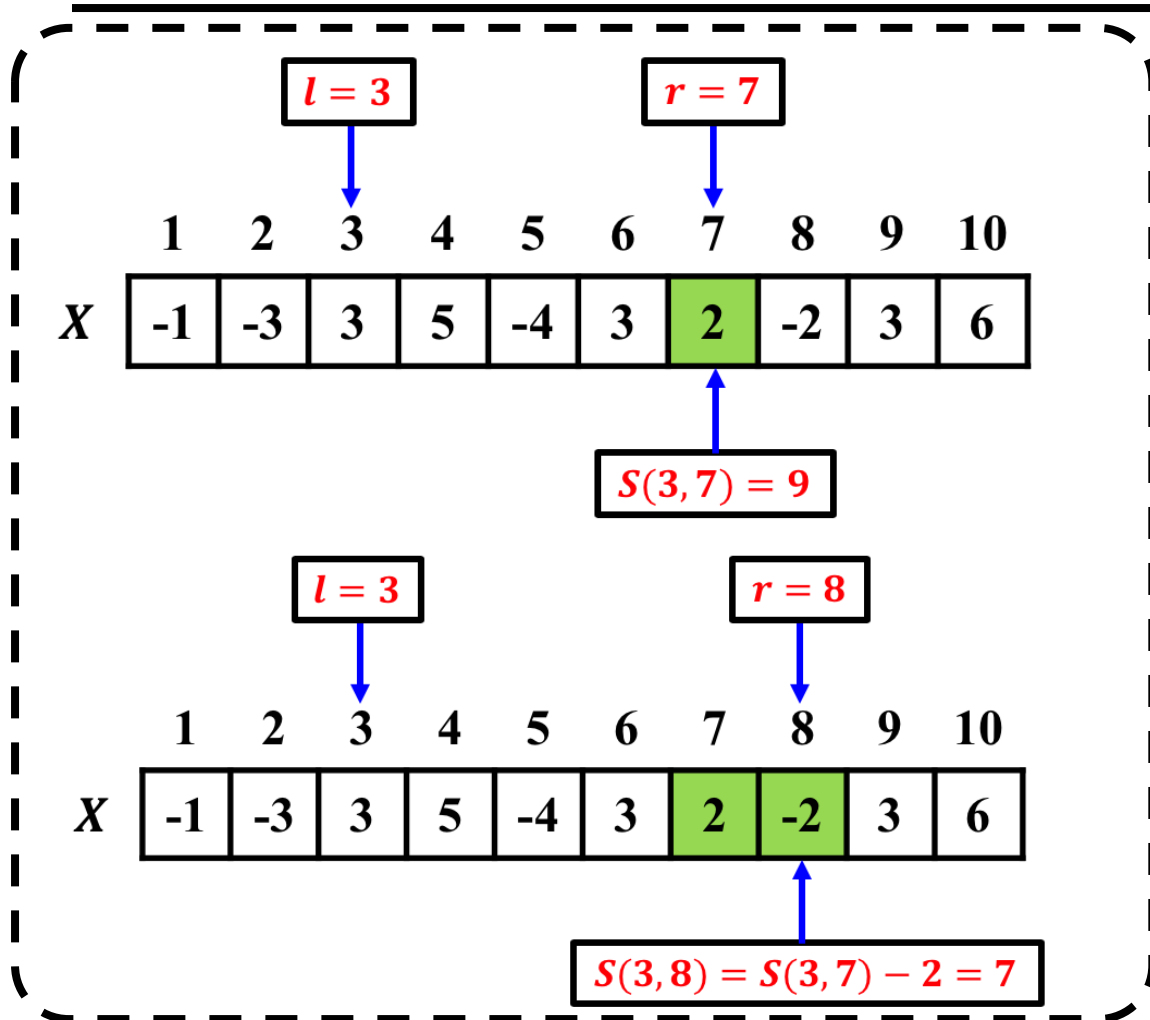
计算 $S(3, 8)$ 操作1次



计算 $S(3, 9)$ 操作1次

重复利用之前已经计算的数据

# 优化枚举：实例分析



$$S(l, r) = \sum_{i=l}^r X[i] = S(l, r-1) + X[r]$$



# 优化枚举：伪代码

- 核心思想:  $S(l, r) = \sum_{i=l}^r X[i] = S(l, r-1) + X[r]$

输入: 数组  $X[1..n]$

输出: 最大子数组之和  $S_{max}$

$S_{max} \leftarrow -\infty$

for  $l \leftarrow 1$  to  $n$  do

$S \leftarrow 0$

    for  $r \leftarrow l$  to  $n$  do

$S \leftarrow S + X[r]$

$S_{max} \leftarrow \max\{S_{max}, S\}$

    end

end

return  $S_{max}$

迭代求解 $S$

# 优化枚举：伪代码

- 核心思想:  $S(l, r) = \sum_{i=l}^r X[i] = S(l, r-1) + X[r]$

输入: 数组  $X[1..n]$

输出: 最大子数组之和  $S_{max}$

$S_{max} \leftarrow -\infty$

for  $l \leftarrow 1$  to  $n$  do

$S \leftarrow 0$

    for  $r \leftarrow l$  to  $n$  do

$S \leftarrow S + X[r]$

$S_{max} \leftarrow \max\{S_{max}, S\}$

    end

end

return  $S_{max}$

更新  $S_{max}$





# 优化枚举：伪代码

- 核心思想:  $S(l, r) = \sum_{i=l}^r X[i] = S(l, r-1) + X[r]$

输入: 数组  $X[1..n]$

输出: 最大子数组之和  $S_{max}$

$S_{max} \leftarrow -\infty$

for  $l \leftarrow 1$  to  $n$  do

$S \leftarrow 0$

    for  $r \leftarrow l$  to  $n$  do

$S \leftarrow S + X[r]$

$S_{max} \leftarrow \max\{S_{max}, S\}$

    end

end

return  $S_{max}$

时间复杂度:  $O(n^2)$

$O(n)$   $O(n^2)$

# 分而治之：一般步骤



**分解**原问题

原问题分解成多个子问题



**解决**子问题

递归地求解各个子问题



**合并**问题解

将结果合并为原问题解

# 分而治之：一般步骤



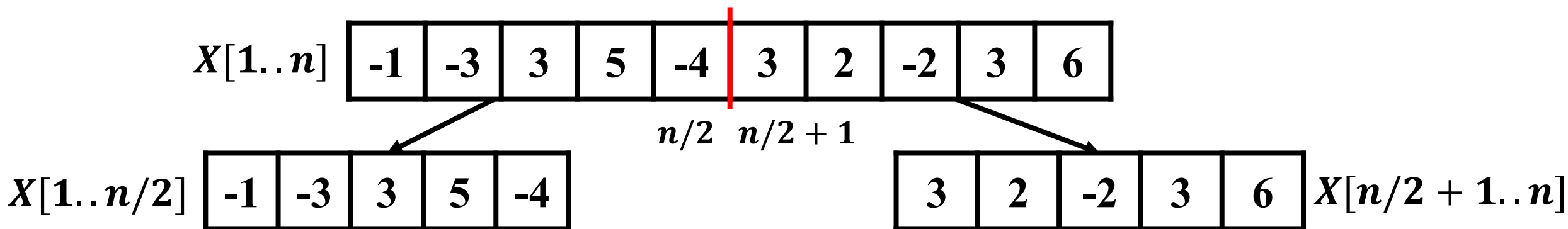
|           |    |    |   |   |    |   |   |    |   |   |
|-----------|----|----|---|---|----|---|---|----|---|---|
| $X[1..n]$ | -1 | -3 | 3 | 5 | -4 | 3 | 2 | -2 | 3 | 6 |
|-----------|----|----|---|---|----|---|---|----|---|---|

分解原问题

解决子问题

合并问题解

# 分而治之：一般步骤



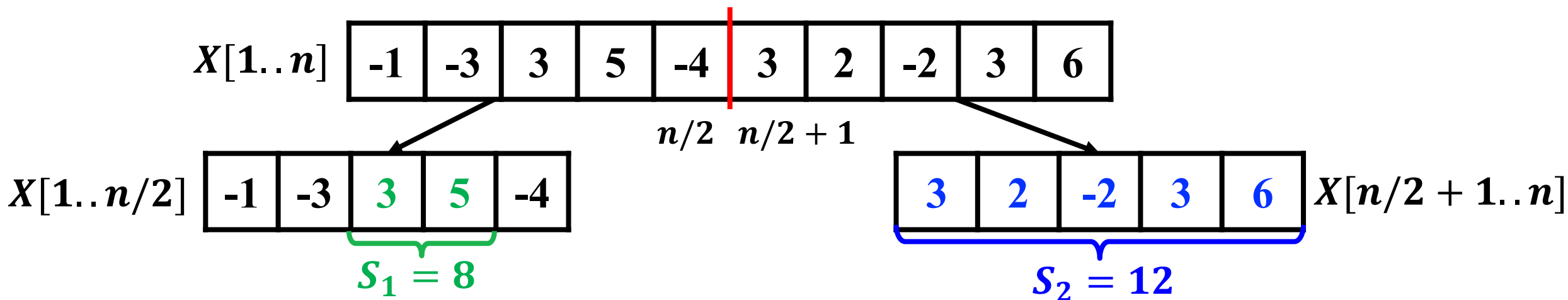
- 将数组  $X[1..n]$  分为  $X[1..n/2]$  和  $X[n/2 + 1..n]$

分解原问题

解决子问题

合并问题解

# 分而治之：一般步骤



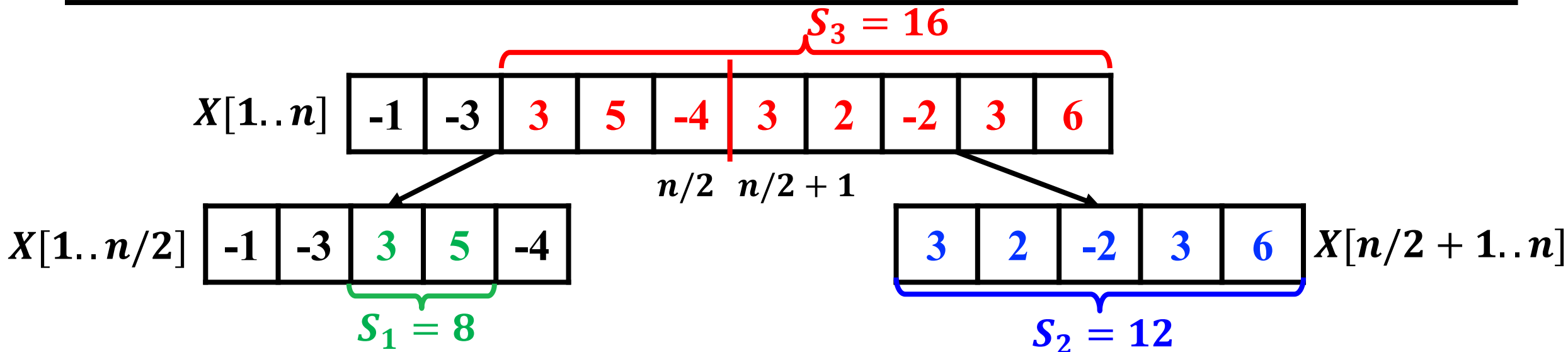
- 将数组  $X[1..n]$  分为  $X[1..n/2]$  和  $X[n/2 + 1..n]$
- 递归求解子问题
  - $S_1$ : 数组  $X[1..n/2]$  的最大子数组
  - $S_2$ : 数组  $X[n/2 + 1..n]$  的最大子数组

分解原问题

解决子问题

合并问题解

# 分而治之：一般步骤



- 将数组  $X[1..n]$  分为  $X[1..n/2]$  和  $X[n/2 + 1..n]$

分解原问题

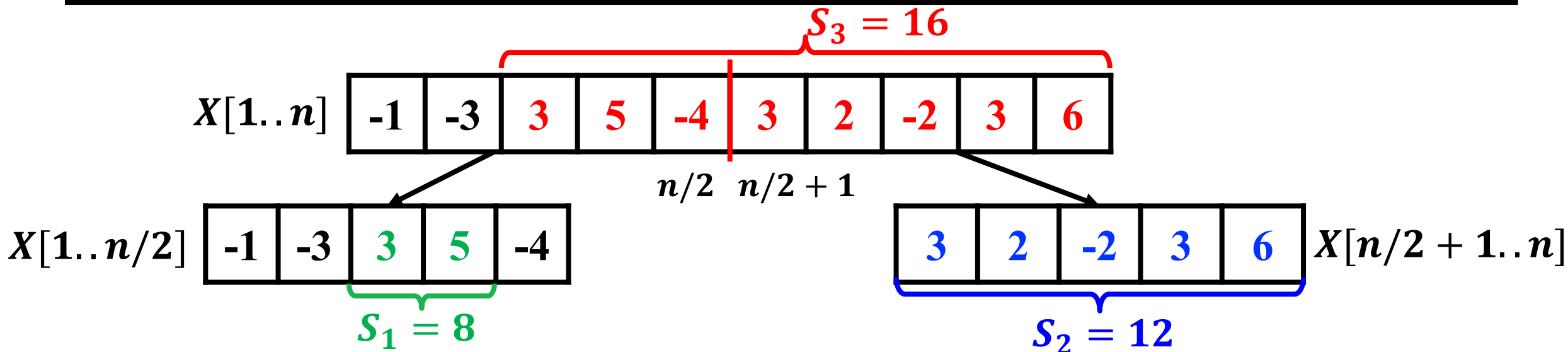
- 递归求解子问题

解决子问题

- $S_1$ : 数组  $X[1..n/2]$  的最大子数组
- $S_2$ : 数组  $X[n/2 + 1..n]$  的最大子数组
- 合并子问题, 得到  $S_{max}$ 
  - $S_3$ : 跨中点的最大子数组

合并问题解

# 分而治之：一般步骤



- 将数组  $X[1..n]$  分为  $X[1..n/2]$  和  $X[n/2 + 1..n]$

分解原问题

- 递归求解子问题

解决子问题

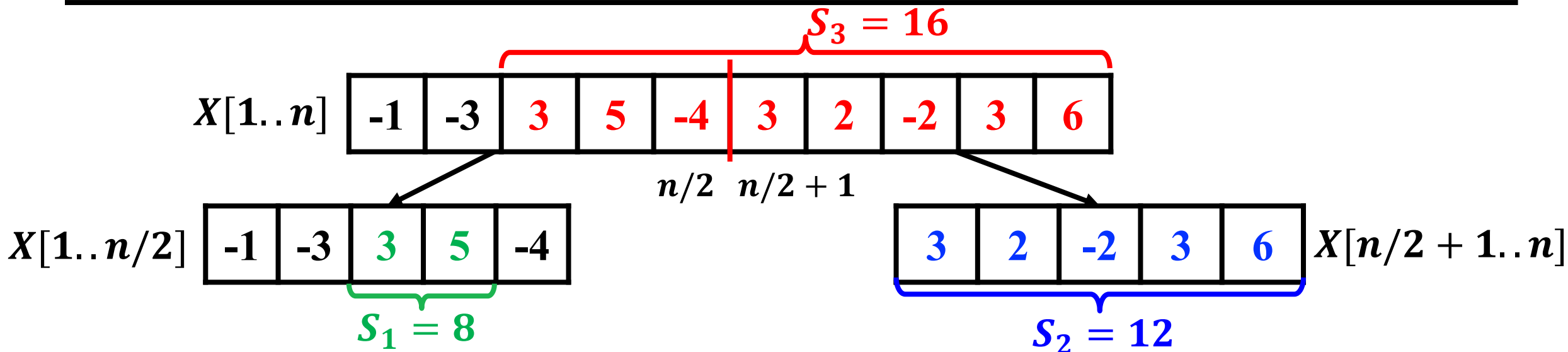
- $S_1$ : 数组  $X[1..n/2]$  的最大子数组
- $S_2$ : 数组  $X[n/2 + 1..n]$  的最大子数组

- 合并子问题，得到  $S_{max}$

合并问题解

- $S_3$ : 跨中点的最大子数组
- 数组  $X$  的最大子数组之和  $S_{max} = \max\{S_1, S_2, S_3\}$

# 分而治之：一般步骤



- 将数组  $X[1..n]$  分为  $X[1..n/2]$  和  $X[n/2 + 1..n]$

分解原问题

- 递归求解子问题

解决子问题

- $S_1$ : 数组  $X[1..n/2]$  的最大子数组
  - $S_2$ : 数组  $X[n/2 + 1..n]$  的最大子数组

可递归求解

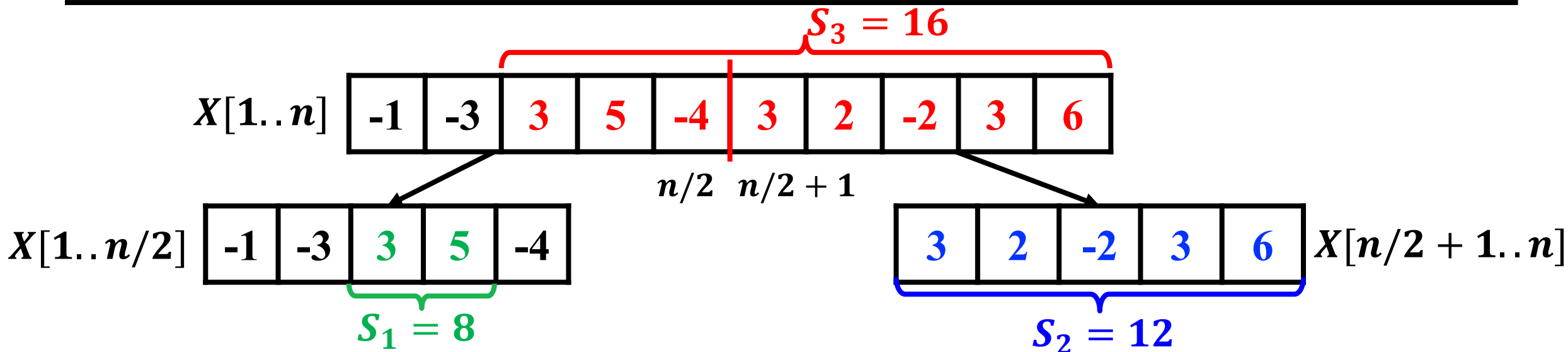
- 合并子问题，得到  $S_{max}$

合并问题解

- $S_3$ : 跨中点的最大子数组
  - 数组  $X$  的最大子数组之和  $S_{max} = \max\{S_1, S_2, S_3\}$



# 分而治之：一般步骤



- 将数组  $X[1..n]$  分为  $X[1..n/2]$  和  $X[n/2 + 1..n]$

分解原问题

- 递归求解子问题

解决子问题

- $S_1$ : 数组  $X[1..n/2]$  的最大子数组
- $S_2$ : 数组  $X[n/2 + 1..n]$  的最大子数组

可递归求解

- 合并子问题，得到  $S_{max}$

问题：如何求解  $S_3$ ?

合并问题解

- $S_3$ : 跨中点的最大子数组

- 数组  $X$  的最大子数组之和  $S_{max} = \max\{S_1, S_2, S_3\}$

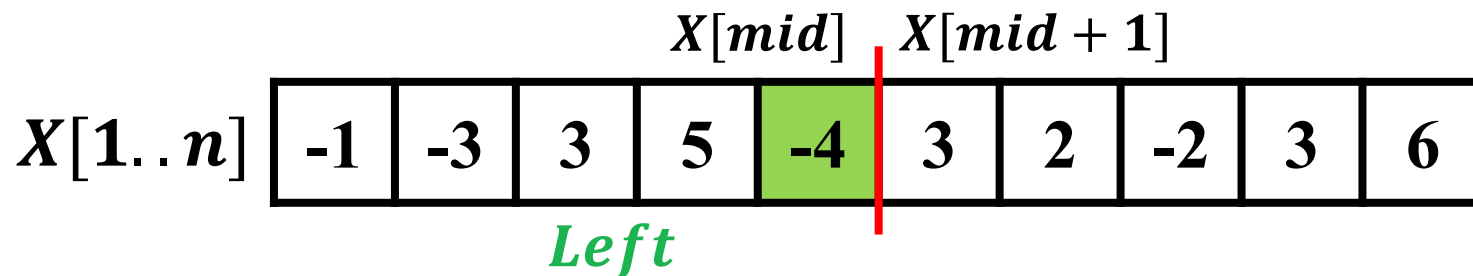
# 合并问题解：求解 $S_3$



|           |          |    |   |   |              |   |   |    |   |   |
|-----------|----------|----|---|---|--------------|---|---|----|---|---|
|           | $X[mid]$ |    |   |   | $X[mid + 1]$ |   |   |    |   |   |
| $X[1..n]$ | -1       | -3 | 3 | 5 | -4           | 3 | 2 | -2 | 3 | 6 |

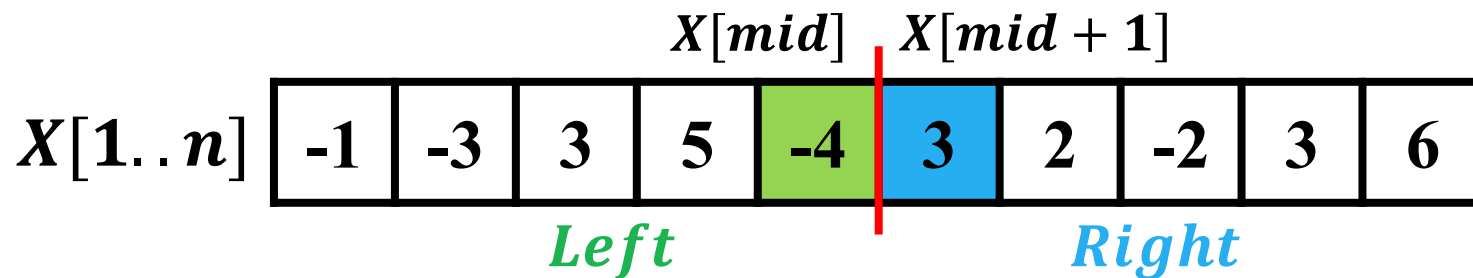
- 记  $mid = n/2$

# 合并问题解：求解 $S_3$



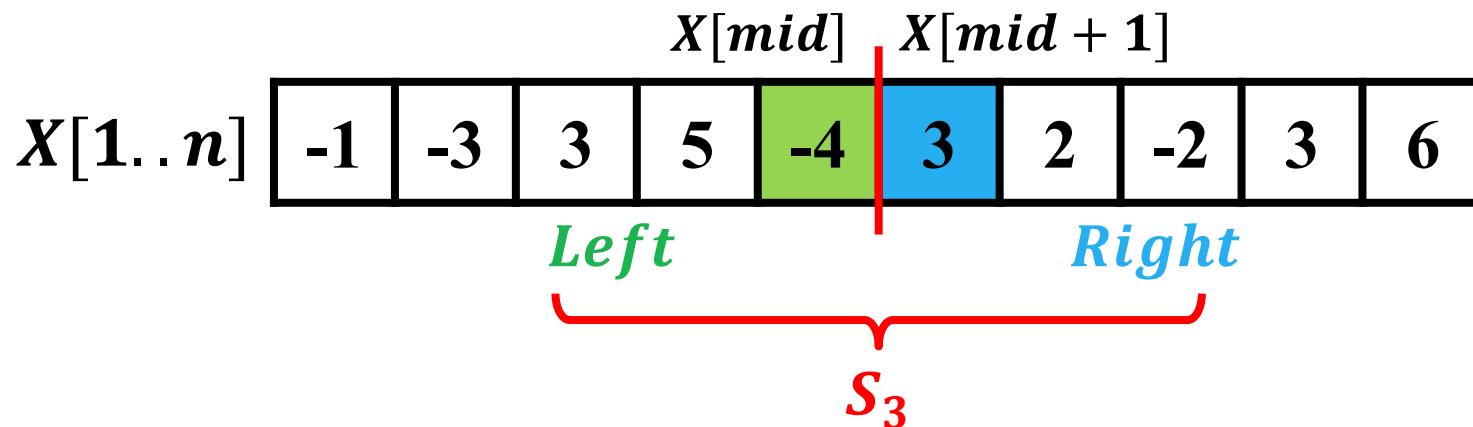
- 记  $mid = n/2$
- $S_3$  可分为左右两部分
  - *Left*: 以  $X[mid]$  为结尾的最大子数组之和

# 合并问题解：求解 $S_3$



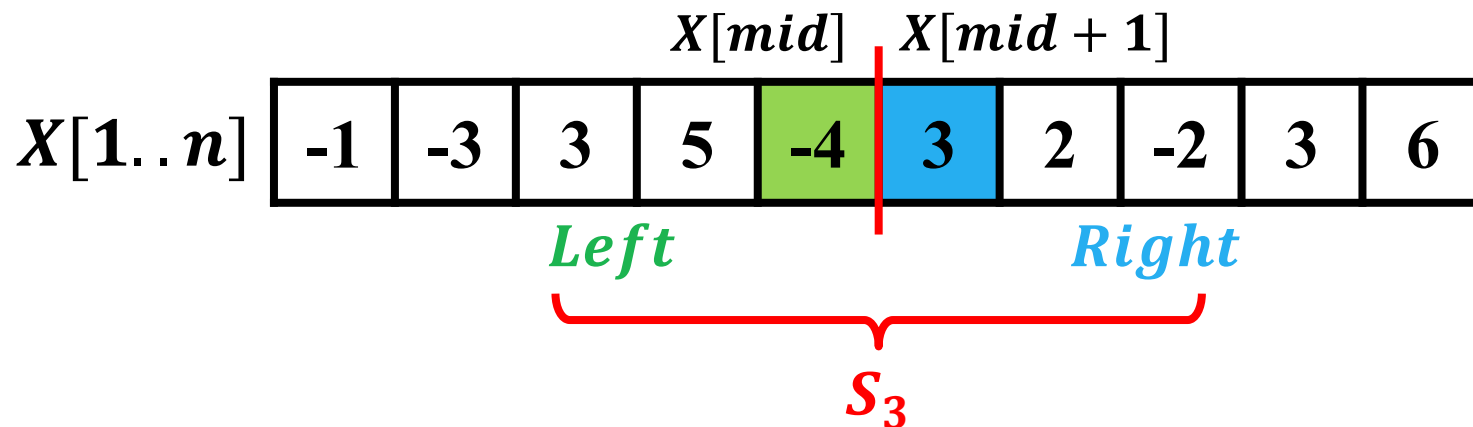
- 记  $mid = n/2$
- $S_3$  可分为左右两部分
  - **Left:** 以  $X[mid]$  为结尾的最大子数组之和
  - **Right:** 以  $X[mid + 1]$  为开头的最大子数组之和

# 合并问题解：求解 $S_3$



- 记  $mid = n/2$
- $S_3$  可分为左右两部分,  $S_3 = \text{Left} + \text{Right}$ 
  - **Left**: 以  $X[mid]$  为结尾的最大子数组之和
  - **Right**: 以  $X[mid+1]$  为开头的最大子数组之和

# 合并问题解：求解 $S_3$

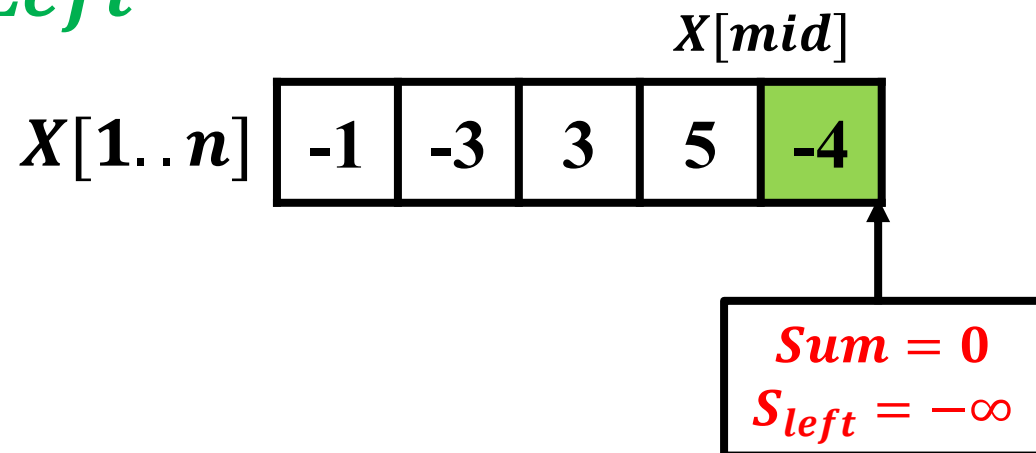


- 记  $mid = n/2$
- $S_3$  可分为左右两部分,  $S_3 = Left + Right$ 
  - **Left**: 以  $X[mid]$  为结尾的最大子数组之和
  - **Right**: 以  $X[mid + 1]$  为开头的最大子数组之和
  - 分别求出 **Left** 和 **Right**, 便可求出  $S_3$

# 合并问题解：求解 $S_3$



- 求解 $Left$

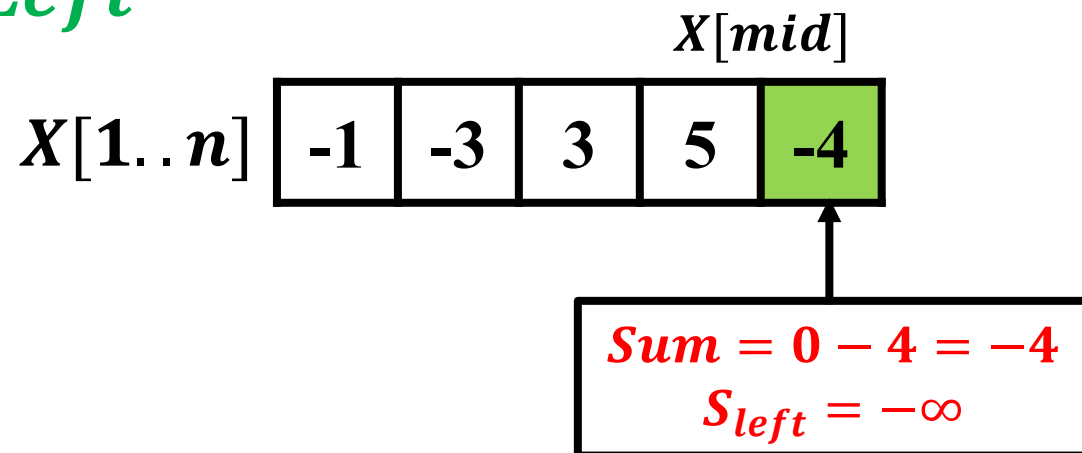


- 记 $mid = n/2$
- 从 $X[mid]$ 向前遍历求和，并记录最大值

# 合并问题解：求解 $S_3$



- 求解 $Left$



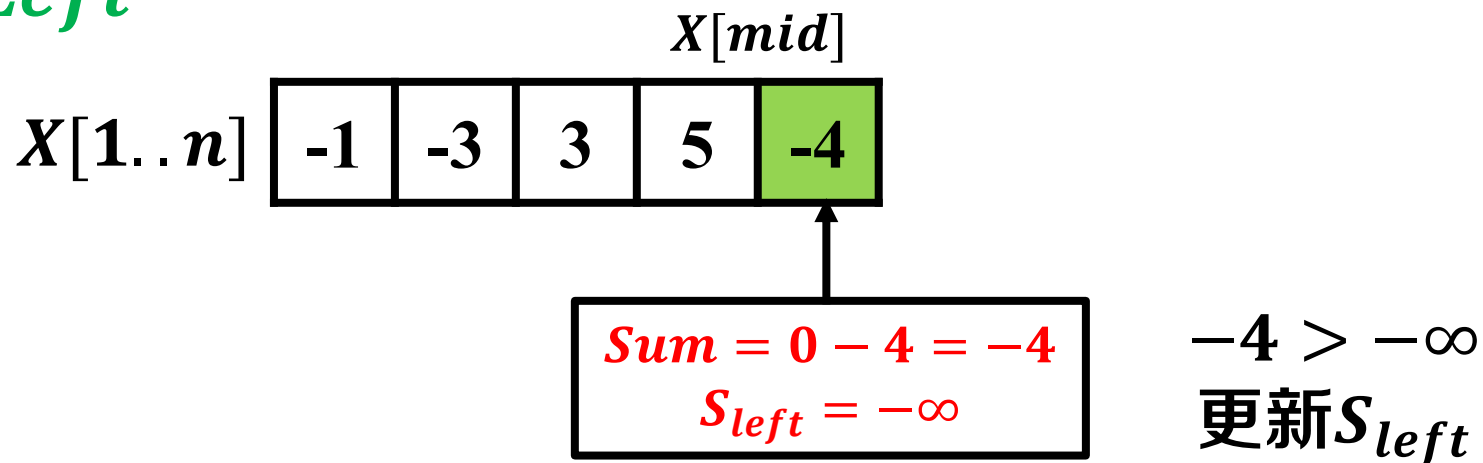
- 记 $mid = n/2$
- 从 $X[mid]$ 向前遍历求和，并记录最大值



# 合并问题解：求解 $S_3$



- 求解 $Left$

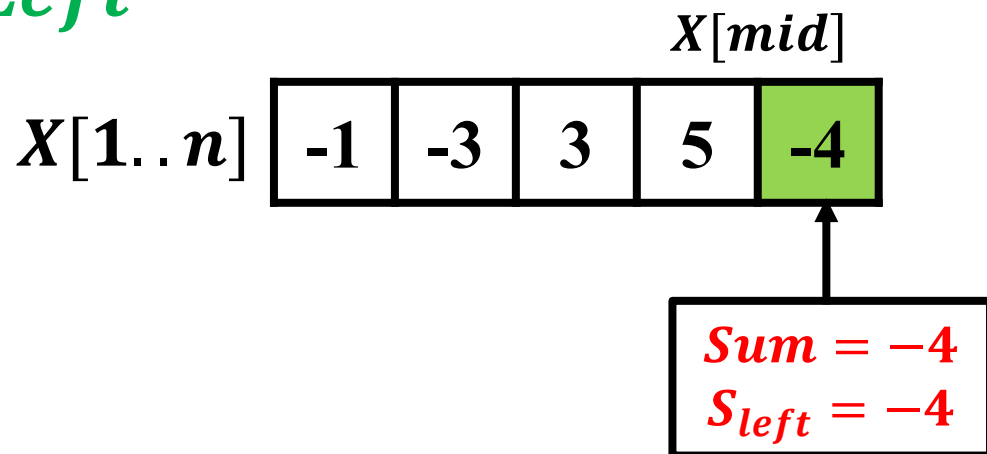


- 记 $mid = n/2$
- 从 $X[mid]$ 向前遍历求和，并记录最大值

# 合并问题解：求解 $S_3$



- 求解 $Left$

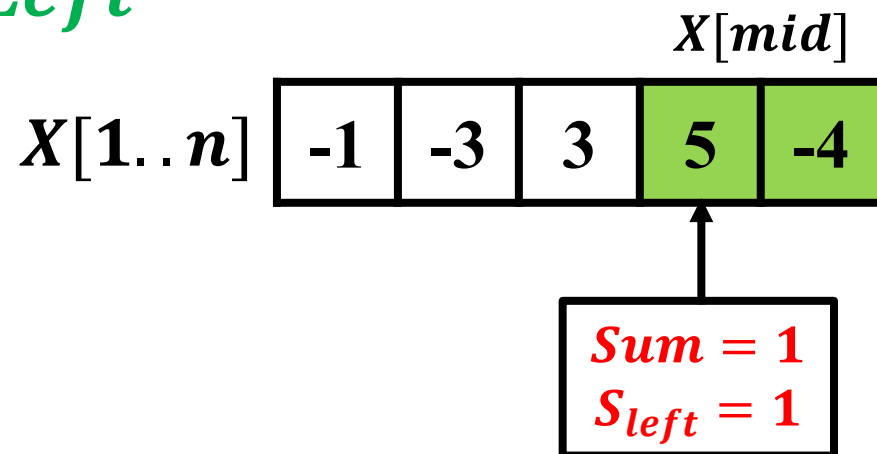


- 记 $mid = n/2$
- 从 $X[mid]$ 向前遍历求和，并记录最大值

# 合并问题解：求解 $S_3$



- 求解 $Left$

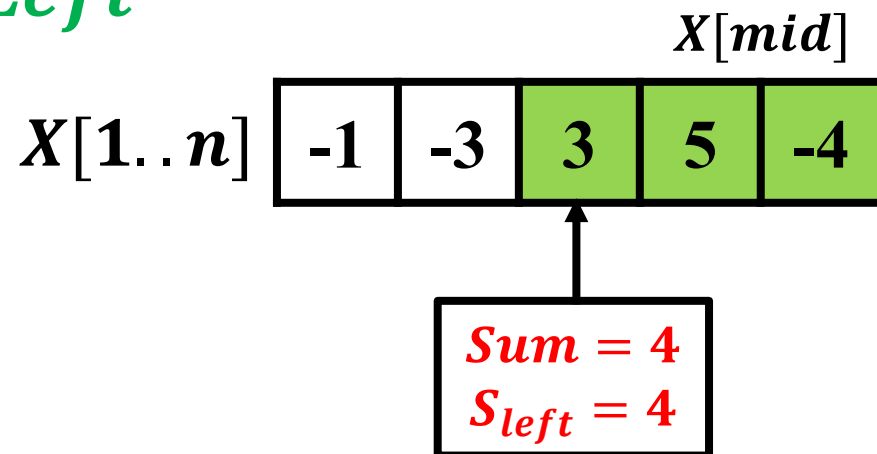


- 记 $mid = n/2$
- 从 $X[mid]$ 向前遍历求和，并记录最大值

# 合并问题解：求解 $S_3$



- 求解 $Left$

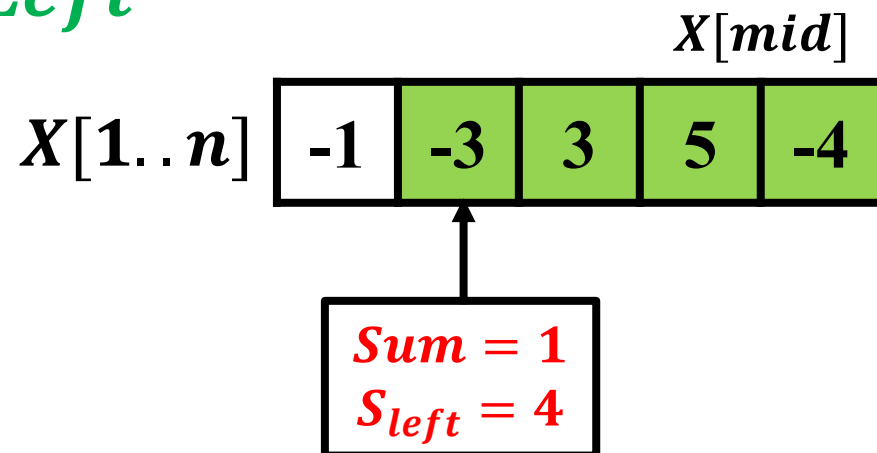


- 记 $mid = n/2$
- 从 $X[mid]$ 向前遍历求和，并记录最大值

# 合并问题解：求解 $S_3$



- 求解 $Left$

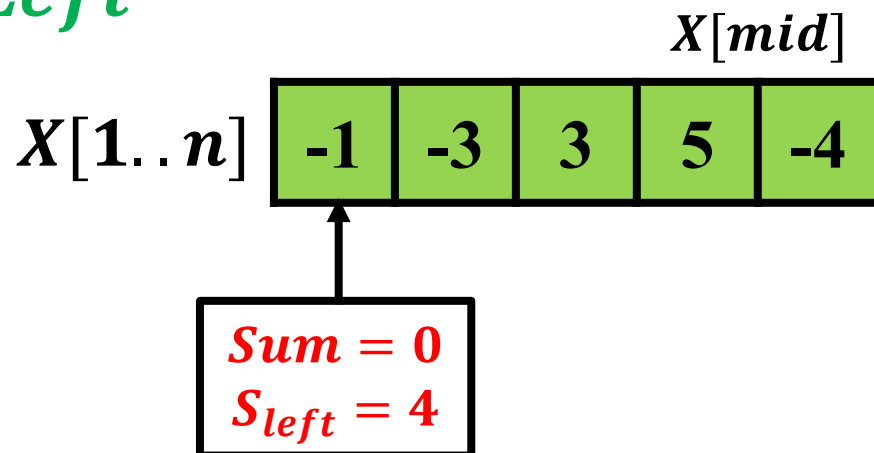


- 记 $mid = n/2$
- 从 $X[mid]$ 向前遍历求和，并记录最大值

# 合并问题解：求解 $S_3$



- 求解 $Left$

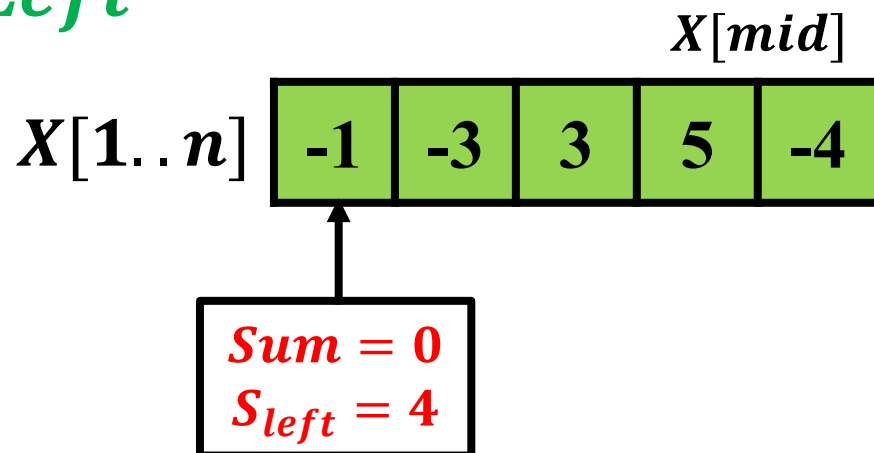


- 记 $mid = n/2$
- 从 $X[mid]$ 向前遍历求和，并记录最大值

# 合并问题解：求解 $S_3$



- 求解 $Left$

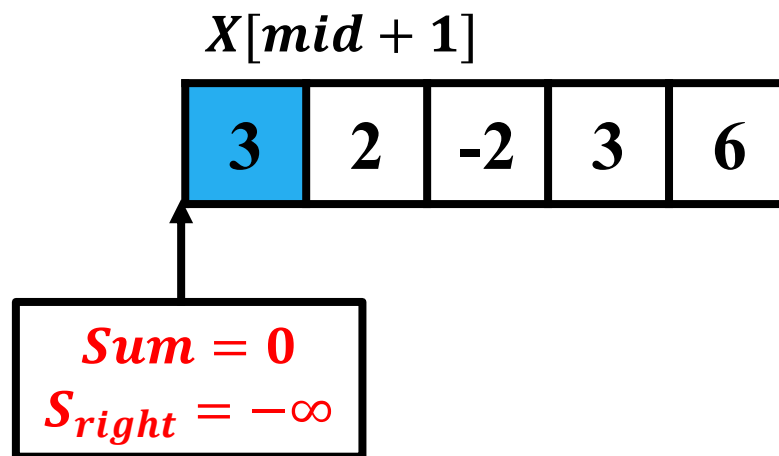


- 记 $mid = n/2$
- 从 $X[mid]$ 向前遍历求和，并记录最大值
- 以 $X[mid]$ 为结尾的最大子数组之和 $S_{left} = 4$

# 合并问题解：求解 $S_3$



- 求解 $Right$



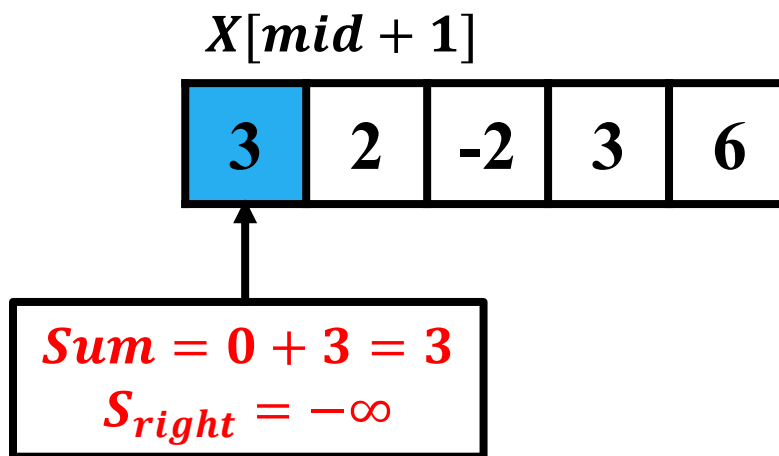
- 记 $mid = n/2$
- 从 $X[mid + 1]$ 向后遍历求和，并记录最大值



# 合并问题解：求解 $S_3$



- 求解 $Right$

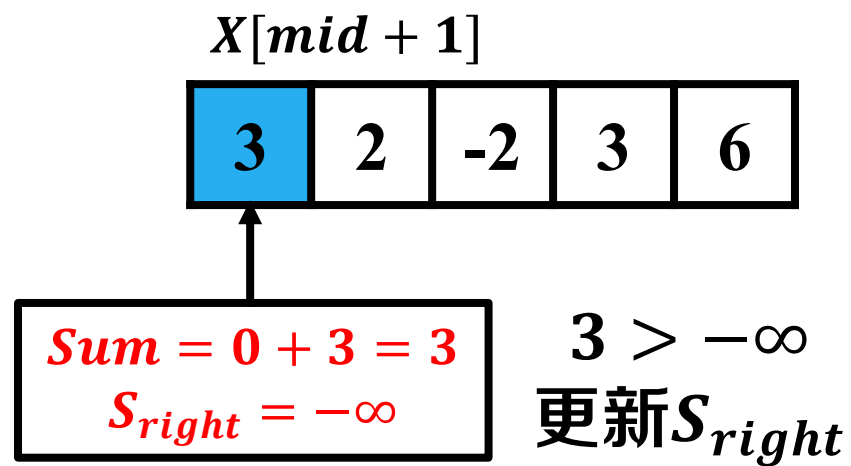


- 记 $mid = n/2$
- 从 $X[mid + 1]$ 向后遍历求和，并记录最大值

# 合并问题解：求解 $S_3$



- 求解 $Right$

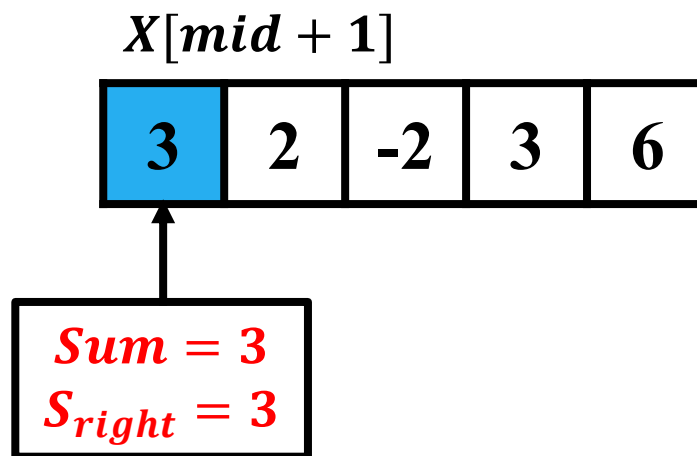


- 记 $mid = n/2$
- 从 $X[mid + 1]$ 向后遍历求和，并记录最大值

# 合并问题解：求解 $S_3$



- 求解 $Right$

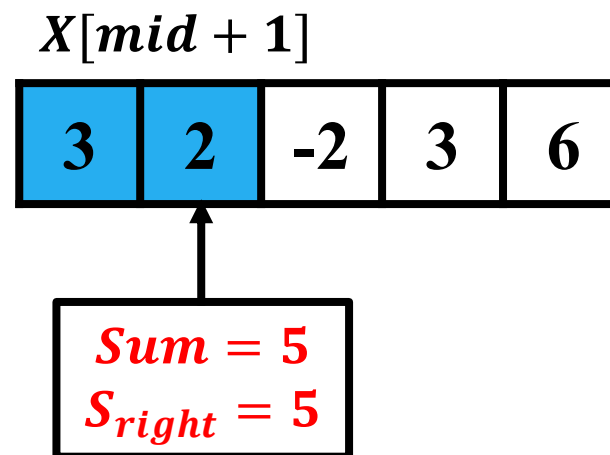


- 记 $mid = n/2$
- 从 $X[mid + 1]$ 向后遍历求和，并记录最大值

# 合并问题解：求解 $S_3$



- 求解 $Right$

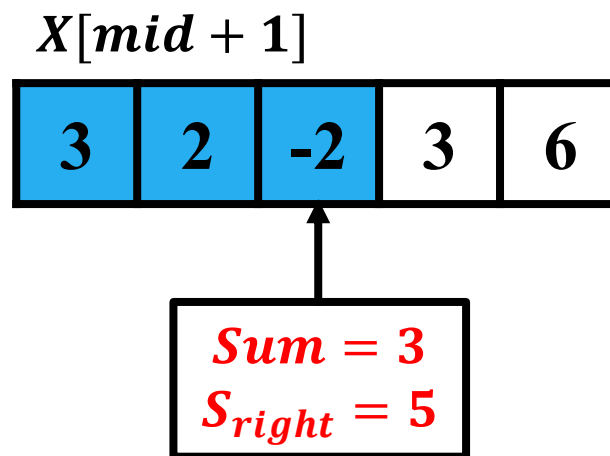


- 记 $mid = n/2$
- 从 $X[mid + 1]$ 向后遍历求和，并记录最大值

# 合并问题解：求解 $S_3$



- 求解 $Right$

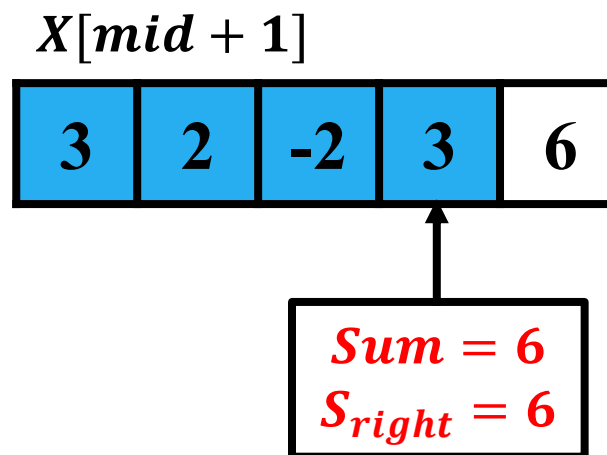


- 记 $mid = n/2$
- 从 $X[mid + 1]$ 向后遍历求和，并记录最大值

# 合并问题解：求解 $S_3$



- 求解 $Right$

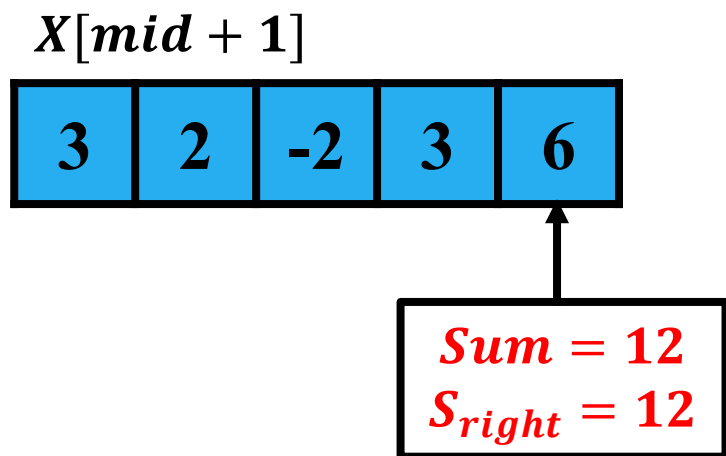


- 记 $mid = n/2$
- 从 $X[mid + 1]$ 向后遍历求和，并记录最大值

# 合并问题解：求解 $S_3$



- 求解 $Right$

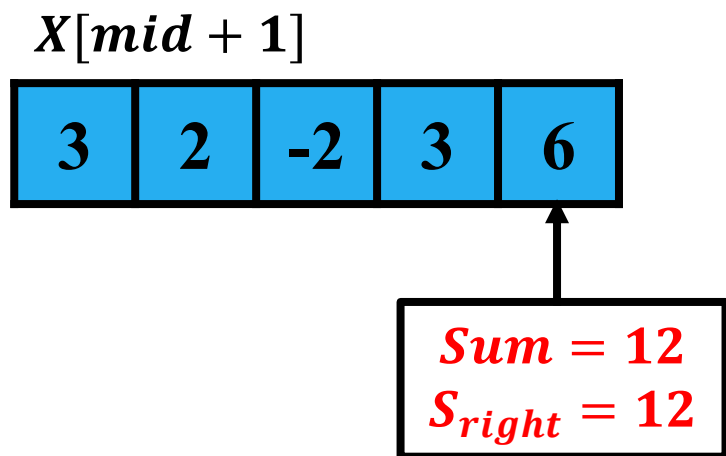


- 记 $mid = n/2$
- 从 $X[mid + 1]$ 向后遍历求和，并记录最大值

# 合并问题解：求解 $S_3$



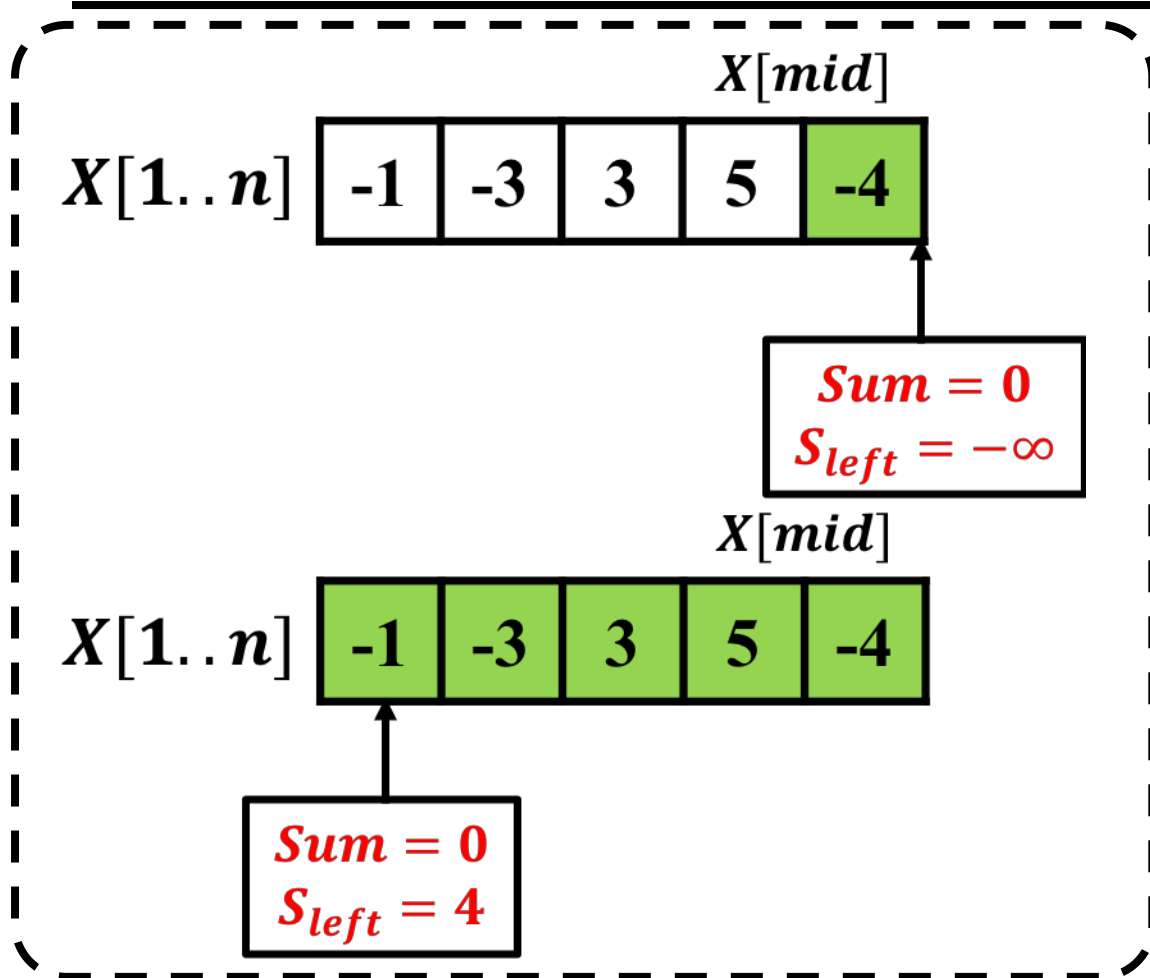
- 求解 $Right$



- 记 $mid = n/2$
- 从 $X[mid + 1]$ 向后遍历求和，并记录最大值
- 以 $X[mid + 1]$ 为开头的最大子数组之和 $S_{right} = 12$

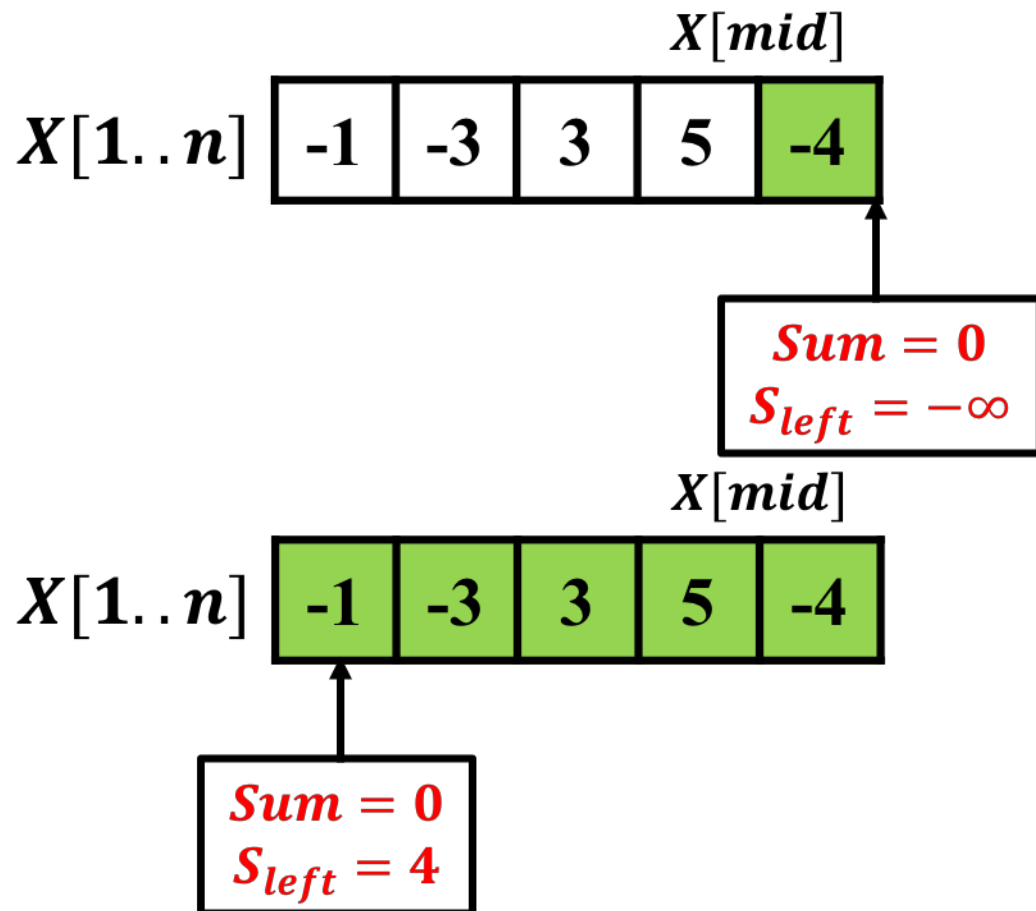


# 求解 $S_3$ : 时间复杂度

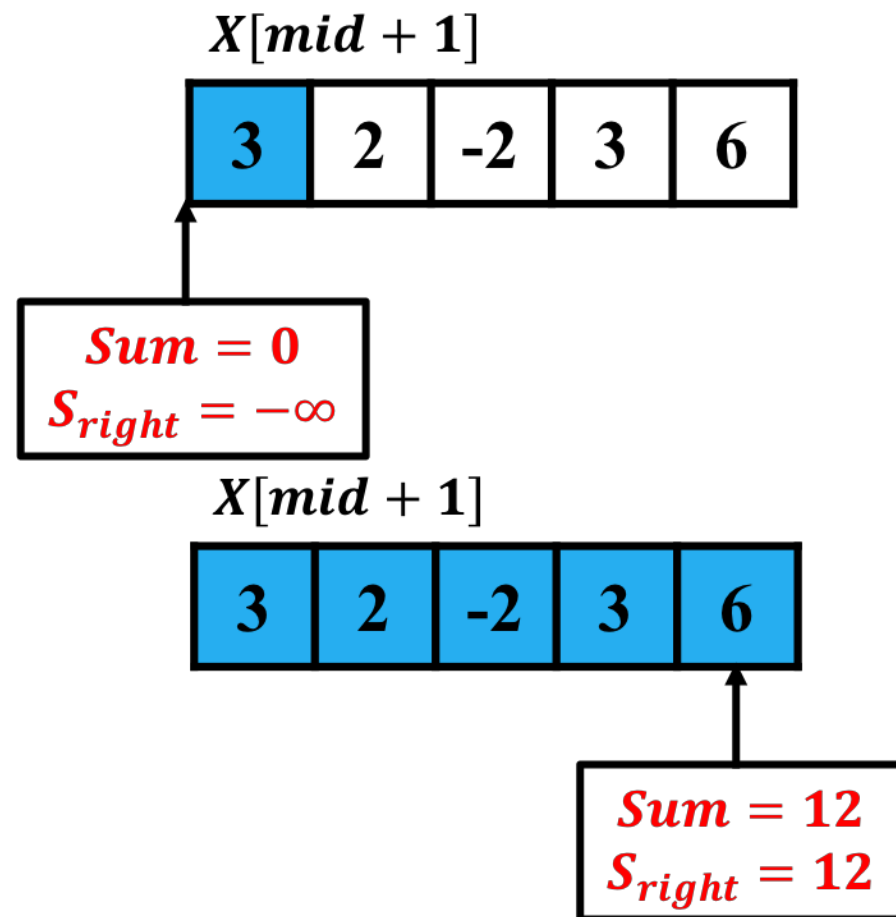


求解 $Left$ 时间复杂度:  $O(mid)$

# 求解 $S_3$ : 时间复杂度

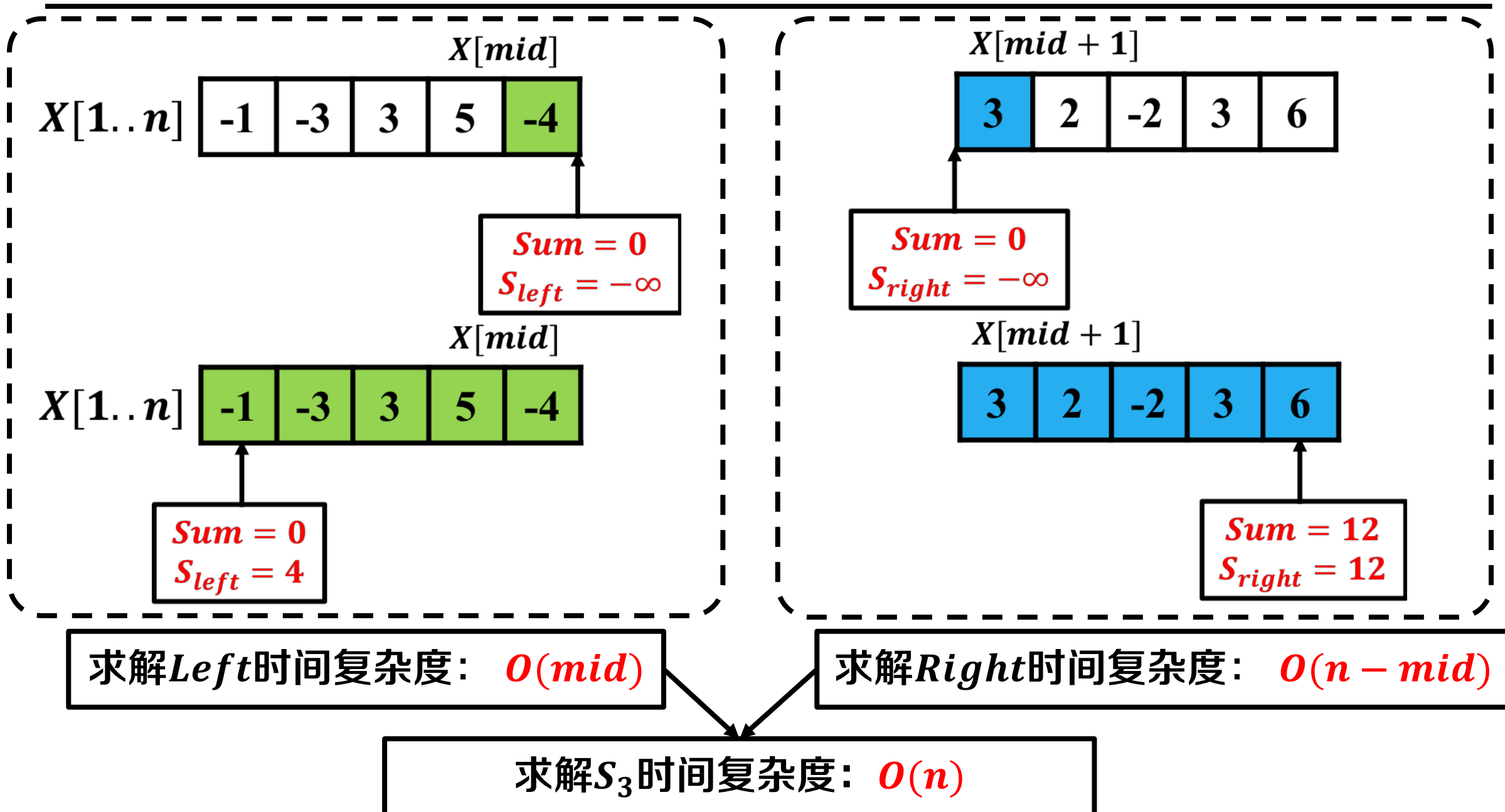


求解 $Left$ 时间复杂度:  $O(mid)$



求解 $Right$ 时间复杂度:  $O(n - mid)$

# 求解 $S_3$ : 时间复杂度



# 求解 $S_3$ : 伪代码

---



- **CrossingSubArray( $X, low, mid, high$ )**

# 求解 $S_3$ ：伪代码



- CrossingSubArray( $X, low, mid, high$ )

输入: 数组 $X$ , 数组下标 $low, mid, high$

输出: 跨越中点的最大子数组之和 $S_3$

```
 $S_{left} \leftarrow -\infty$
 $Sum \leftarrow 0$
for $l \leftarrow mid$ downto low do
| $Sum \leftarrow Sum + X[l]$
| $S_{left} \leftarrow \max\{S_{left}, Sum\}$
end
```

求解 $Left$

```
 $S_{right} \leftarrow -\infty$
 $Sum \leftarrow 0$
for $r \leftarrow mid + 1$ to $high$ do
| $Sum \leftarrow Sum + X[r]$
| $S_{right} \leftarrow \max\{S_{right}, Sum\}$
end
 $S_3 \leftarrow S_{left} + S_{right}$
return S_3
```

# 求解 $S_3$ : 伪代码



- **CrossingSubArray( $X, low, mid, high$ )**

输入: 数组  $X$ , 数组下标  $low, mid, high$

输出: 跨越中点的最大子数组之和  $S_3$

$S_{left} \leftarrow -\infty$

$Sum \leftarrow 0$

**for**  $l \leftarrow mid$  **downto**  $low$  **do**

$Sum \leftarrow Sum + X[l]$

$S_{left} \leftarrow \max\{S_{left}, Sum\}$

**end**

$S_{right} \leftarrow -\infty$

$Sum \leftarrow 0$

**for**  $r \leftarrow mid + 1$  **to**  $high$  **do**

$Sum \leftarrow Sum + X[r]$

$S_{right} \leftarrow \max\{S_{right}, Sum\}$

**end**

$S_3 \leftarrow S_{left} + S_{right}$

**return**  $S_3$

求解  $Right$

# 求解 $S_3$ ：伪代码



- CrossingSubArray( $X, low, mid, high$ )

输入: 数组  $X$ , 数组下标  $low, mid, high$

输出: 跨越中点的最大子数组之和  $S_3$

$S_{left} \leftarrow -\infty$

$Sum \leftarrow 0$

**for**  $l \leftarrow mid$  **downto**  $low$  **do**

$Sum \leftarrow Sum + X[l]$

$S_{left} \leftarrow \max\{S_{left}, Sum\}$

**end**

$S_{right} \leftarrow -\infty$

$Sum \leftarrow 0$

**for**  $r \leftarrow mid + 1$  **to**  $high$  **do**

$Sum \leftarrow Sum + X[r]$

$S_{right} \leftarrow \max\{S_{right}, Sum\}$

**end**

$S_3 \leftarrow S_{left} + S_{right}$

**return**  $S_3$

求解 $S_3$

# 求解 $S_3$ : 伪代码



- **CrossingSubArray( $X, low, mid, high$ )**

输入: 数组 $X$ , 数组下标 $low, mid, high$

输出: 跨越中点的最大子数组之和 $S_3$

$S_{left} \leftarrow -\infty$

$Sum \leftarrow 0$

**for**  $l \leftarrow mid$  **downto**  $low$  **do**

$Sum \leftarrow Sum + X[l]$

$S_{left} \leftarrow \max\{S_{left}, Sum\}$

**end**

$S_{right} \leftarrow -\infty$

$Sum \leftarrow 0$

**for**  $r \leftarrow mid + 1$  **to**  $high$  **do**

$Sum \leftarrow Sum + X[r]$

$S_{right} \leftarrow \max\{S_{right}, Sum\}$

**end**

$S_3 \leftarrow S_{left} + S_{right}$

**return**  $S_3$

$O(mid)$

$O(n - mid)$

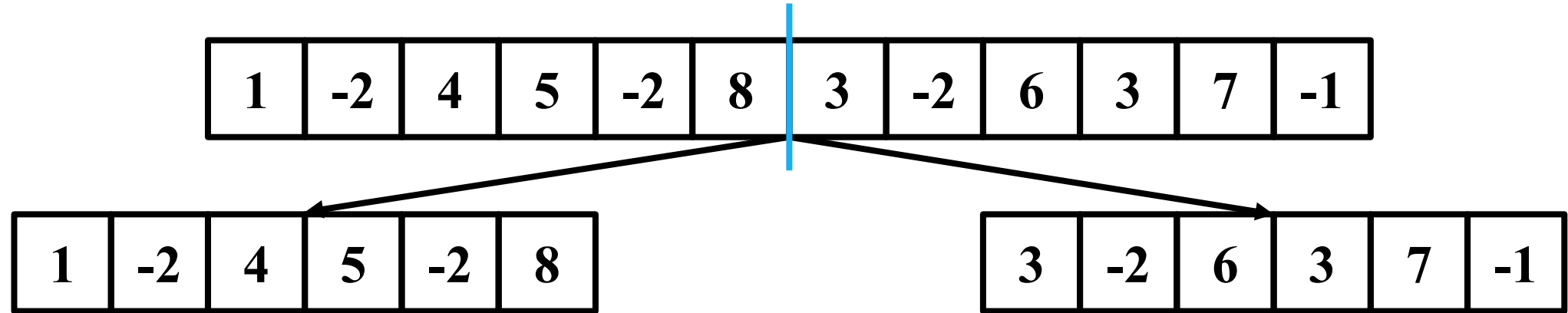
时间复杂度:  $O(n)$

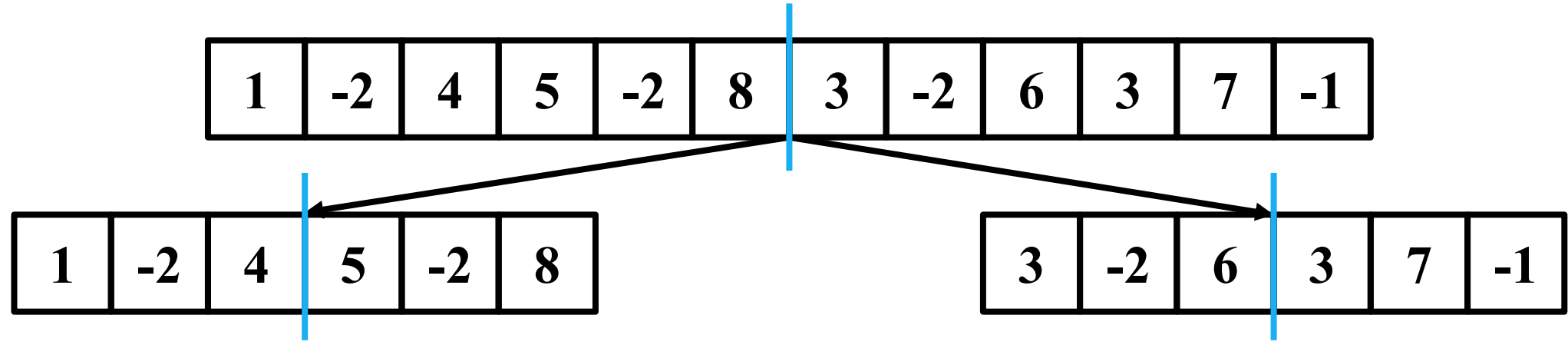


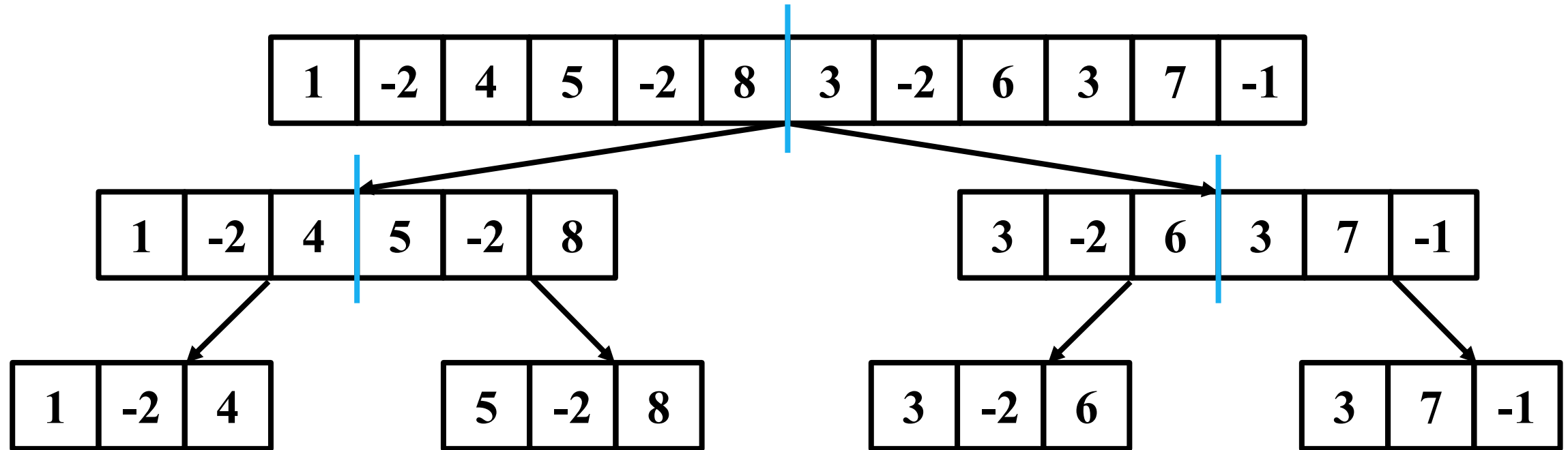
|   |    |   |   |    |   |   |    |   |   |   |    |
|---|----|---|---|----|---|---|----|---|---|---|----|
| 1 | -2 | 4 | 5 | -2 | 8 | 3 | -2 | 6 | 3 | 7 | -1 |
|---|----|---|---|----|---|---|----|---|---|---|----|

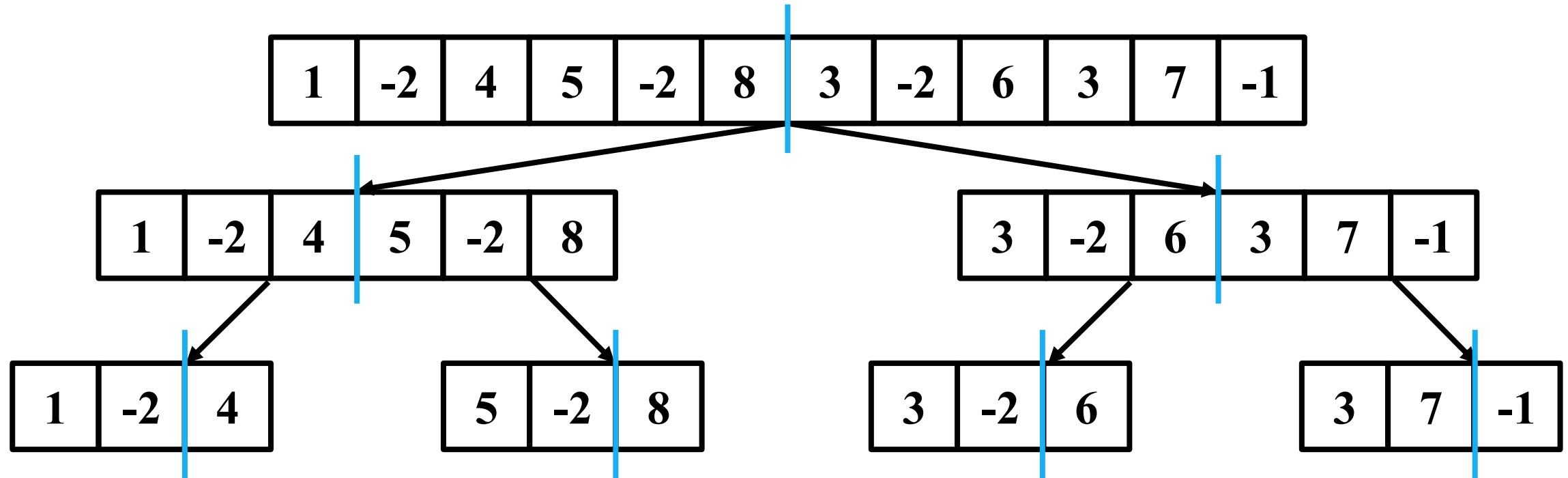
---

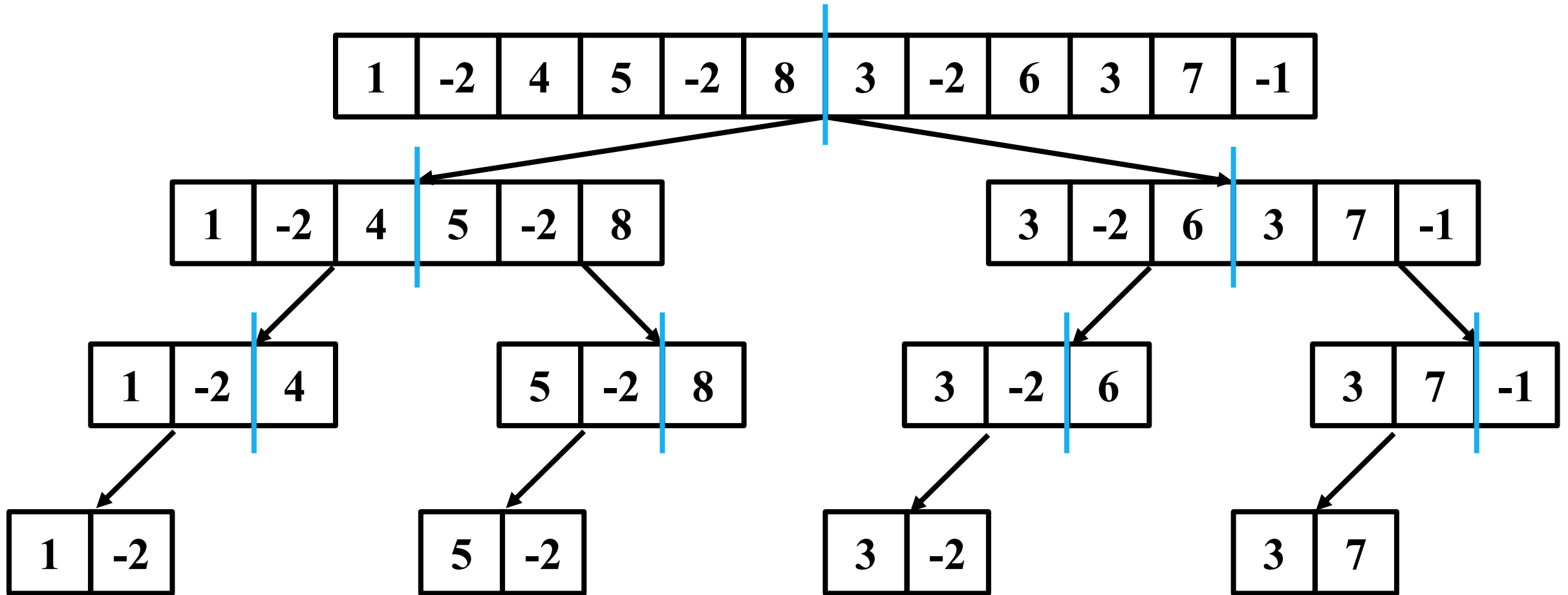
|   |    |   |   |    |   |   |    |   |   |   |    |
|---|----|---|---|----|---|---|----|---|---|---|----|
| 1 | -2 | 4 | 5 | -2 | 8 | 3 | -2 | 6 | 3 | 7 | -1 |
|---|----|---|---|----|---|---|----|---|---|---|----|

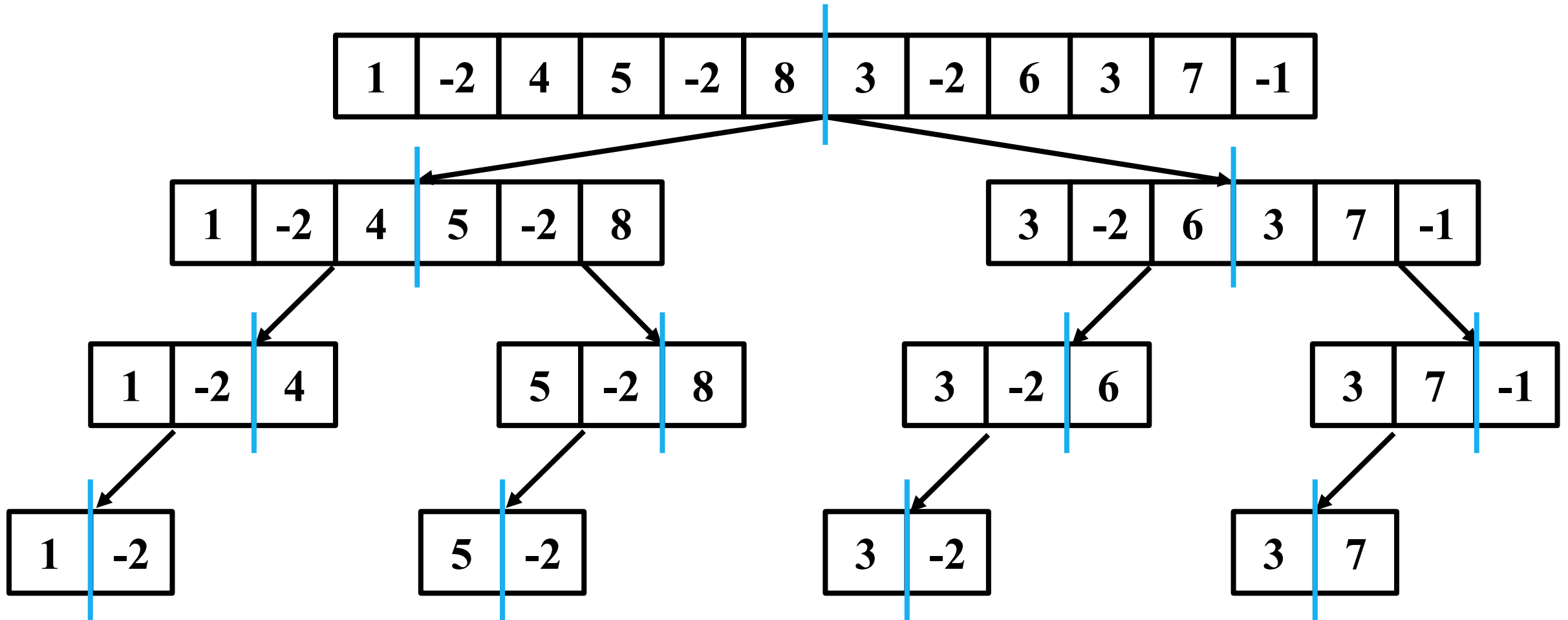




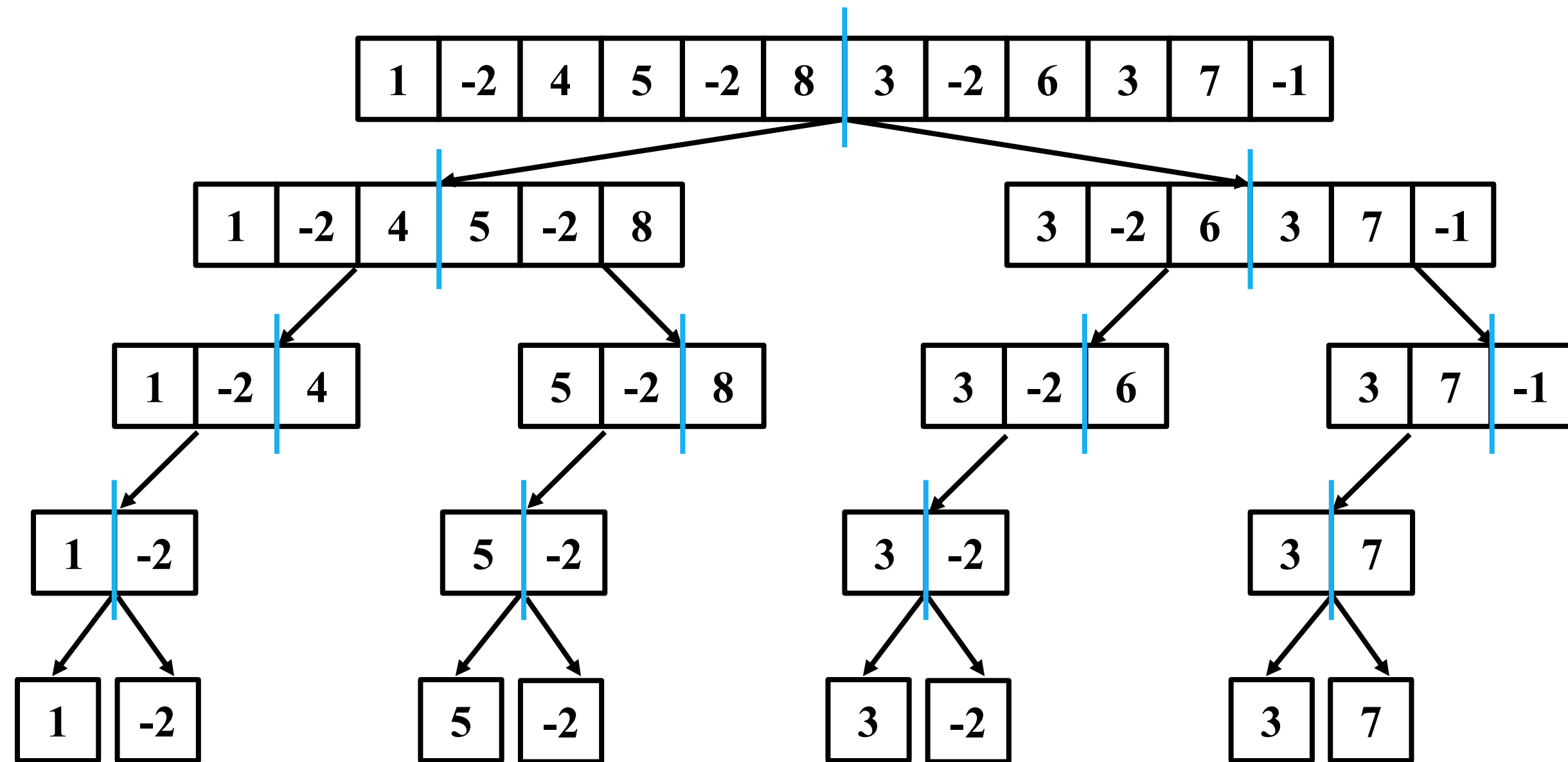


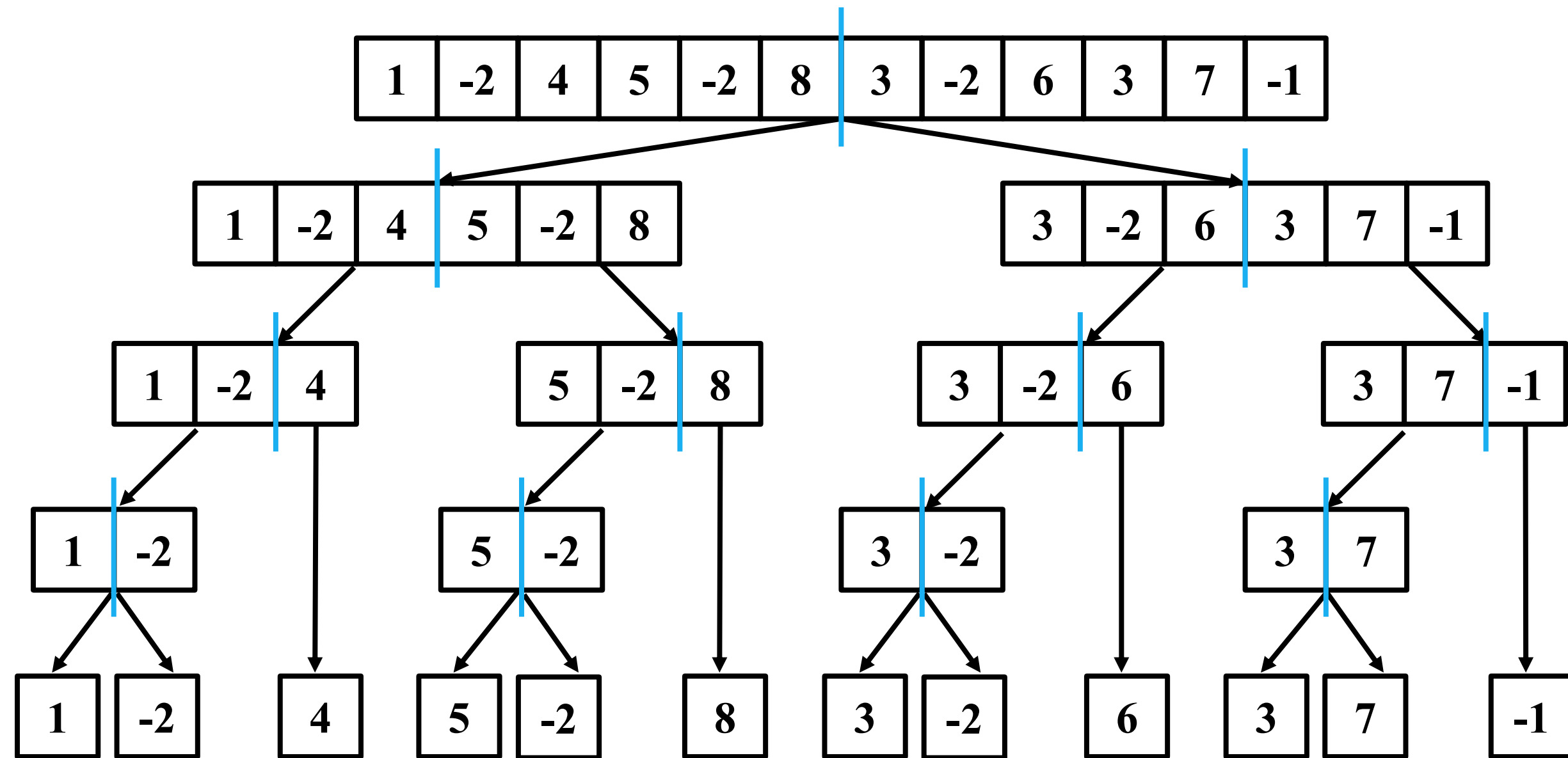




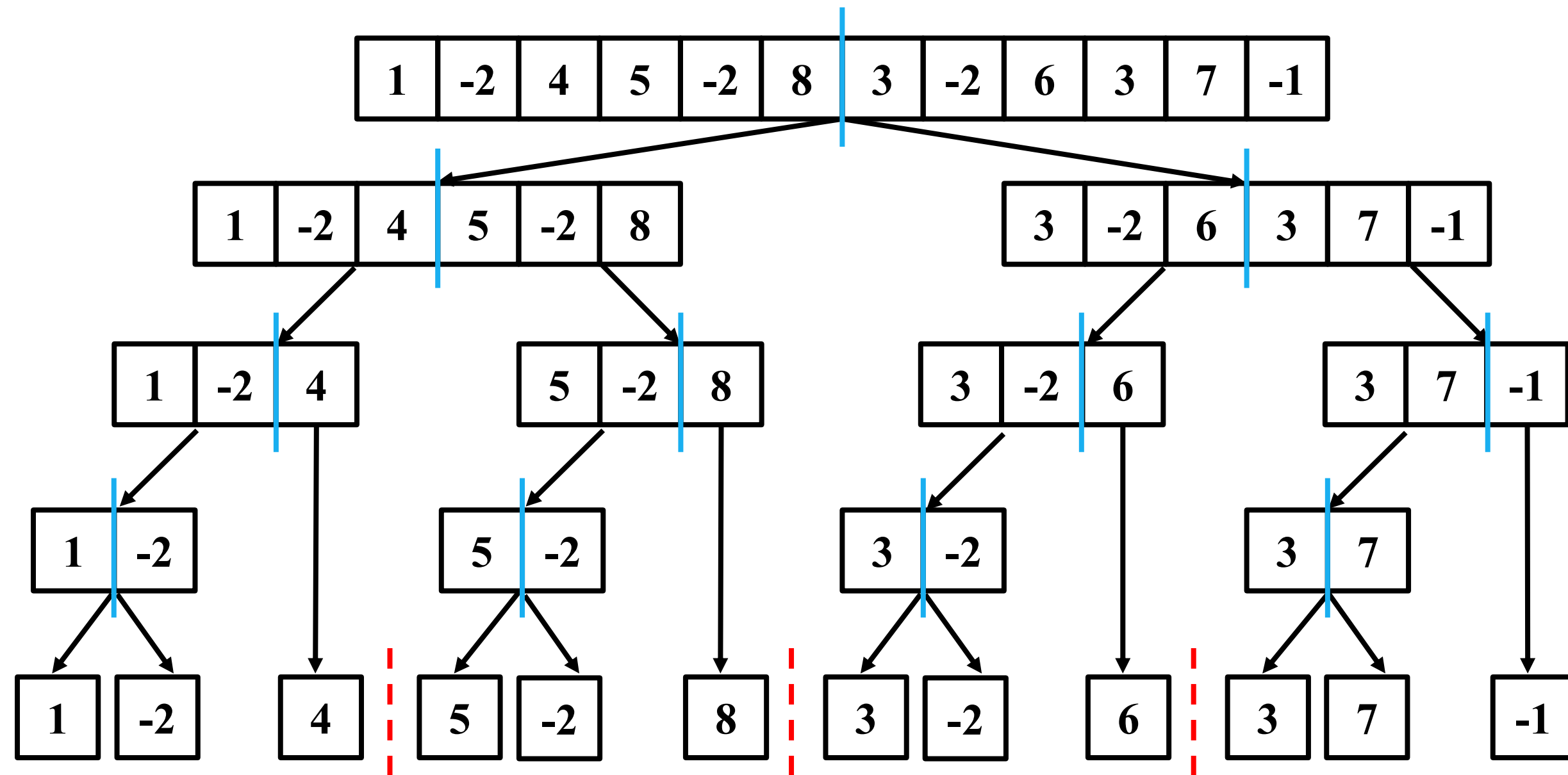


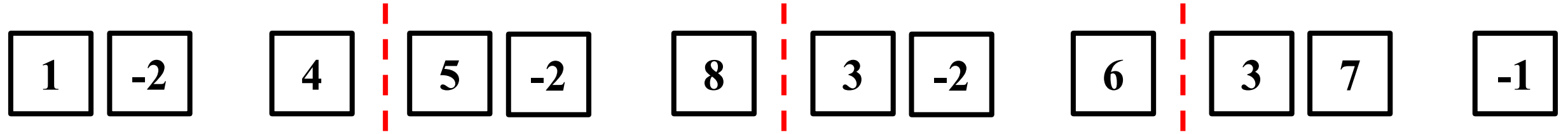




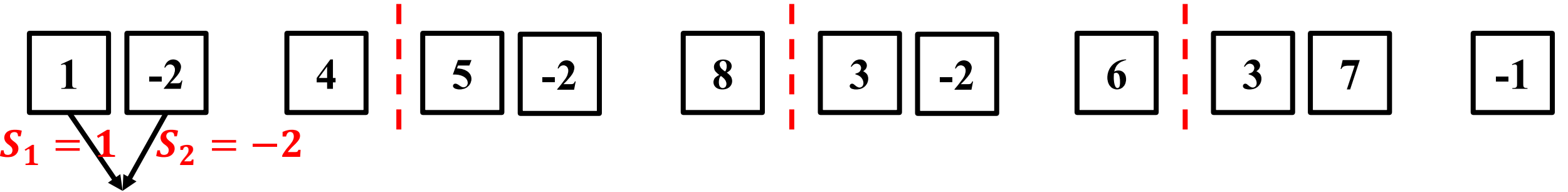


# 算法实例

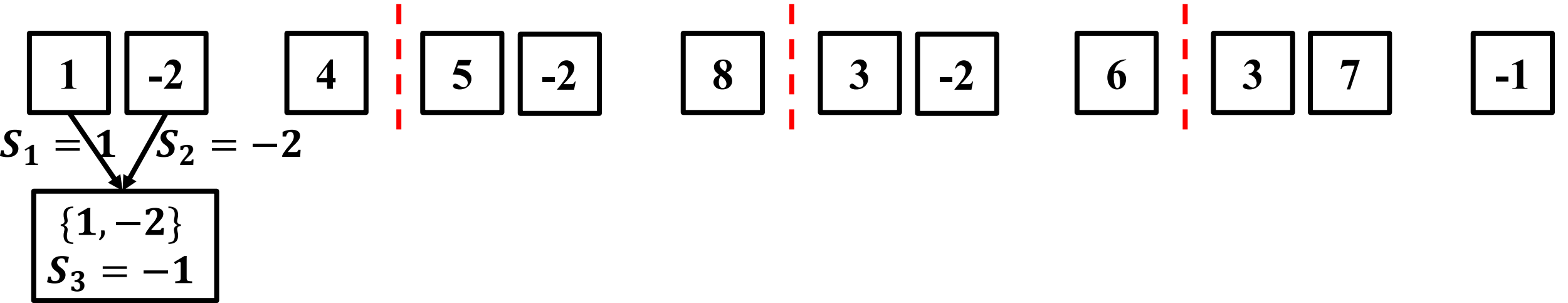




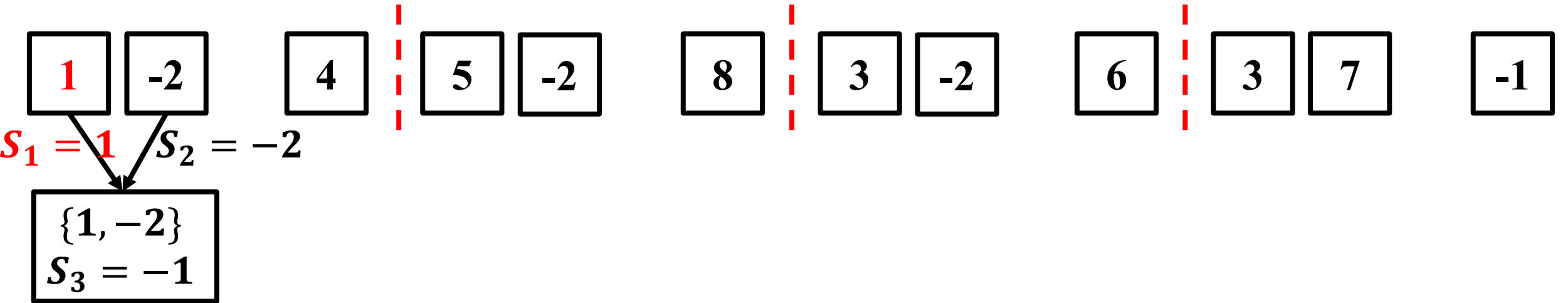
# 算法实例



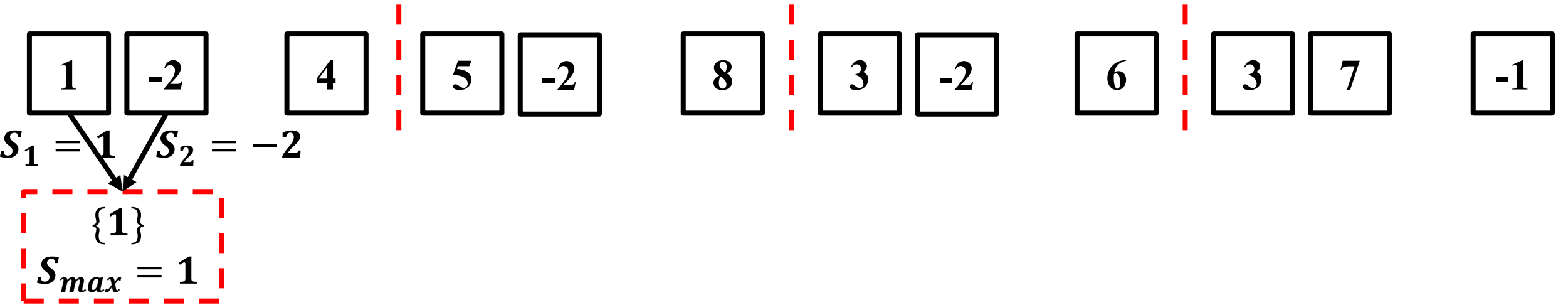
# 算法实例



# 算法实例

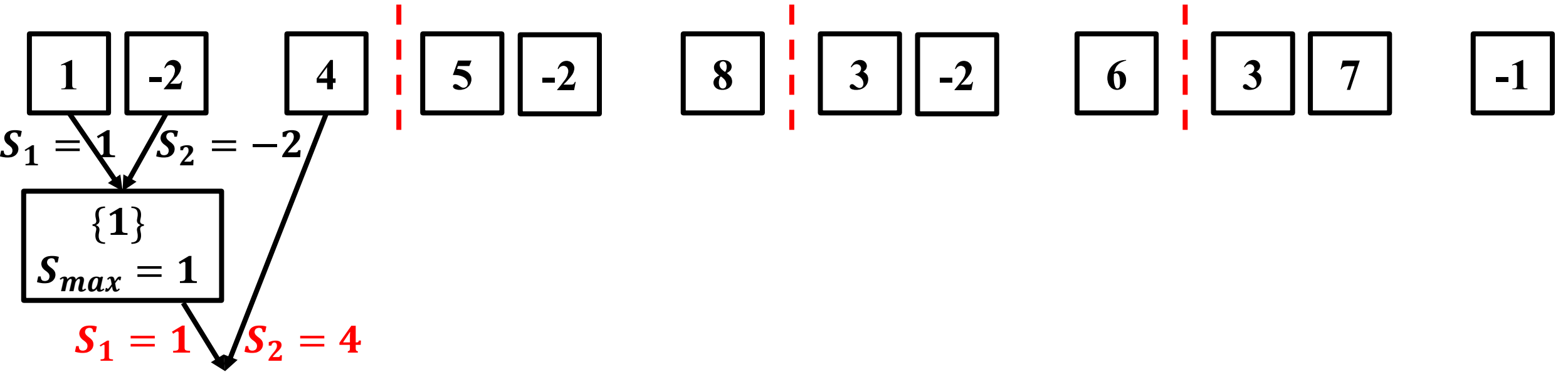


# 算法实例

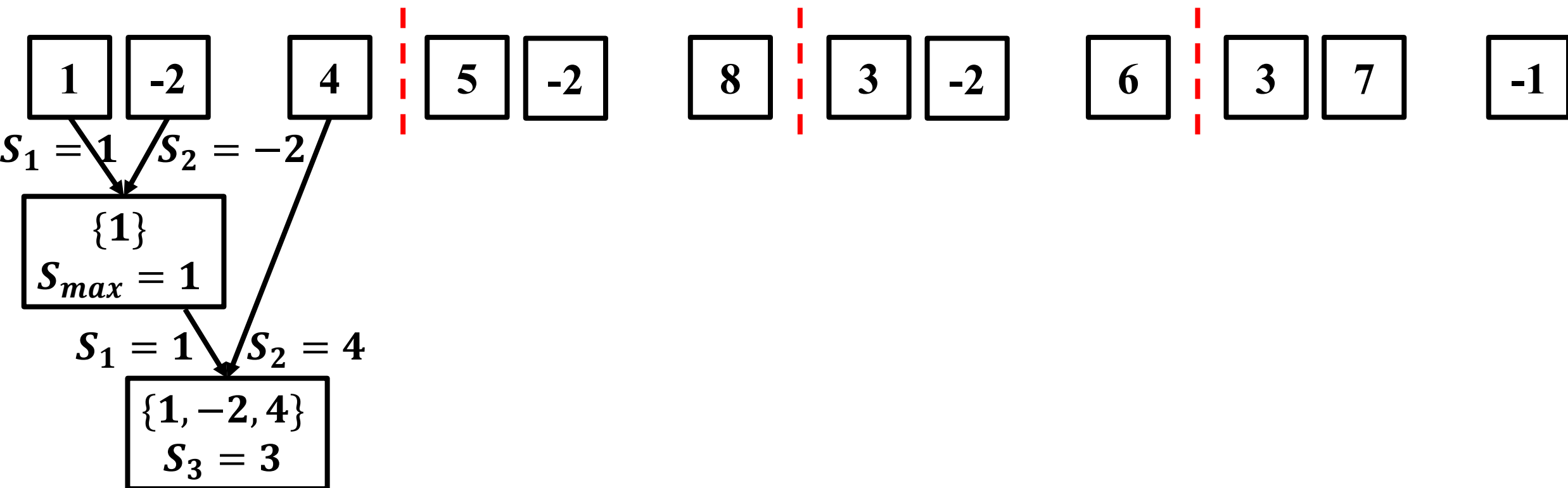




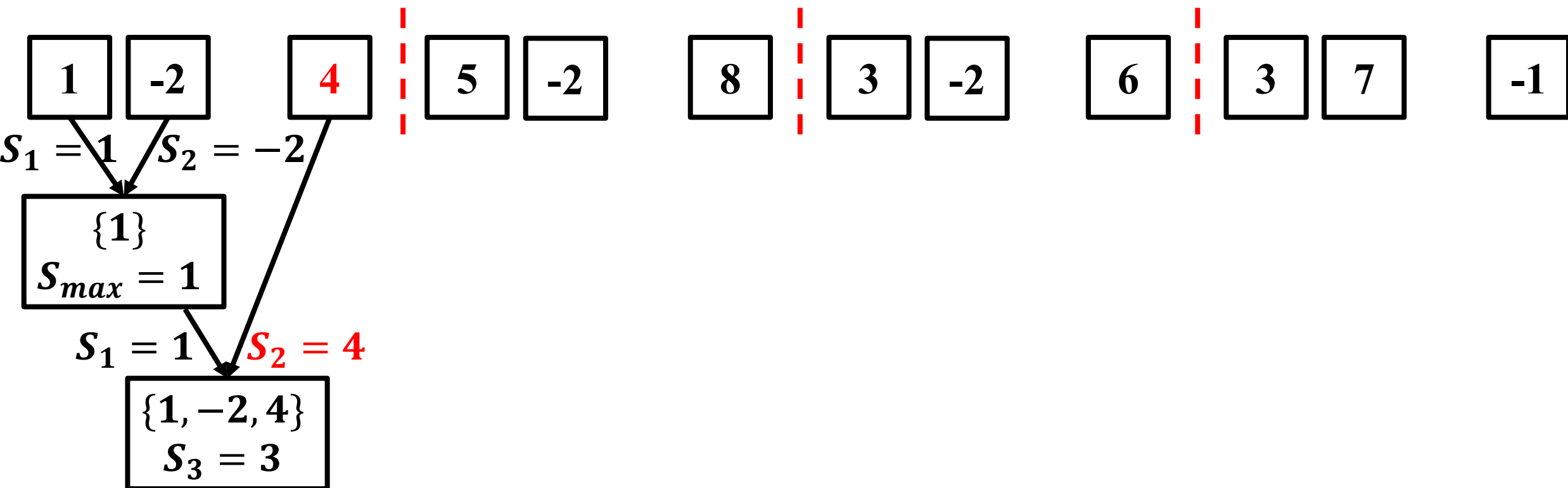
# 算法实例



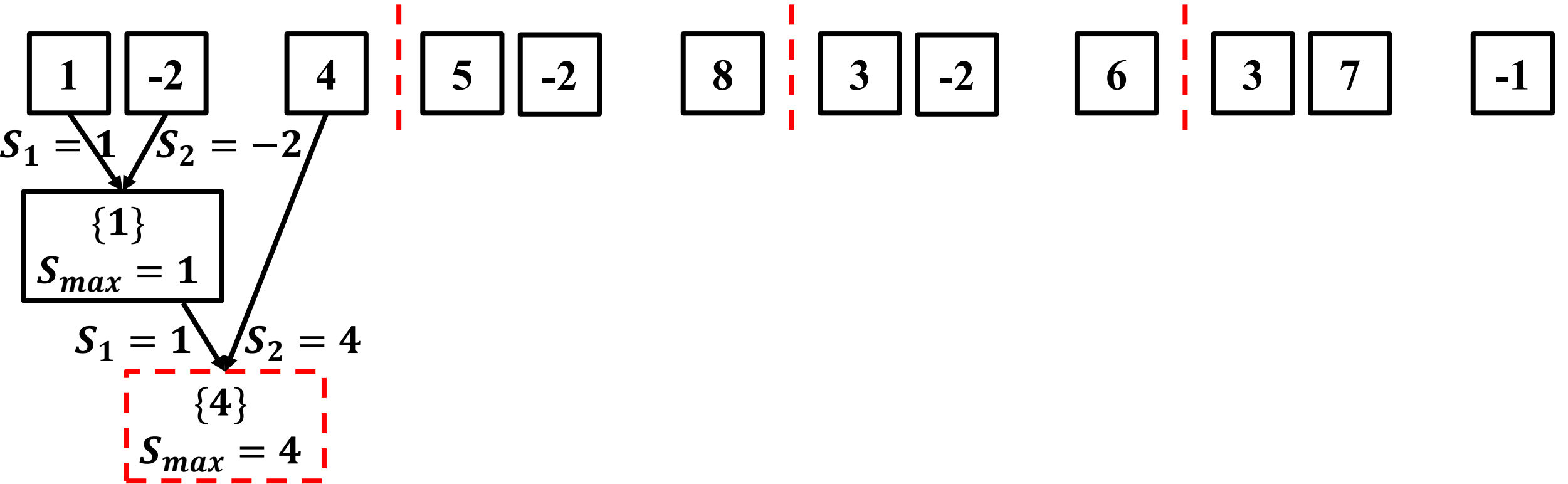
# 算法实例



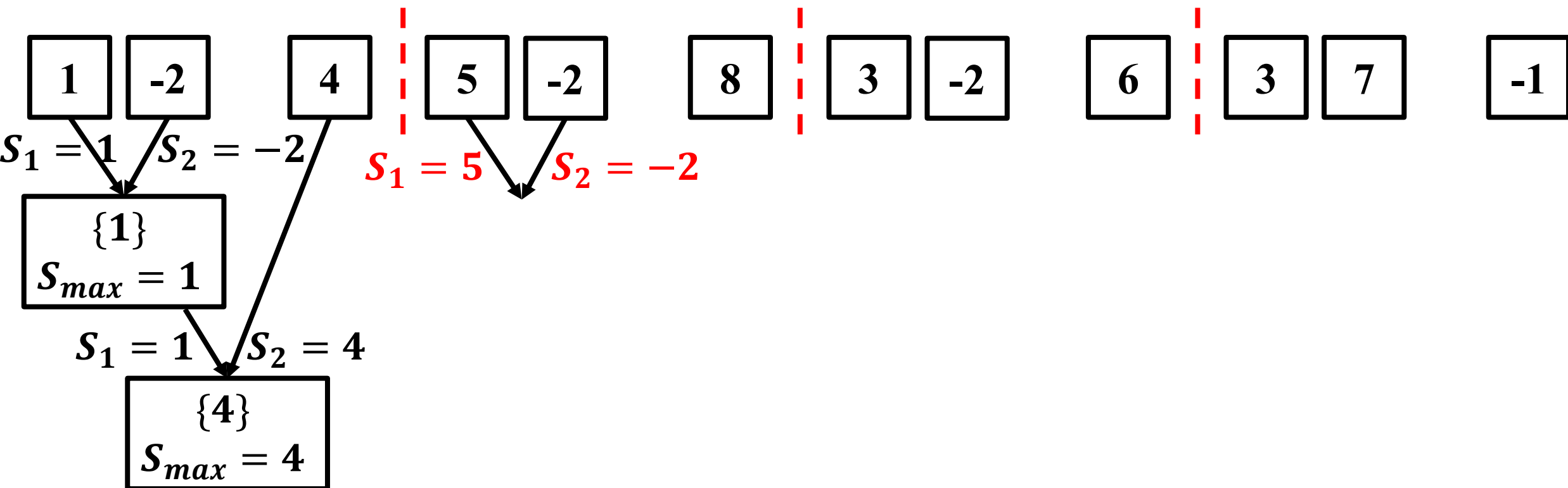
# 算法实例



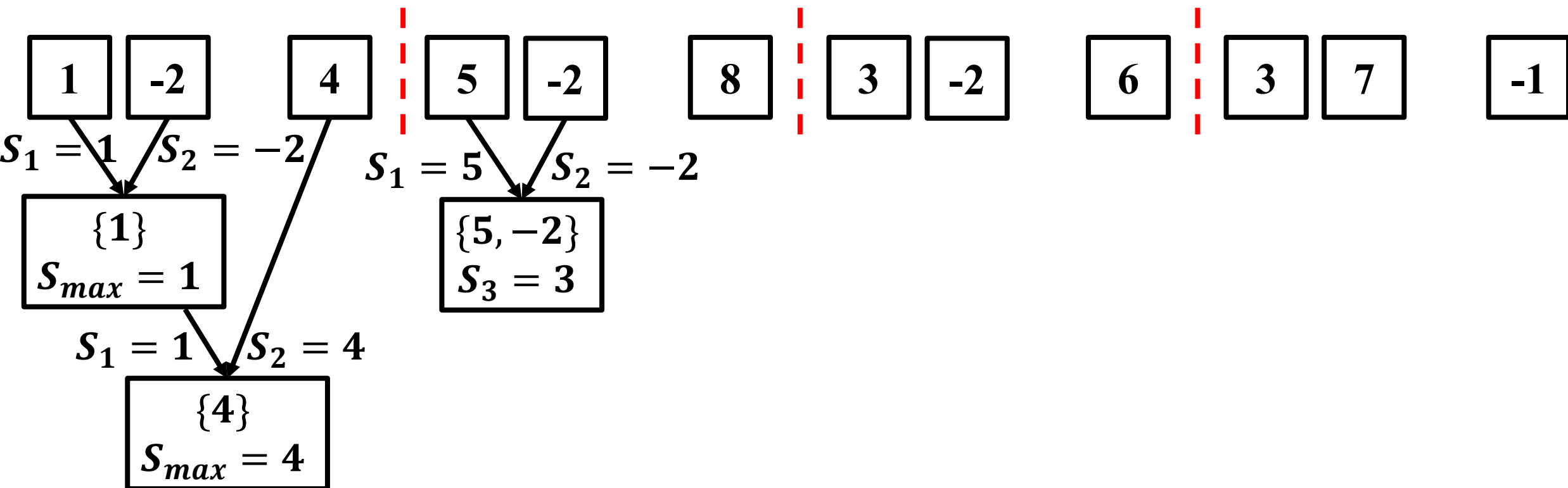
# 算法实例



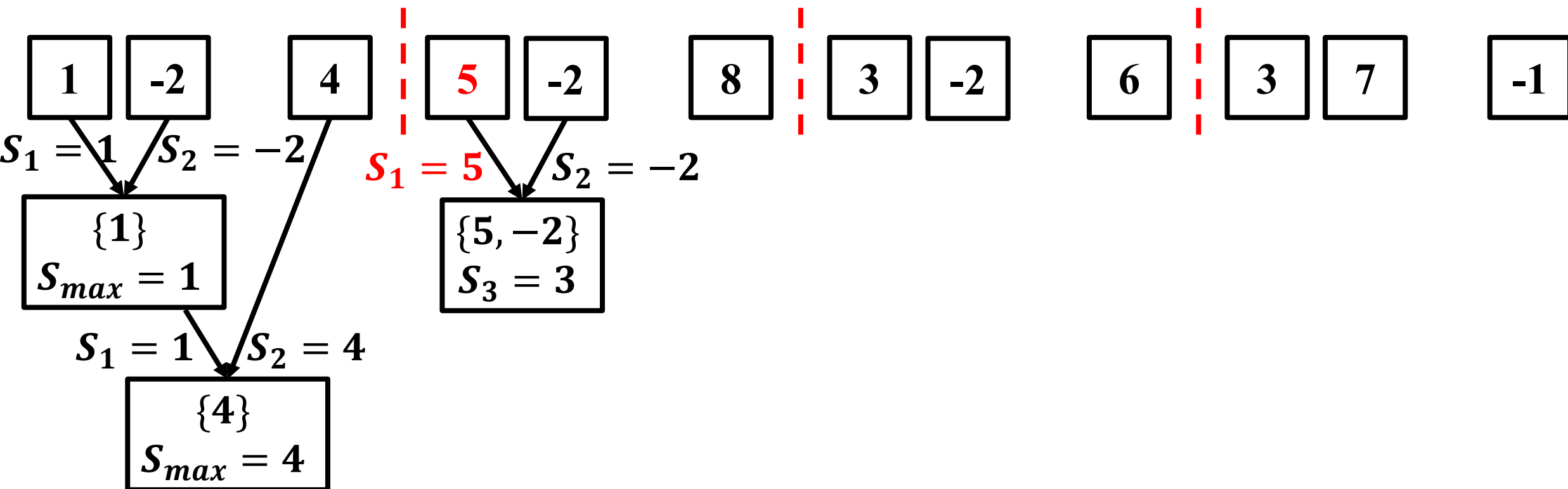
# 算法实例



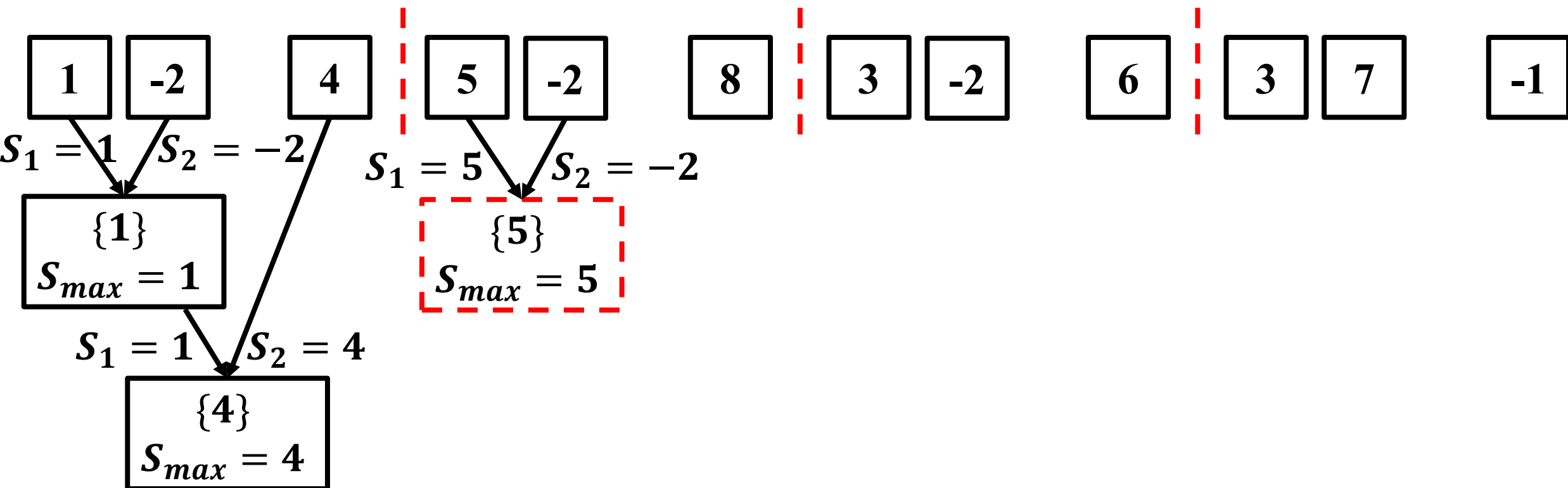
# 算法实例



# 算法实例

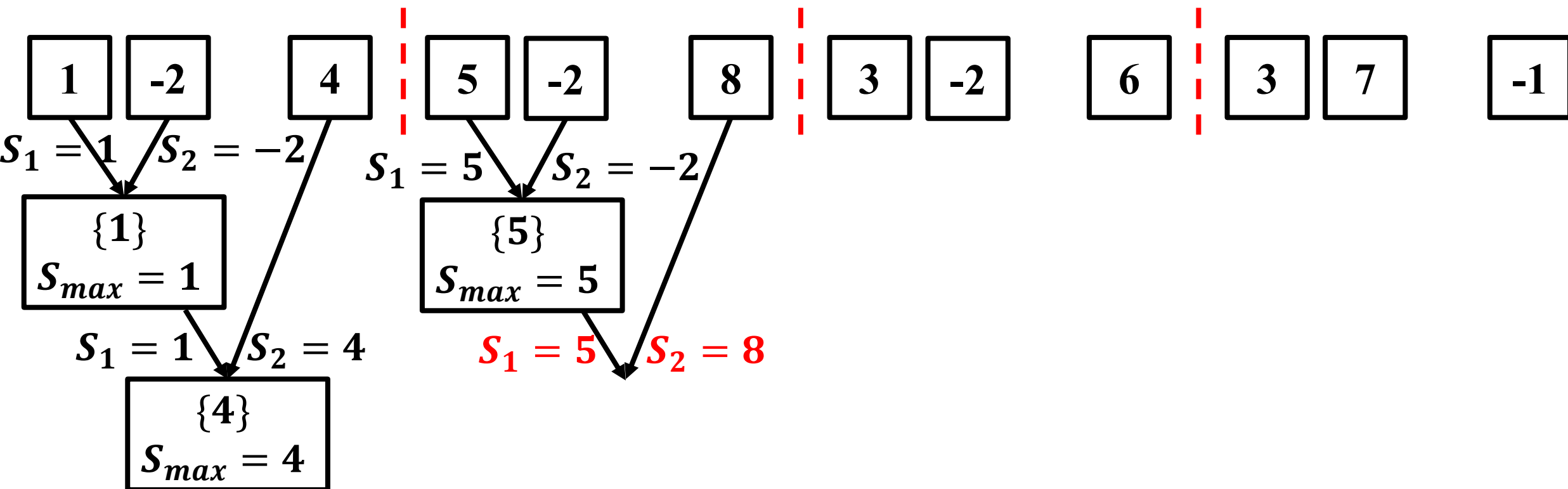


# 算法实例

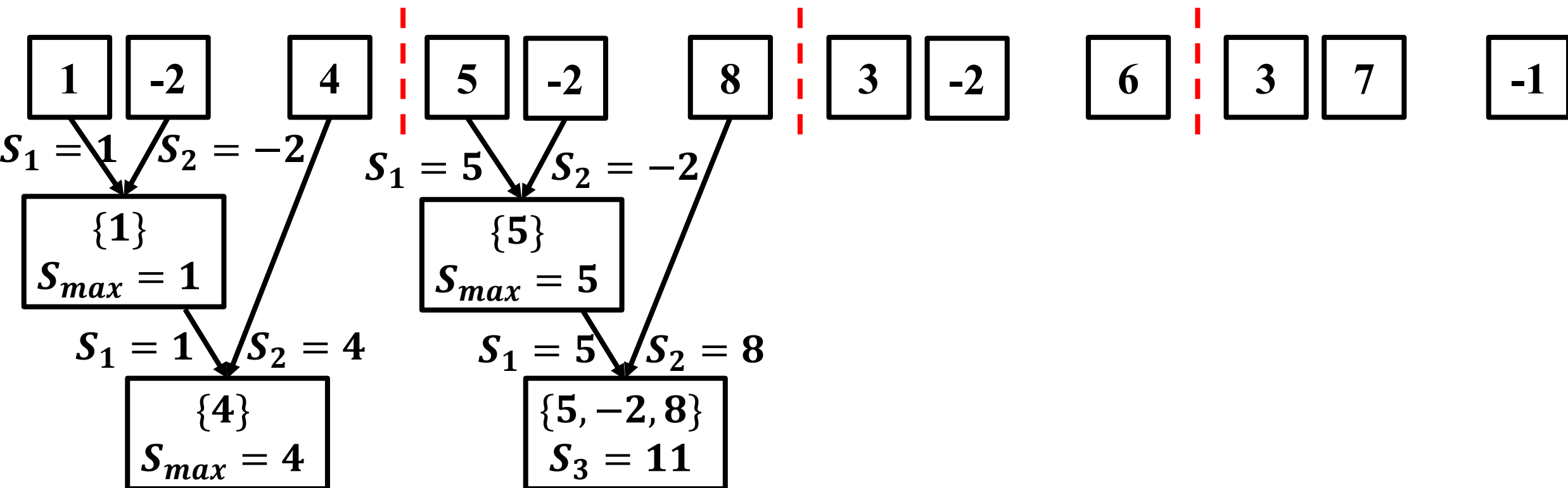




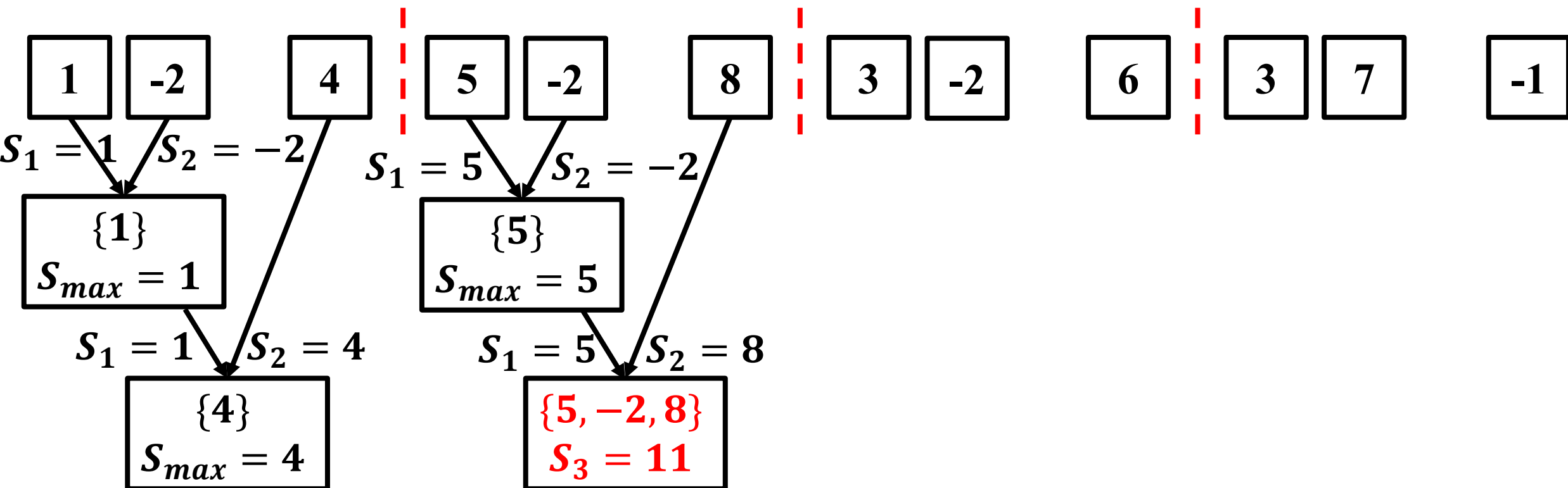
# 算法实例



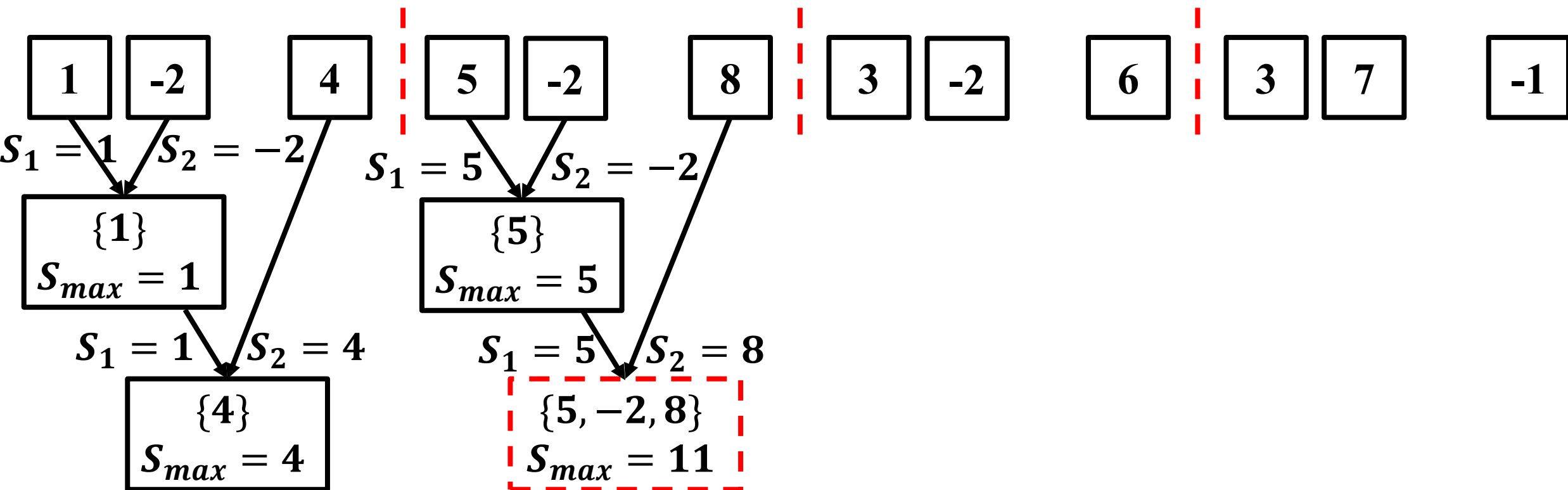
# 算法实例



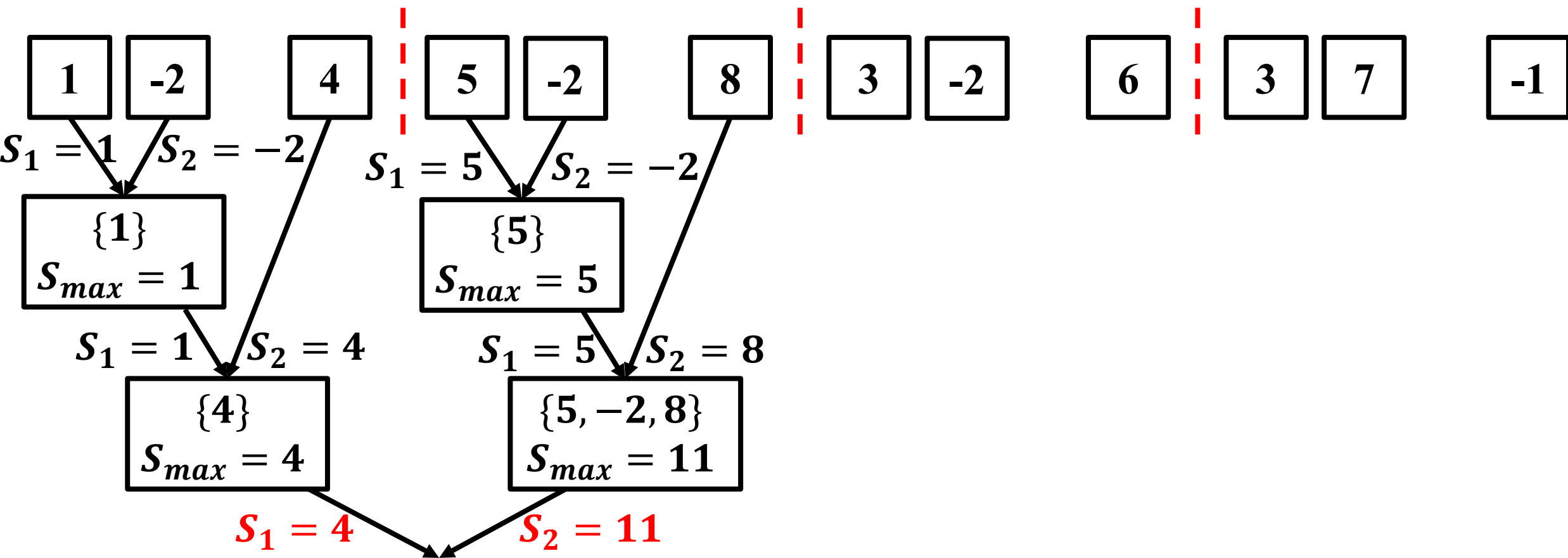
# 算法实例



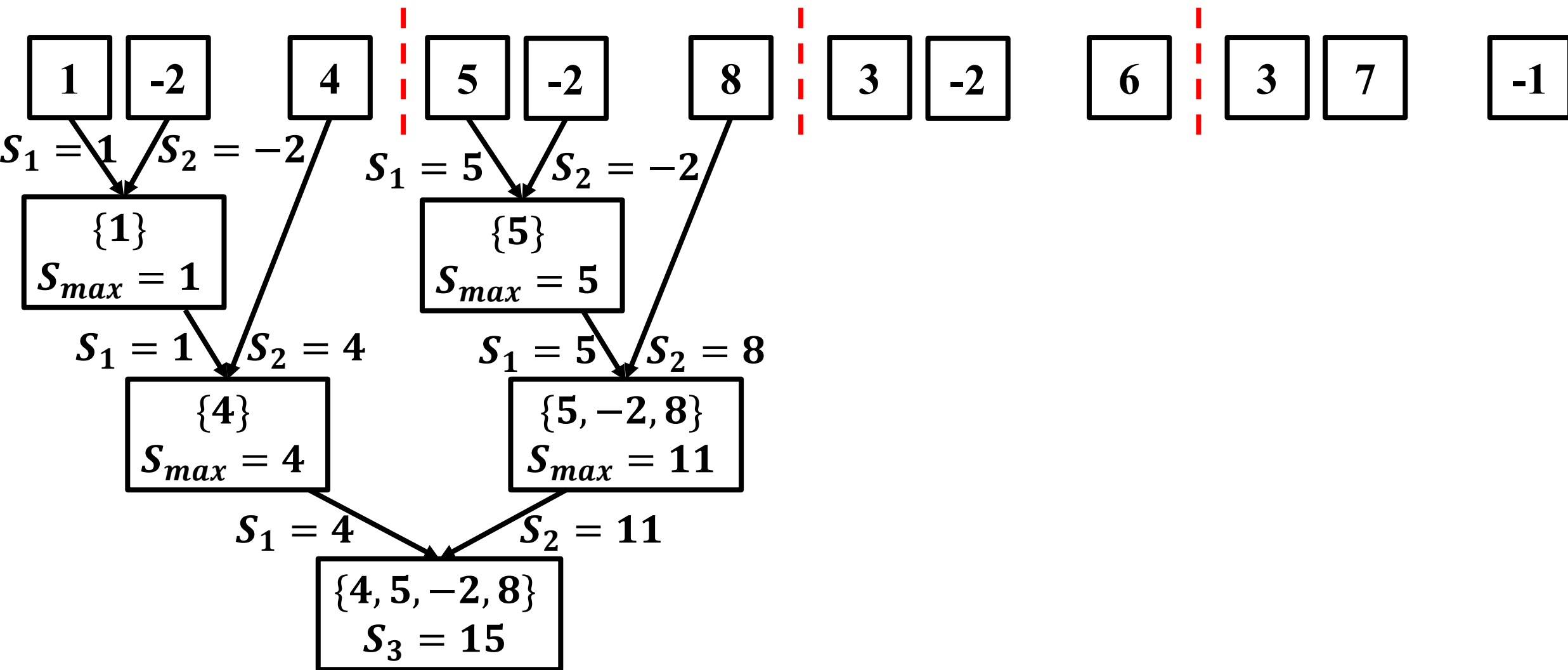
# 算法实例



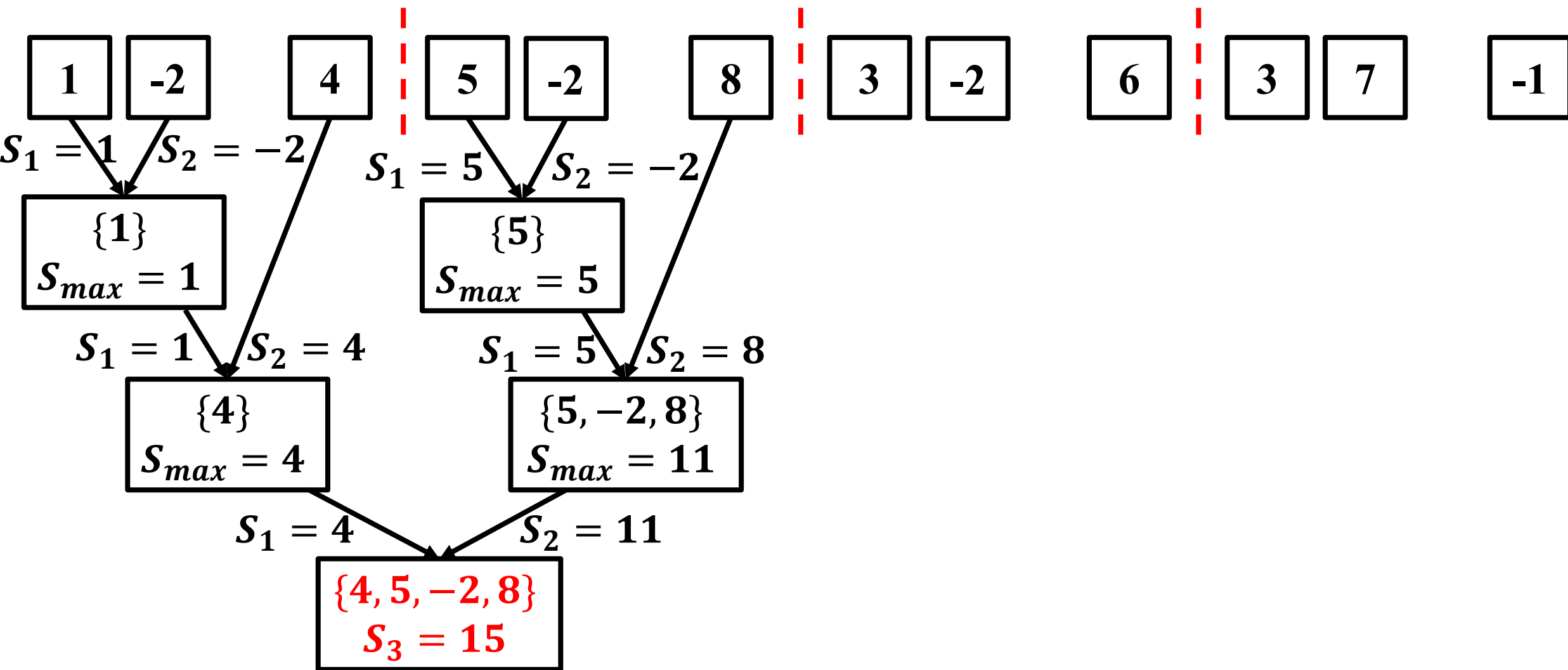
# 算法实例



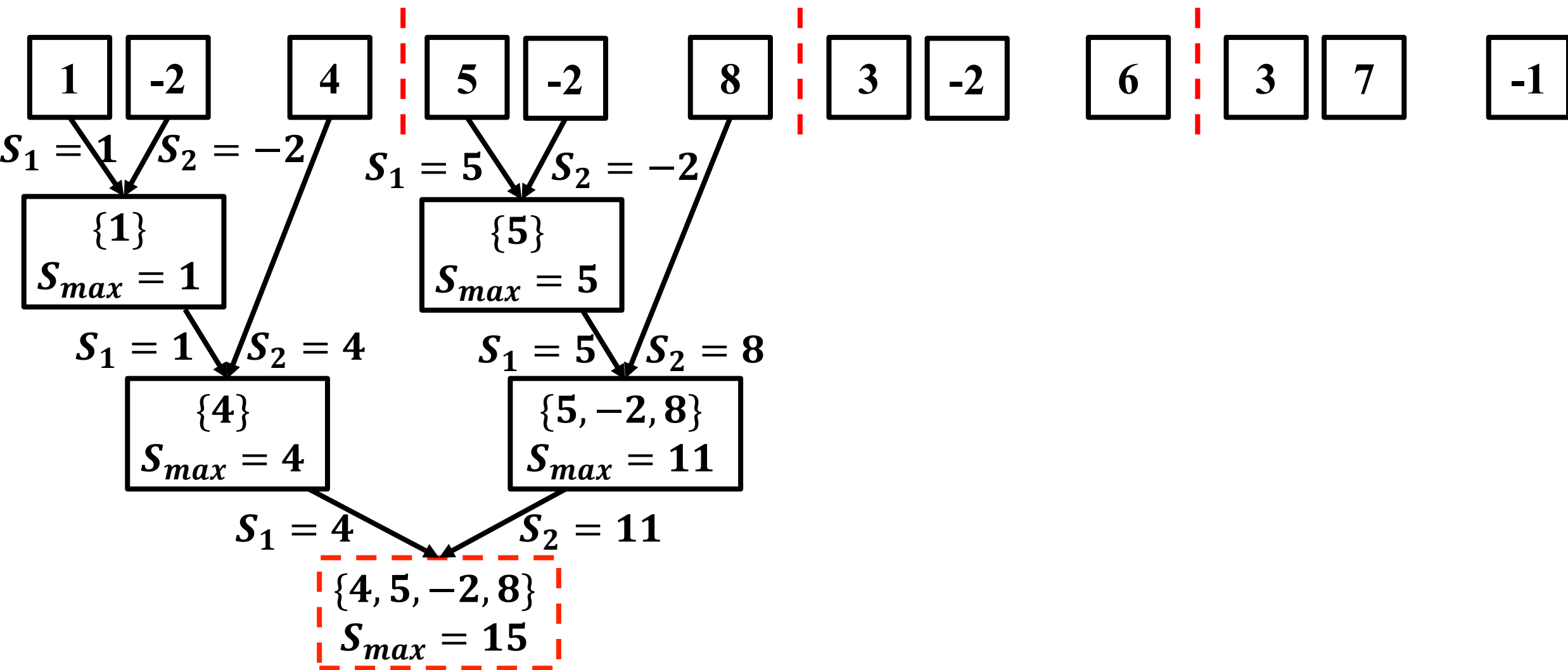
# 算法实例



# 算法实例

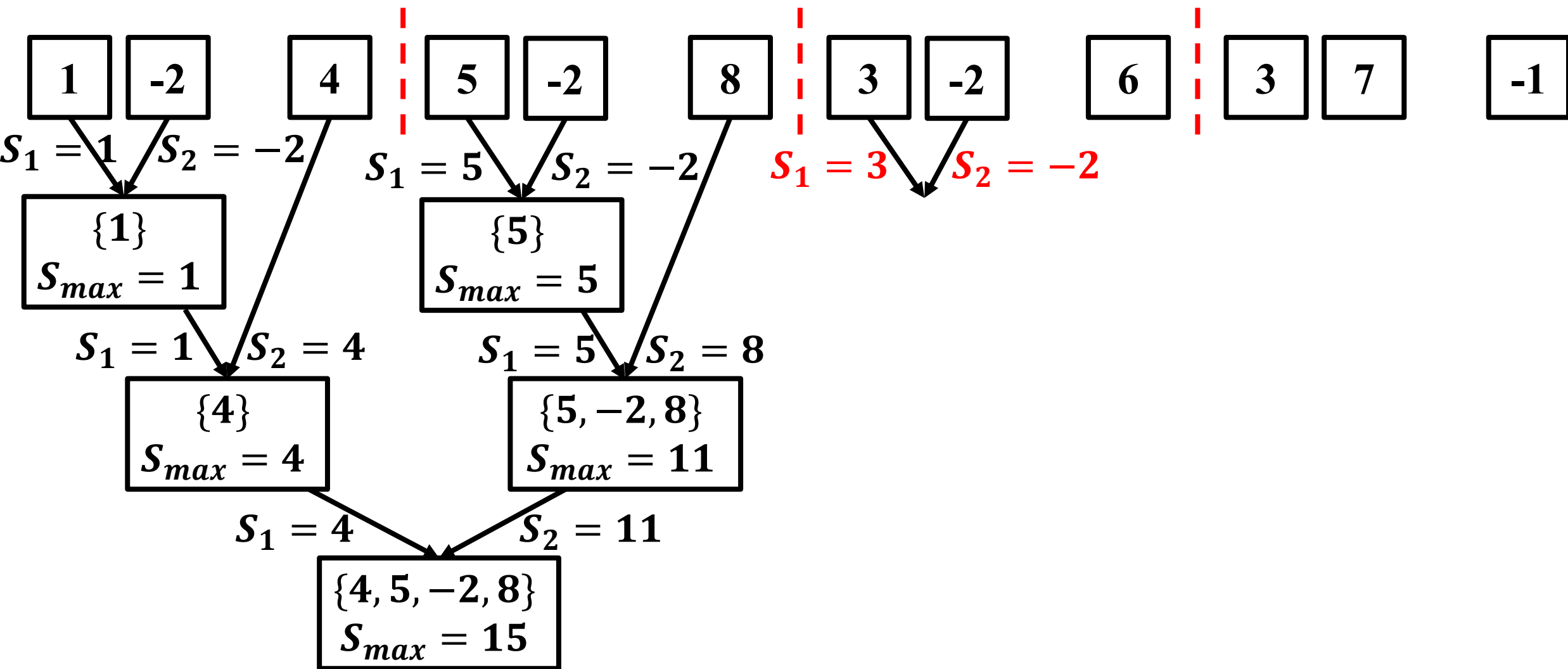


# 算法实例

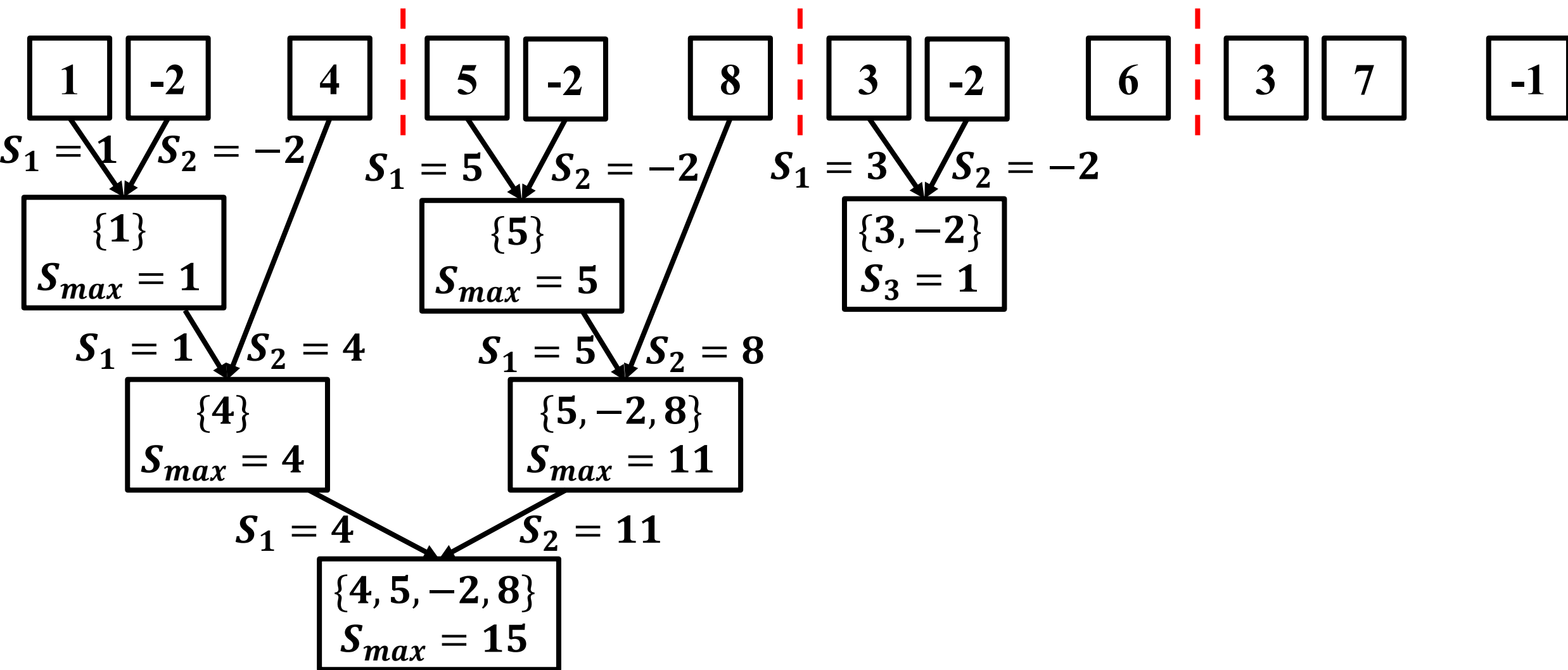




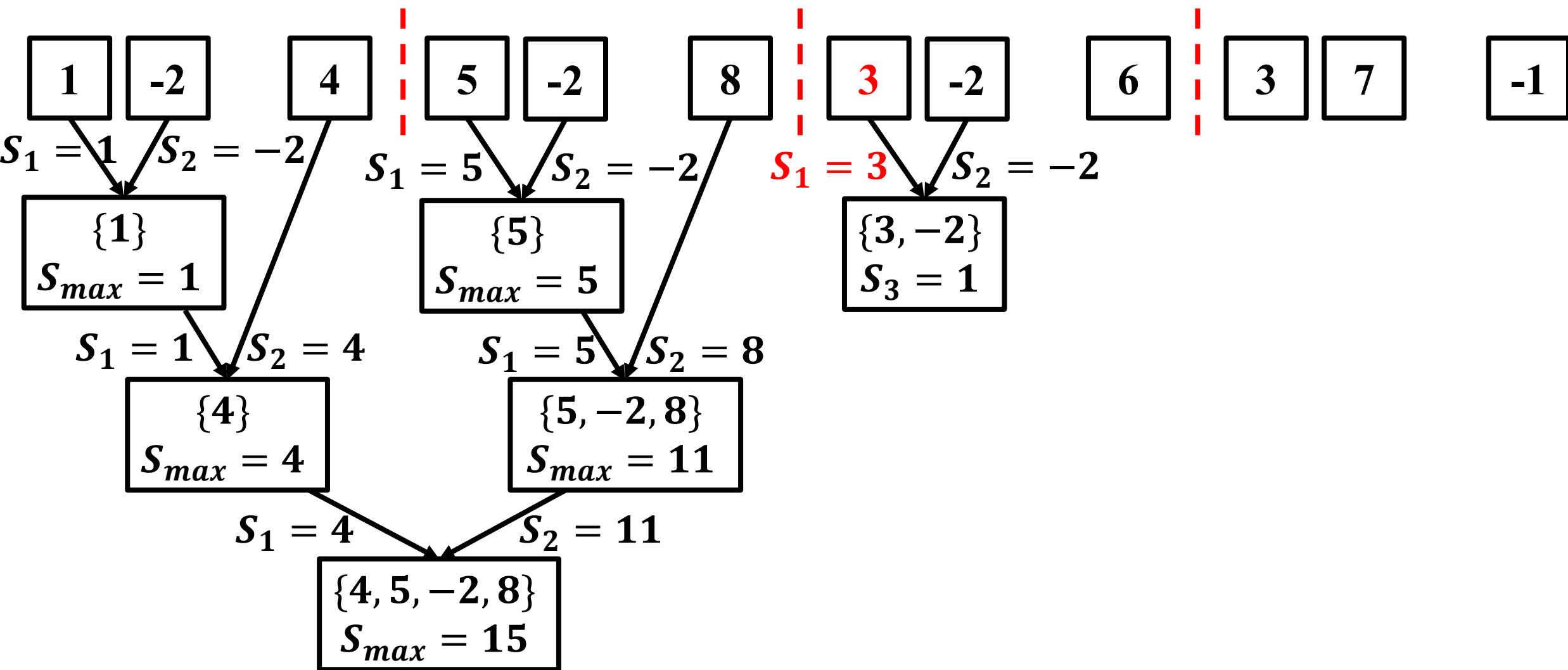
# 算法实例



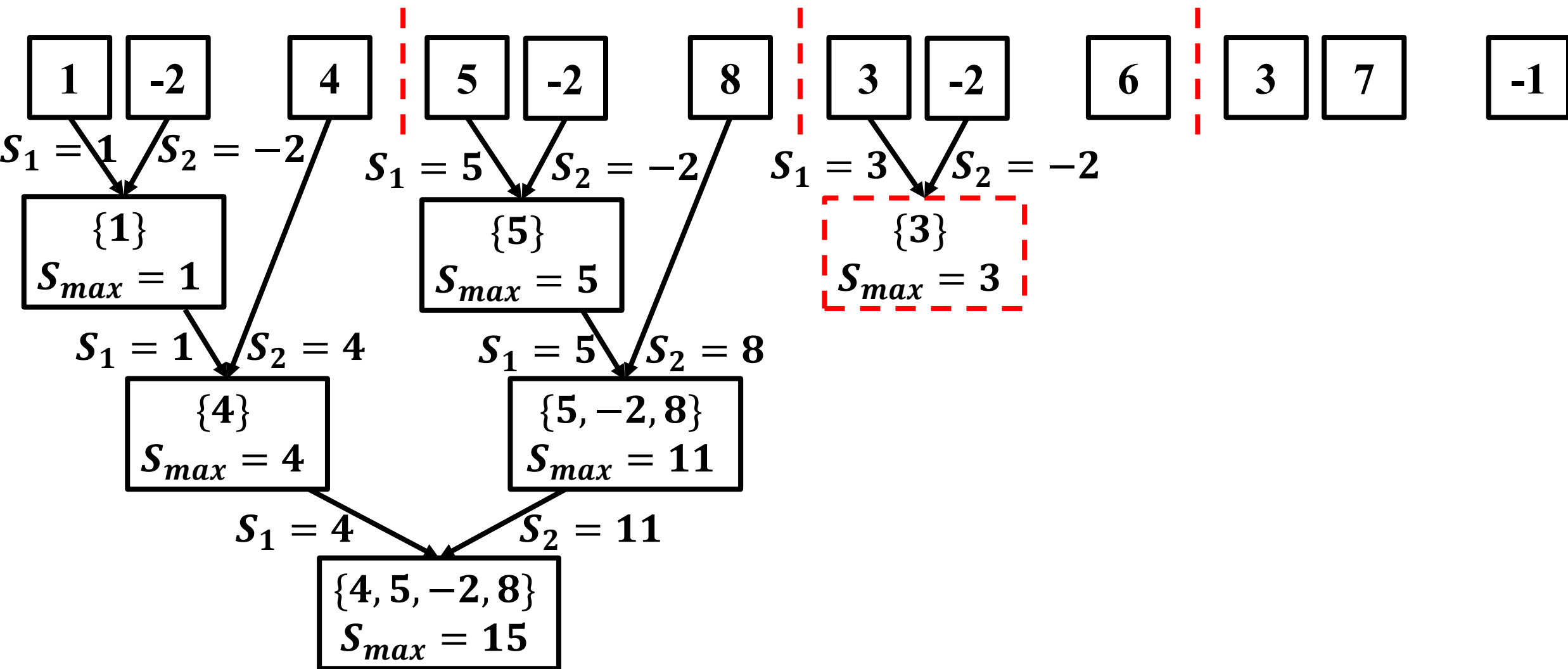
# 算法实例



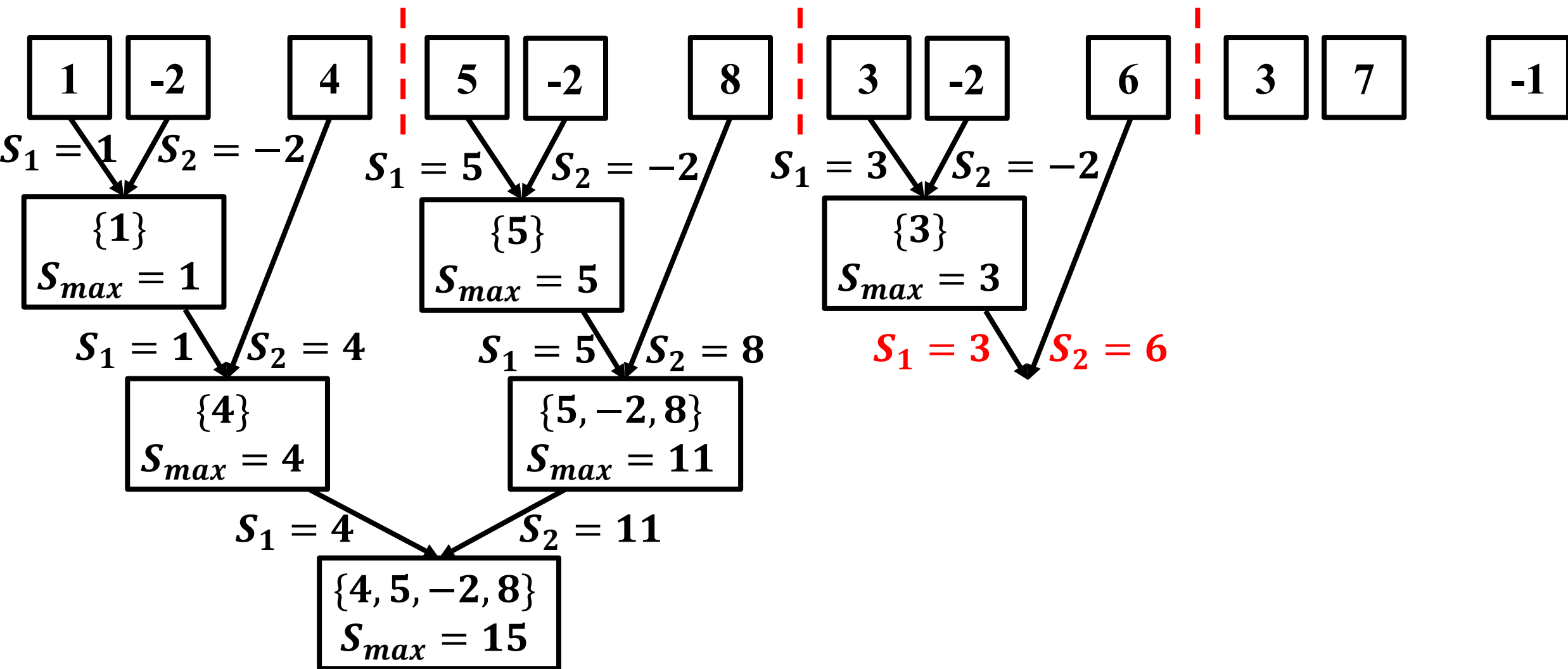
# 算法实例



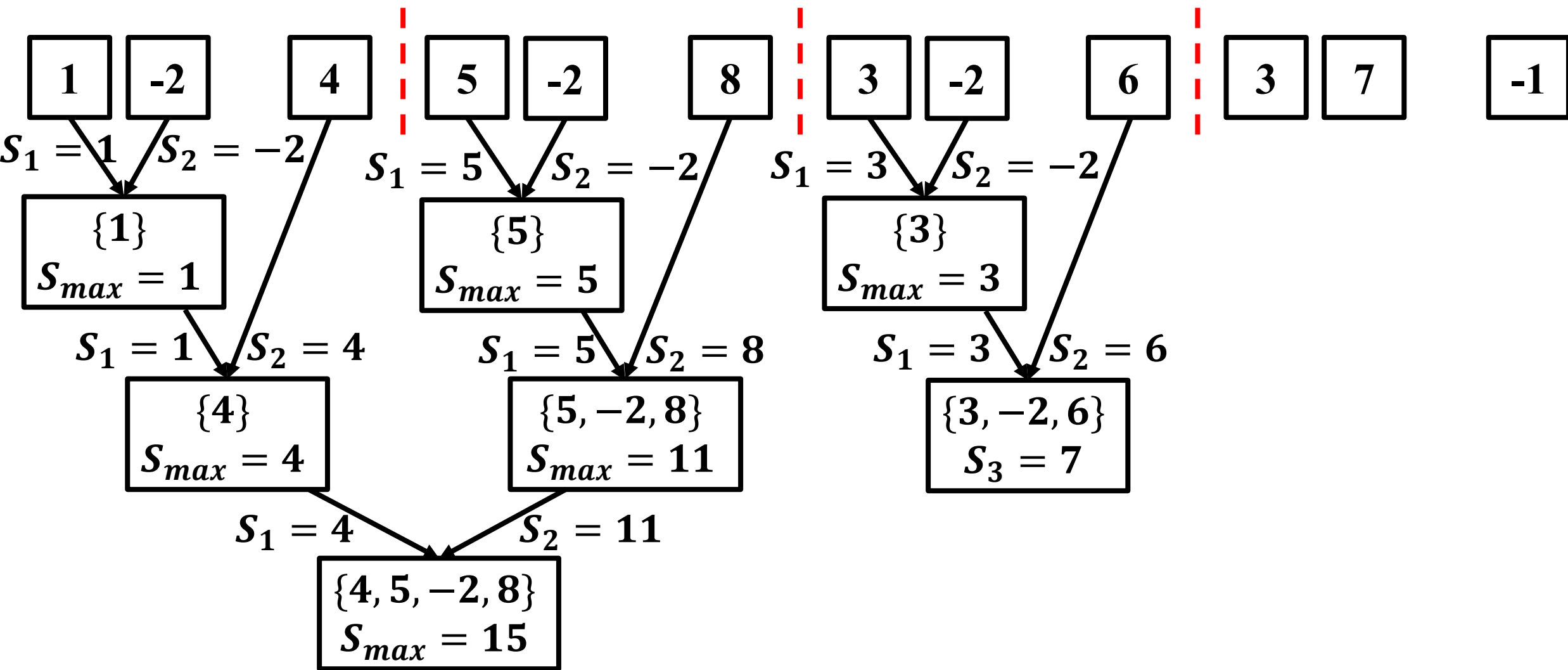
# 算法实例



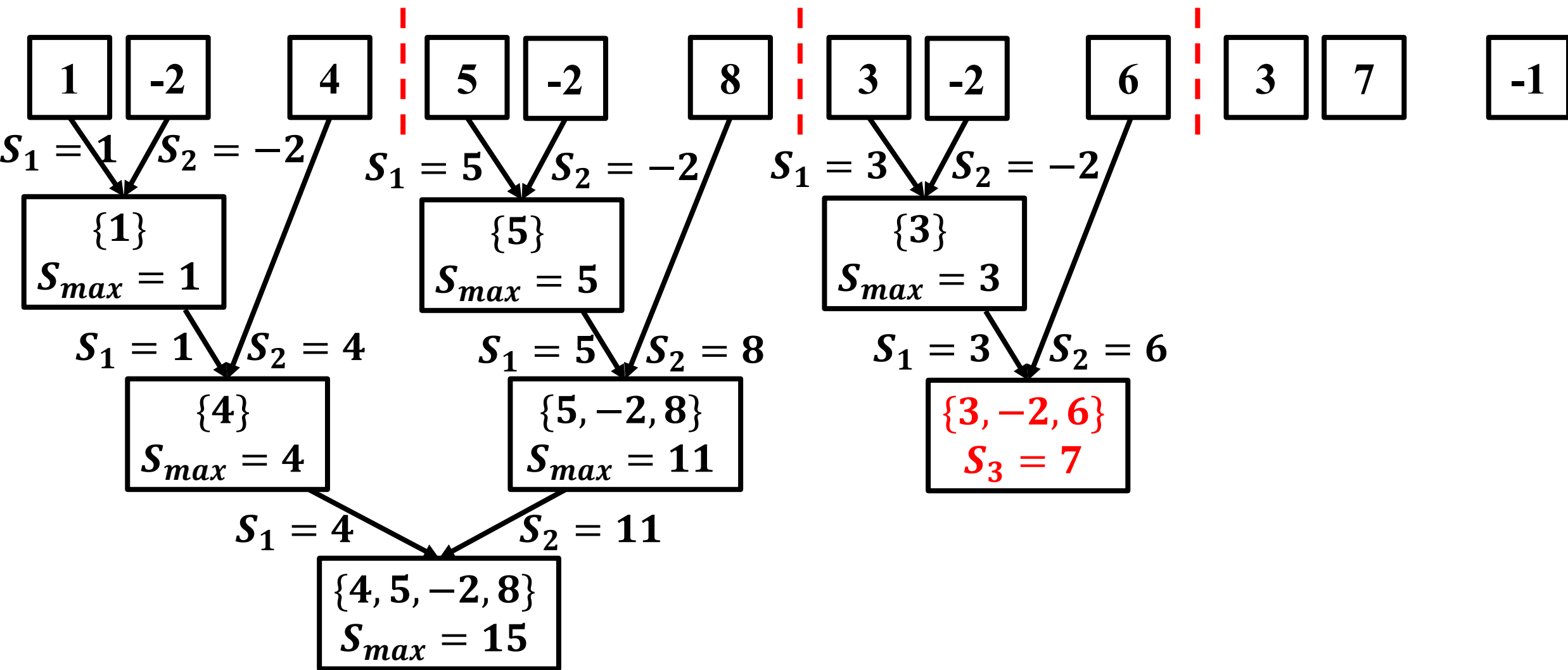
# 算法实例



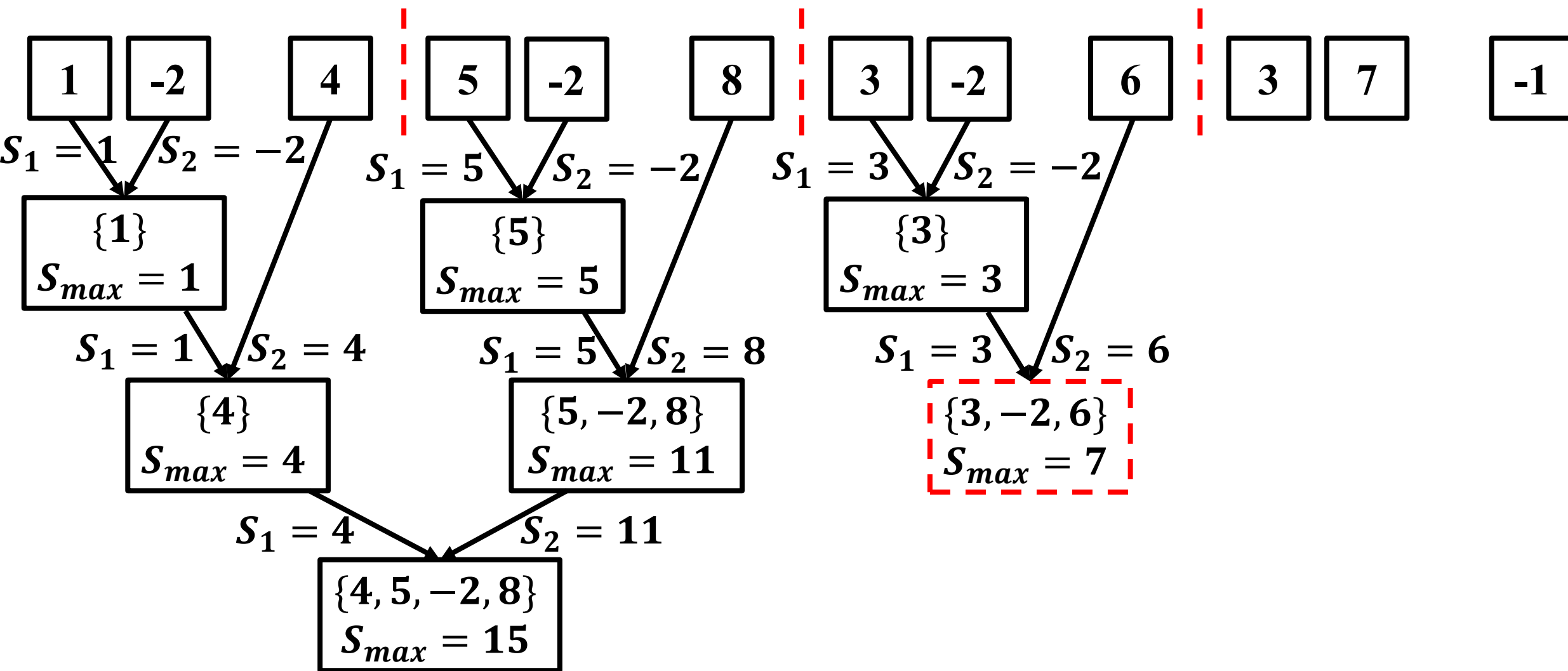
# 算法实例



# 算法实例

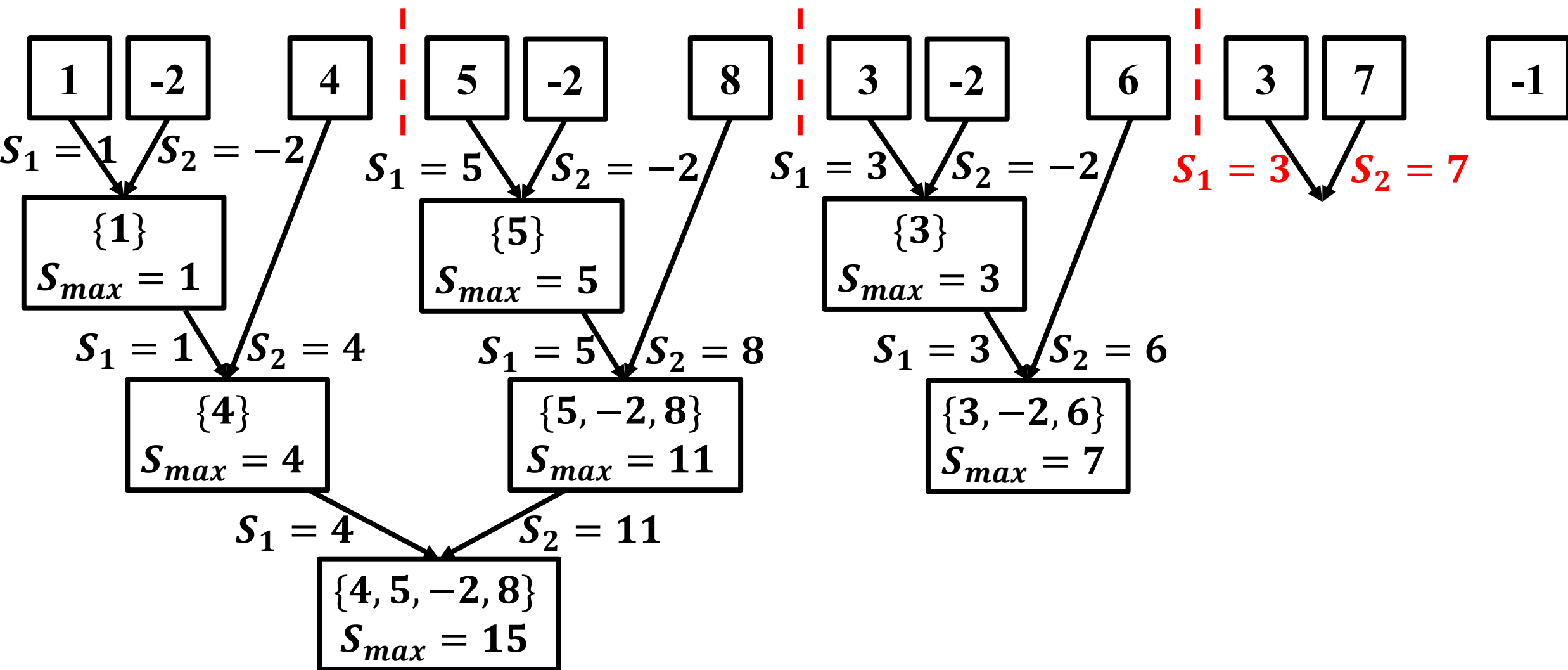


# 算法实例

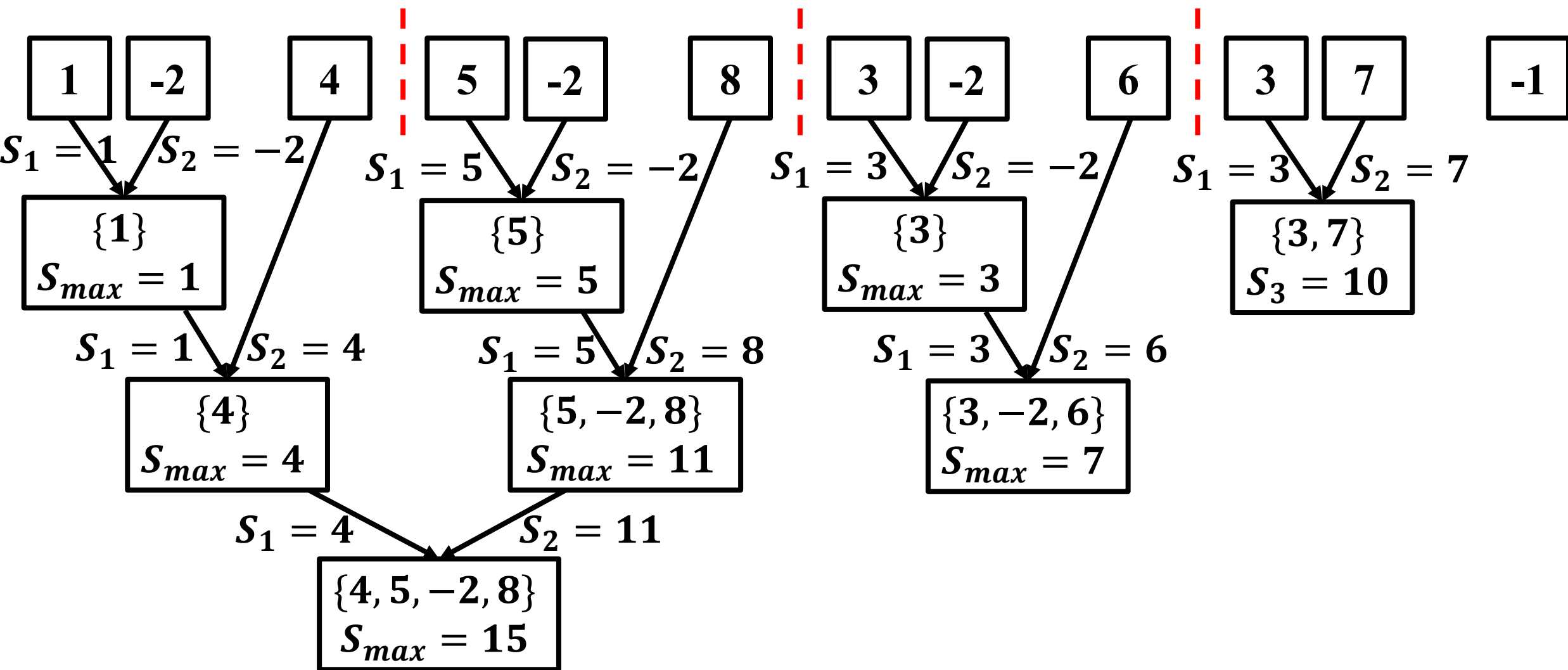




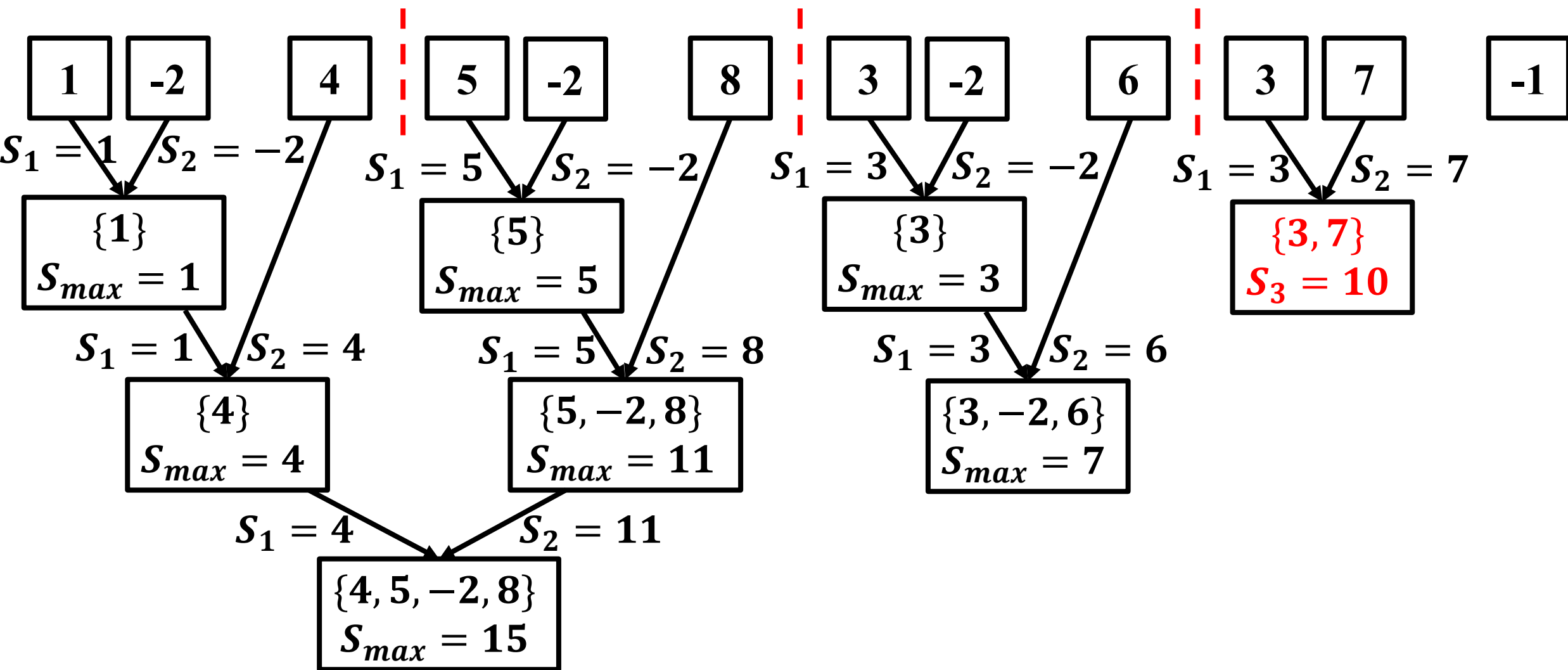
# 算法实例



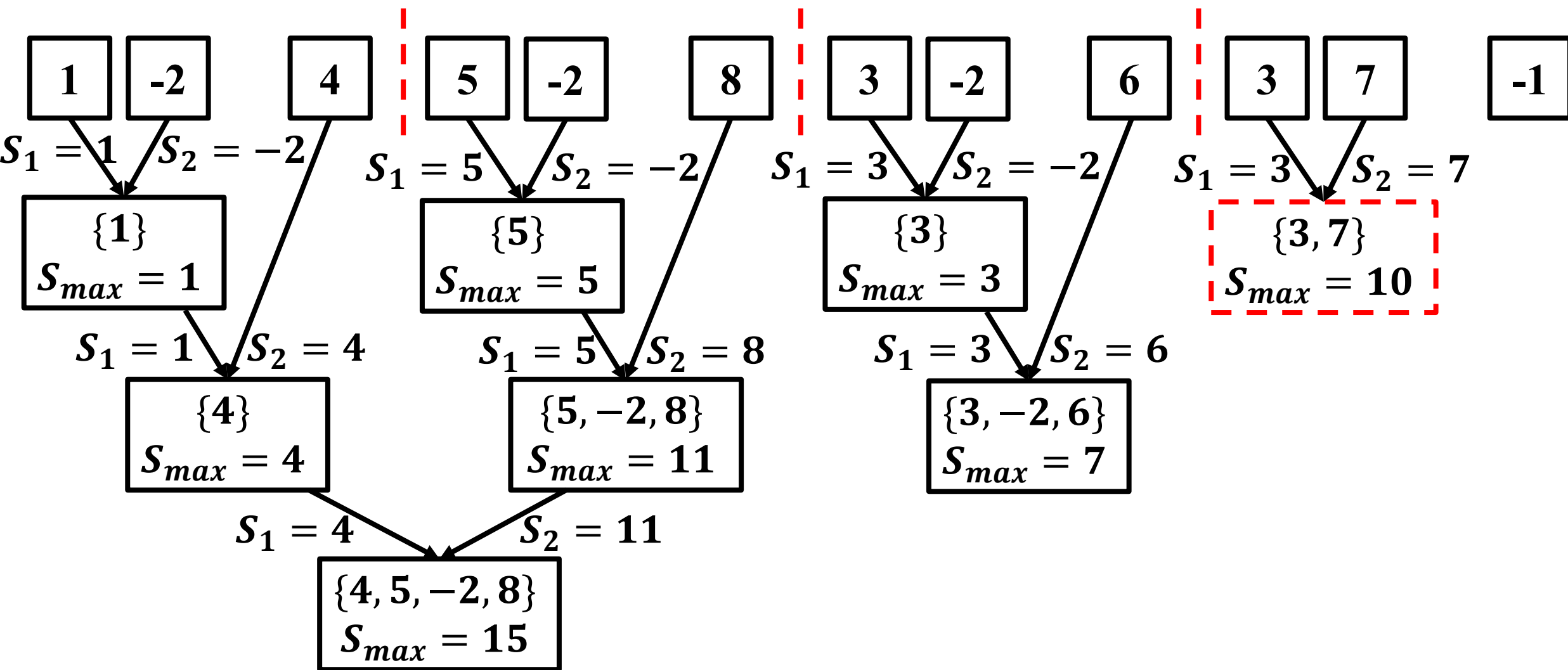
# 算法实例



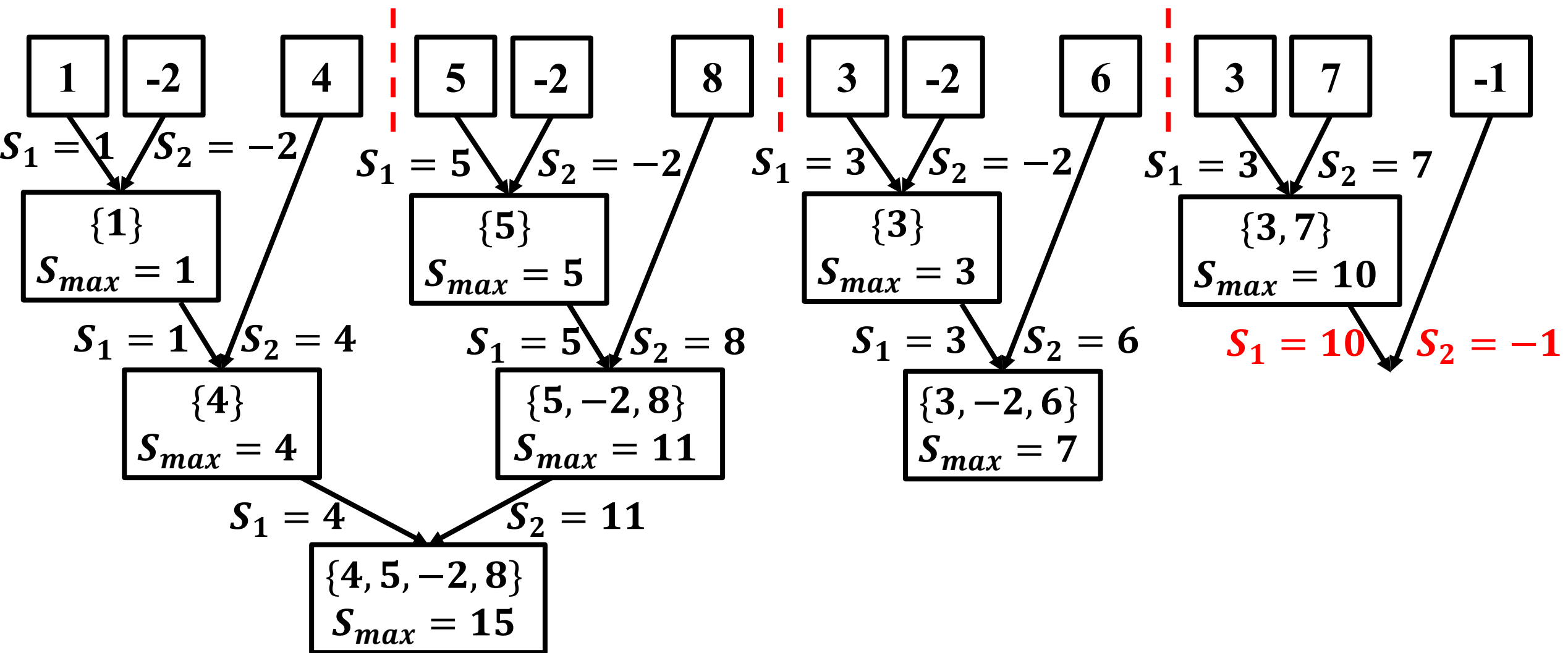
# 算法实例



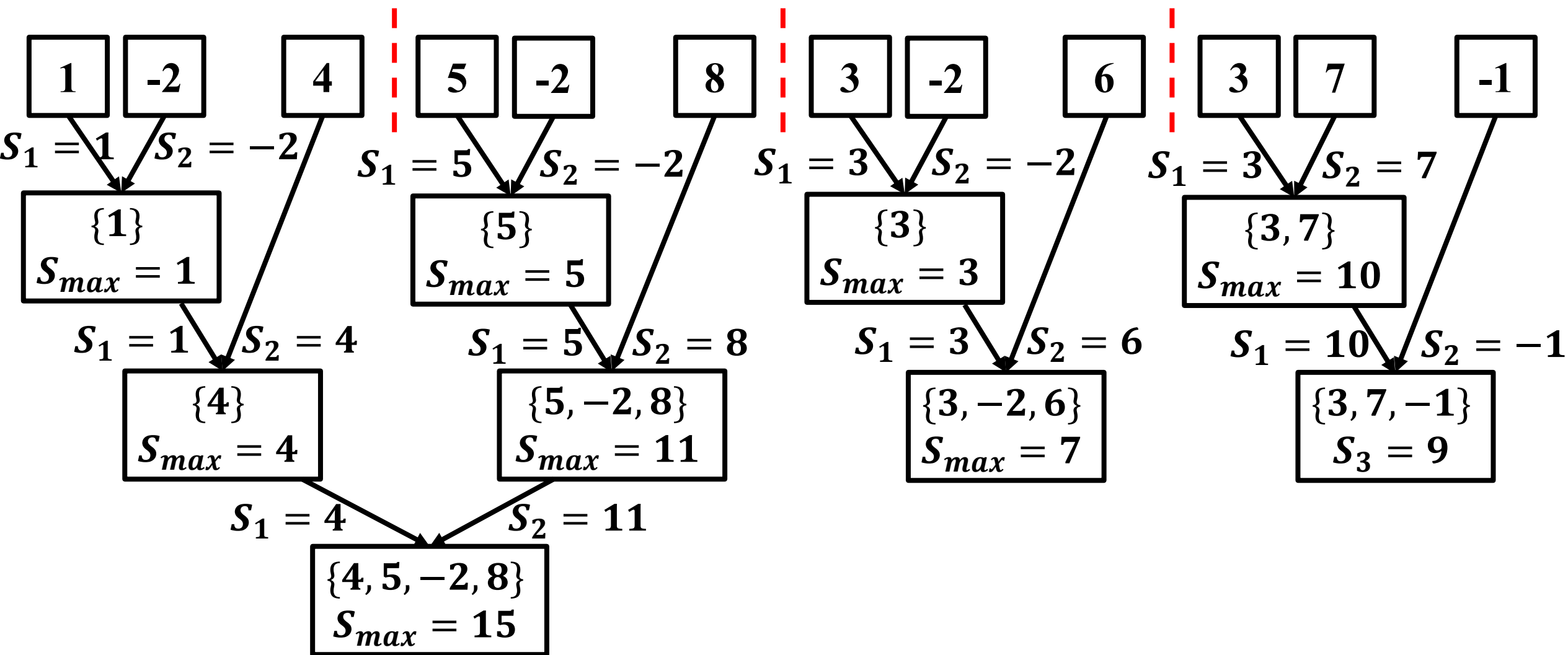
# 算法实例



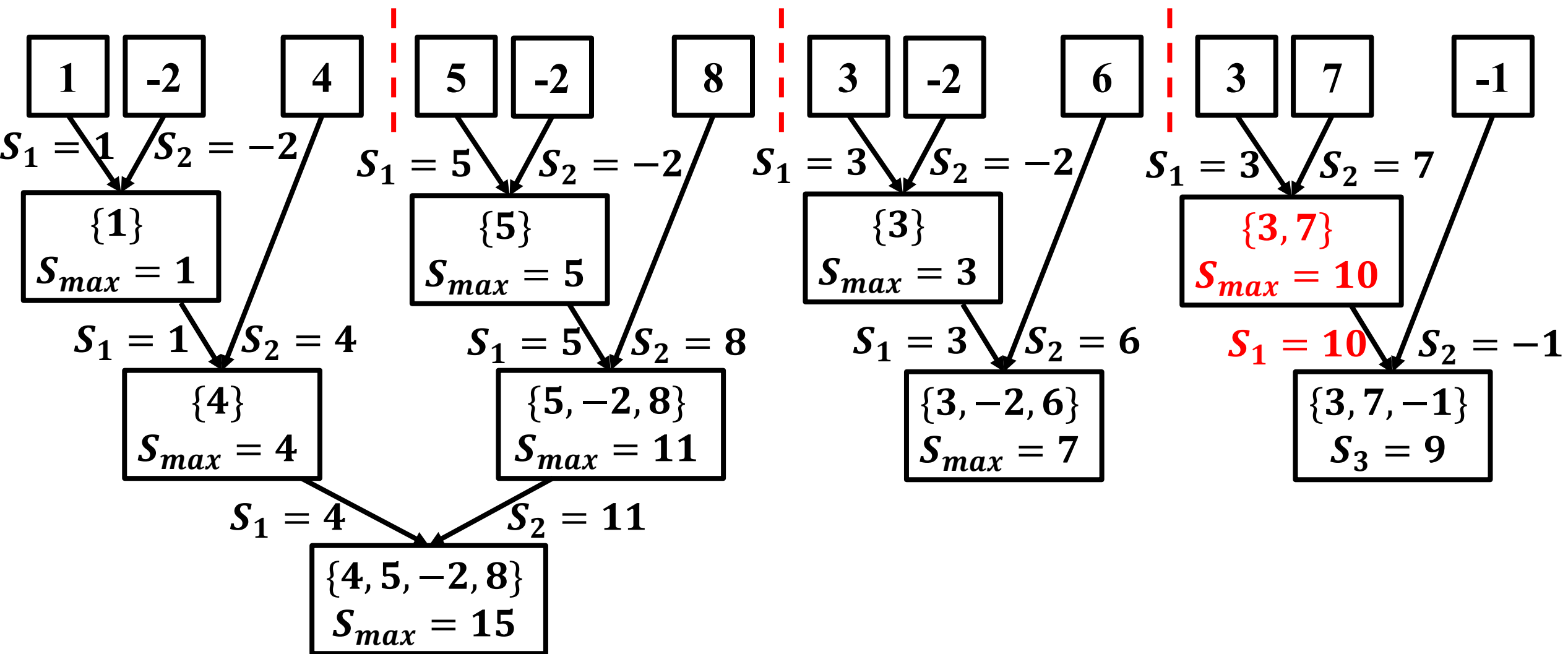
# 算法实例



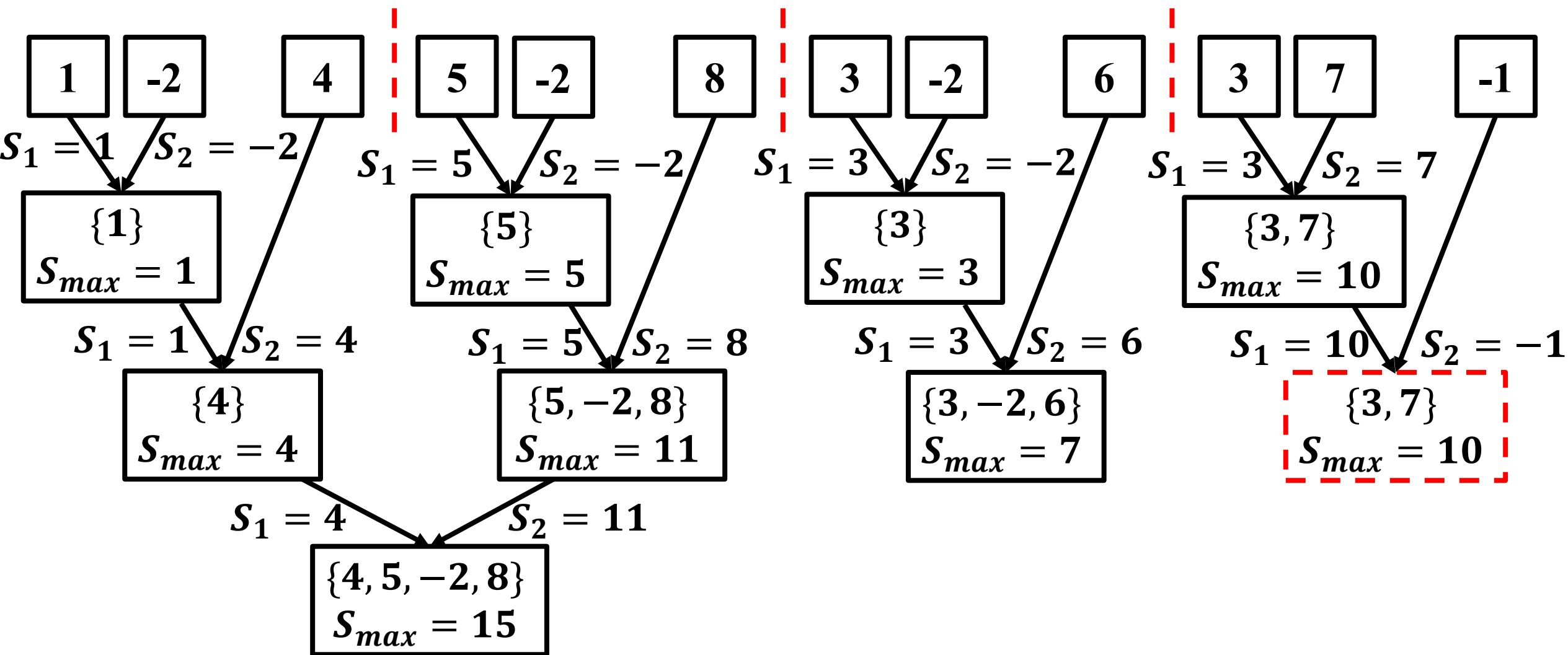
# 算法实例



# 算法实例

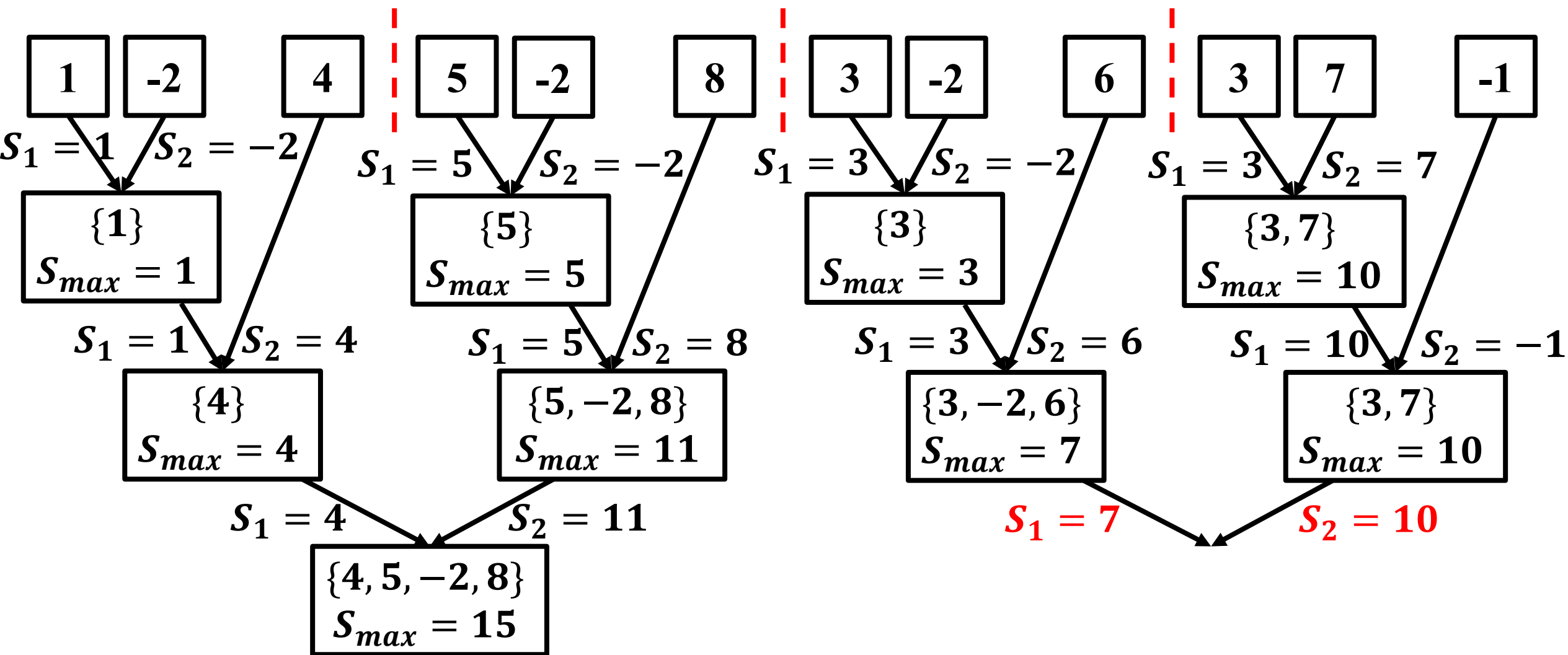


# 算法实例

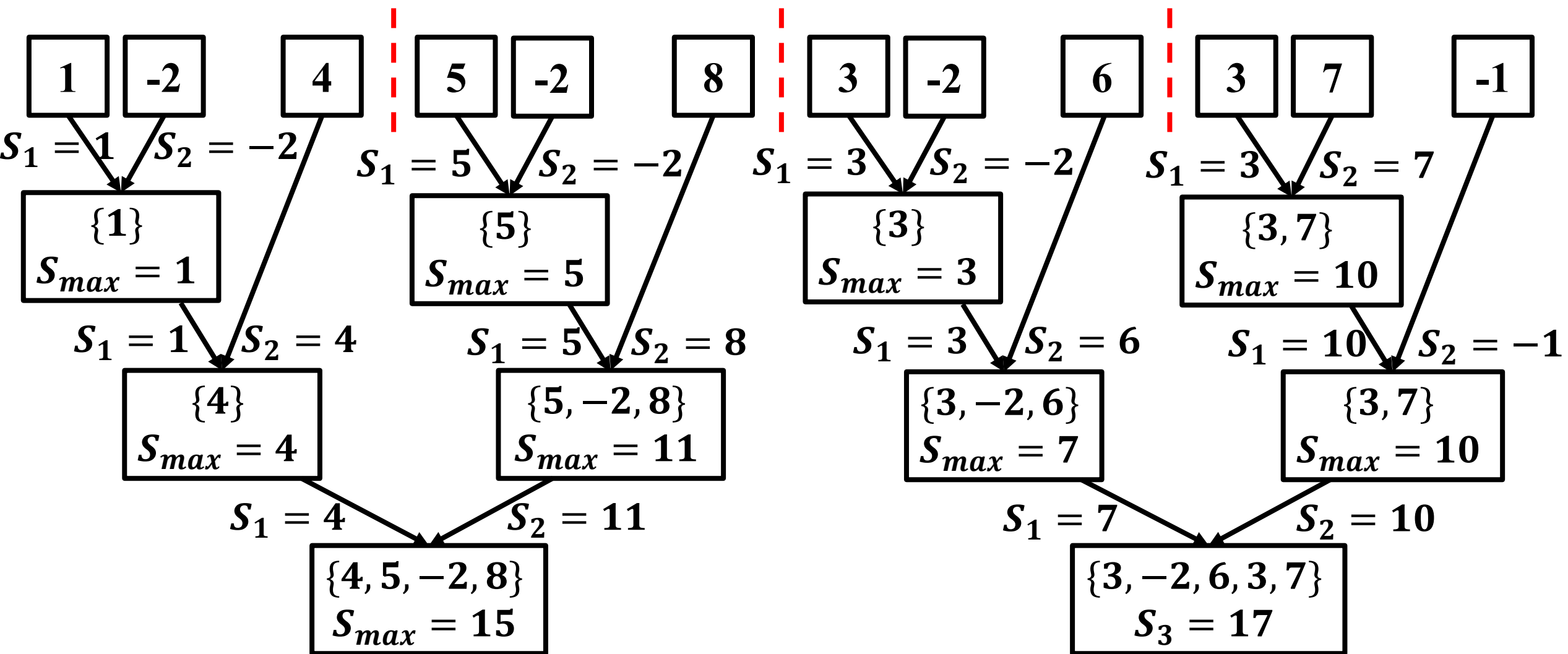




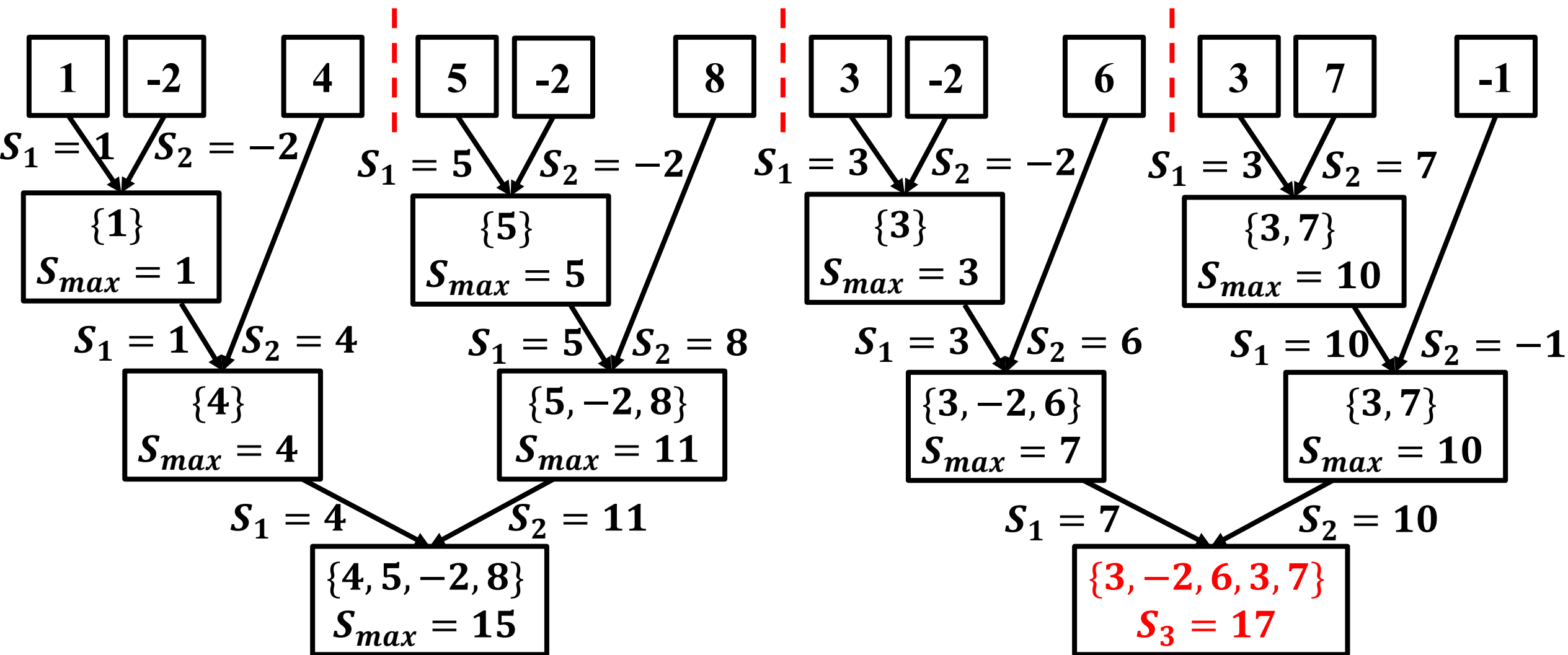
# 算法实例



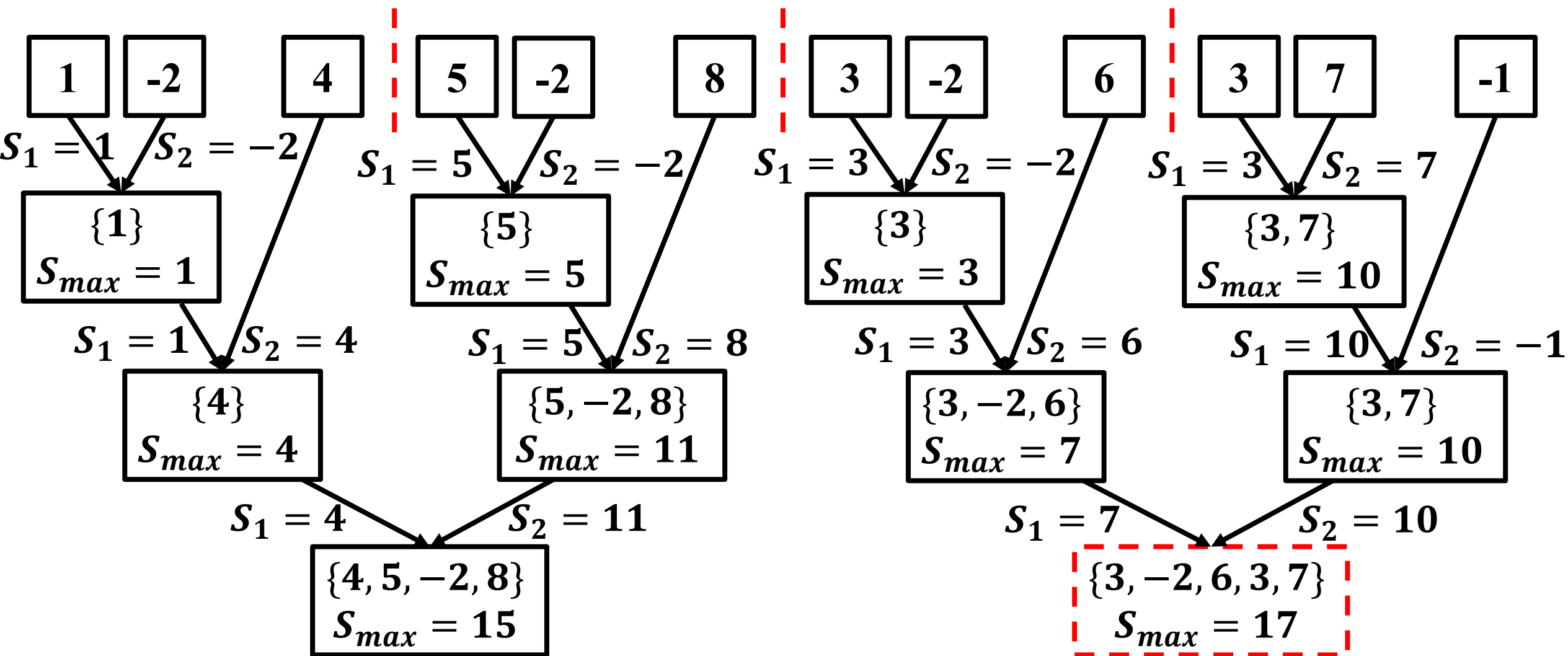
# 算法实例



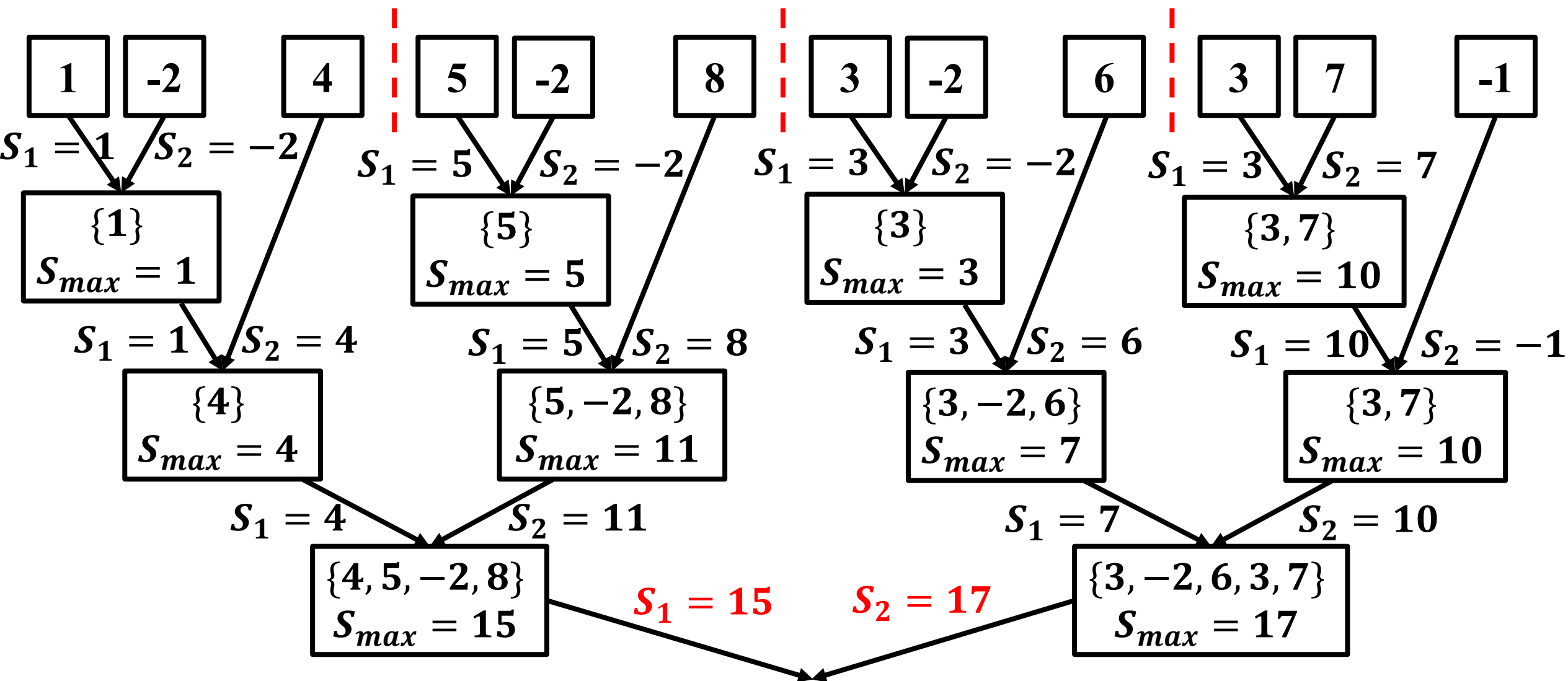
# 算法实例



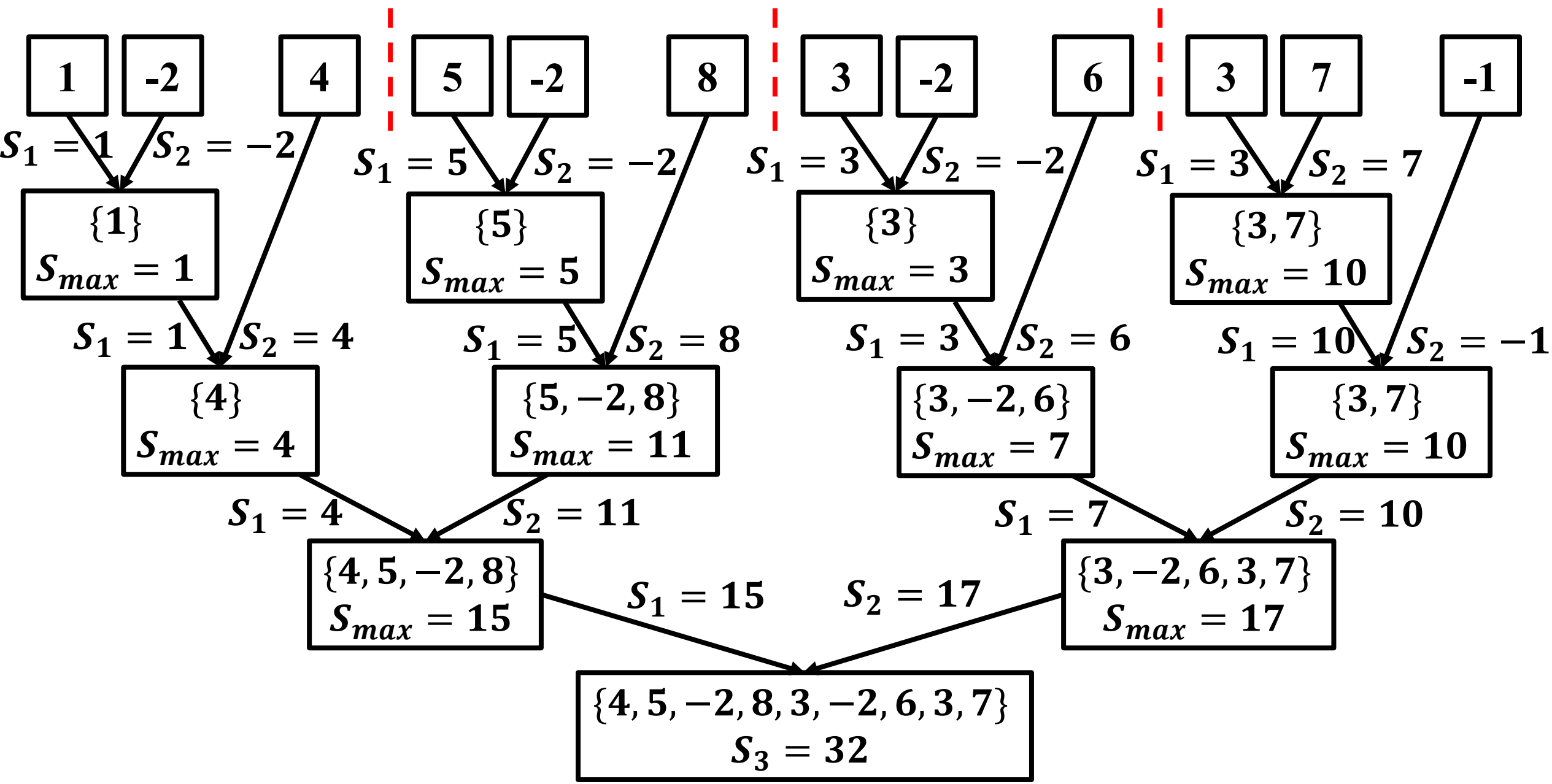
# 算法实例



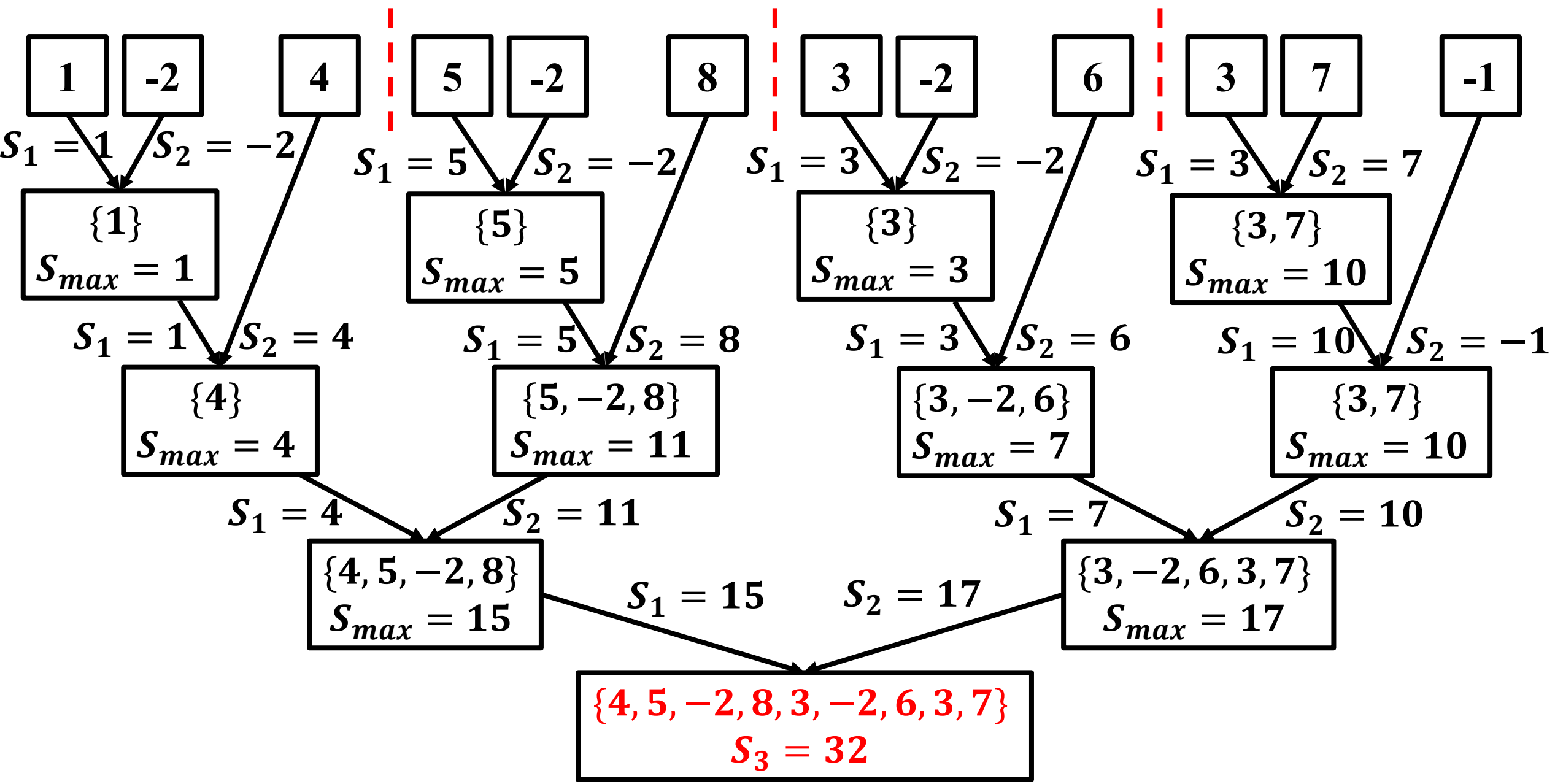
# 算法实例



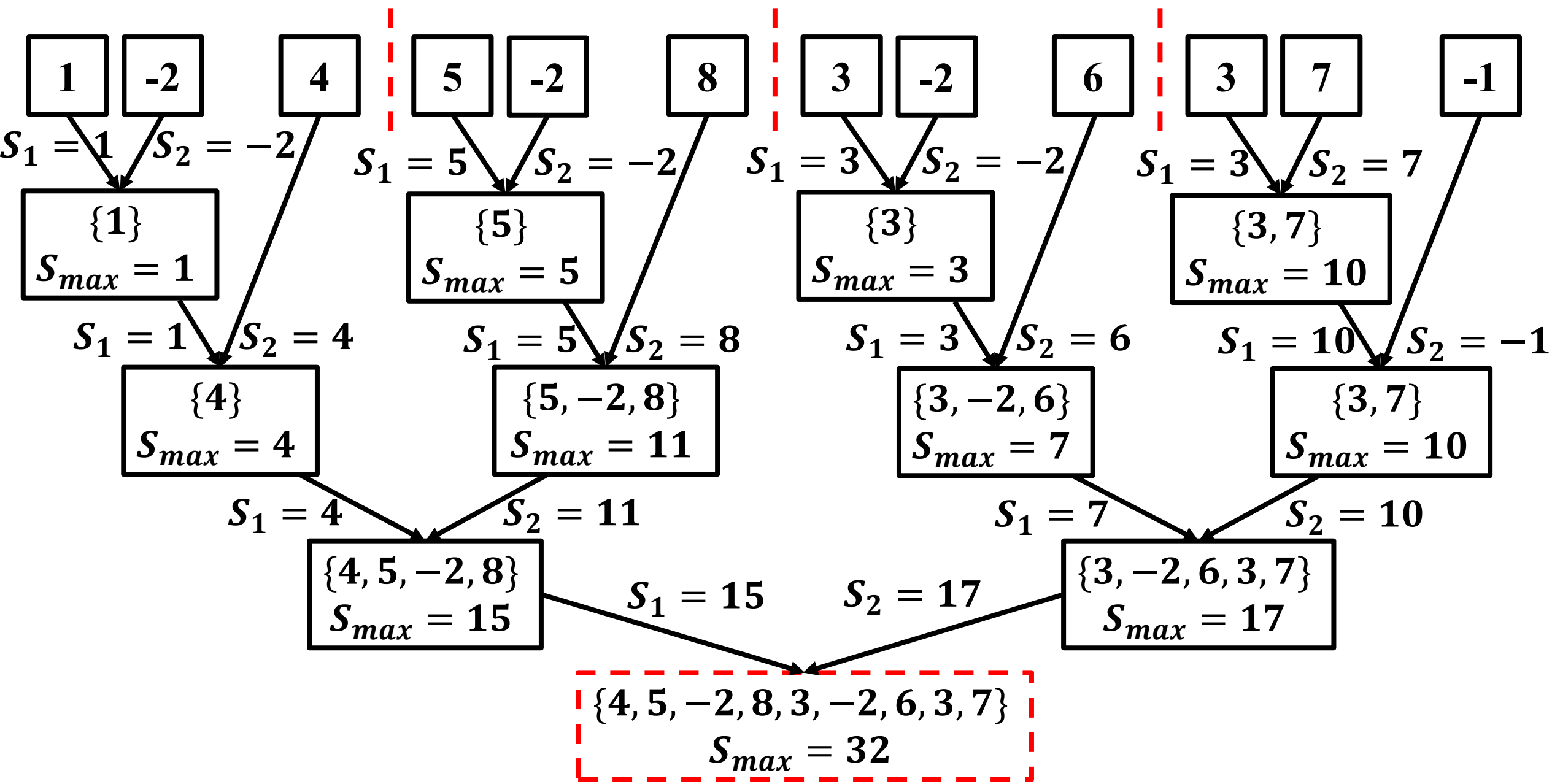
# 算法实例



# 算法实例



# 算法实例





- **MaxSubArray( $X, low, high$ )**

输入: 数组  $X$ , 数组下标  $low, high$

输出: 最大子数组之和  $S_{max}$

```
(if $low = high$ then
 | return $X[low]$
end
else
```

```
 $mid \leftarrow \lfloor \frac{low+high}{2} \rfloor$
```

```
 $S_1 \leftarrow \text{MaxSubArray}(X, low, mid)$
```

```
 $S_2 \leftarrow \text{MaxSubArray}(X, mid+1, high)$
```

```
 $S_3 \leftarrow \text{CrossingSubArray}(X, low, mid, high)$
```

```
 $S_{max} \leftarrow \max\{S_1, S_2, S_3\}$
```

```
 return S_{max}
```

```
end
```

递归终止条件

- **MaxSubArray( $X, low, high$ )**

输入: 数组  $X$ , 数组下标  $low, high$

输出: 最大子数组之和  $S_{max}$

if  $low = high$  then

    | return  $X[low]$

end

else

    |  $mid \leftarrow \lfloor \frac{low+high}{2} \rfloor$

    |  $S_1 \leftarrow \text{MaxSubArray}(X, low, mid)$

    |  $S_2 \leftarrow \text{MaxSubArray}(X, mid+1, high)$

    |  $S_3 \leftarrow \text{CrossingSubArray}(X, low, mid, high)$

    |  $S_{max} \leftarrow \max\{S_1, S_2, S_3\}$

    | return  $S_{max}$

end

拆分原问题

- **MaxSubArray( $X, low, high$ )**

输入: 数组  $X$ , 数组下标  $low, high$

输出: 最大子数组之和  $S_{max}$

if  $low = high$  then

    | return  $X[low]$

end

else

    |  $mid \leftarrow \lfloor \frac{low+high}{2} \rfloor$

    |  $S_1 \leftarrow \text{MaxSubArray}(X, low, mid)$

    |  $S_2 \leftarrow \text{MaxSubArray}(X, mid+1, high)$

    |  $S_3 \leftarrow \text{CrossingSubArray}(X, low, mid, high)$

    |  $S_{max} \leftarrow \max\{S_1, S_2, S_3\}$

    | return  $S_{max}$

end

求解子问题

- **MaxSubArray( $X, low, high$ )**

输入: 数组  $X$ , 数组下标  $low, high$

输出: 最大子数组之和  $S_{max}$

if  $low = high$  then

    | return  $X[low]$

end

else

$mid \leftarrow \lfloor \frac{low+high}{2} \rfloor$

$S_1 \leftarrow \text{MaxSubArray}(X, low, mid)$

$S_2 \leftarrow \text{MaxSubArray}(X, mid+1, high)$

$S_3 \leftarrow \text{CrossingSubArray}(X, low, mid, high)$

$S_{max} \leftarrow \max\{S_1, S_2, S_3\}$

    return  $S_{max}$

end

合并问题解

- **MaxSubArray( $X, low, high$ )**

初始调用: MaxSubArray( $X, 1, n$ )

输入: 数组  $X$ , 数组下标  $low, high$

输出: 最大子数组之和  $S_{max}$

if  $low = high$  then

    | return  $X[low]$

end

else

    |  $mid \leftarrow \lfloor \frac{low+high}{2} \rfloor$

    |  $S_1 \leftarrow \text{MaxSubArray}(X, low, mid)$

    |  $S_2 \leftarrow \text{MaxSubArray}(X, mid+1, high)$

    |  $S_3 \leftarrow \text{CrossingSubArray}(X, low, mid, high)$

    |  $S_{max} \leftarrow \max\{S_1, S_2, S_3\}$

    | return  $S_{max}$

end

# 时间复杂度分析



- 输入规模为:  $n$

- $T(n) = \begin{cases} 1, & n = 1 \\ 2 \cdot T(n/2) + O(n), & n > 1 \end{cases}$

输入: 数组  $X$ , 数组下标  $low, high$

输出: 最大子数组之和  $S_{max}$

if  $low = high$  then

    | return  $X[low]$

end

else

    |  $mid \leftarrow \lfloor \frac{low+high}{2} \rfloor$

    |  $S_1 \leftarrow \text{MaxSubArray}(X, low, mid)$

    |  $S_2 \leftarrow \text{MaxSubArray}(X, mid+1, high)$

    |  $S_3 \leftarrow \text{CrossingSubArray}(X, low, mid, high)$

    |  $S_{max} \leftarrow \max\{S_1, S_2, S_3\}$

    | return  $S_{max}$

end

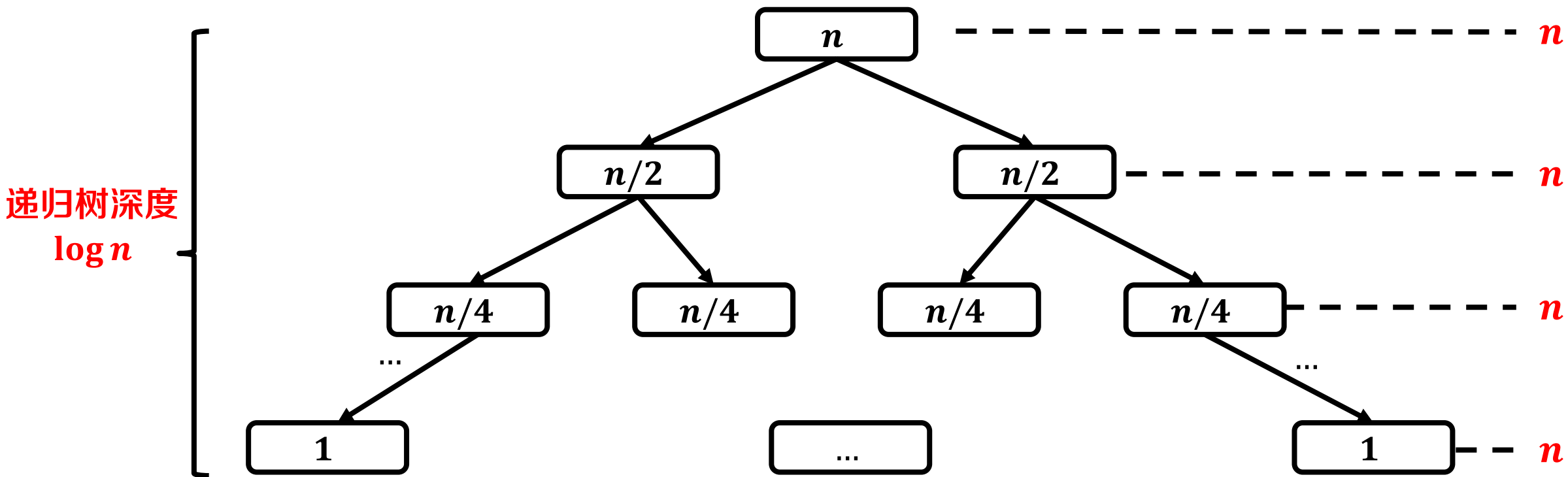
}  $2 \cdot T(n/2)$   
 $O(n)$

# 时间复杂度分析



- 输入规模为:  $n$

- $$T(n) = \begin{cases} 1, & n = 1 \\ 2 \cdot T(n/2) + O(n), & n > 1 \end{cases}$$



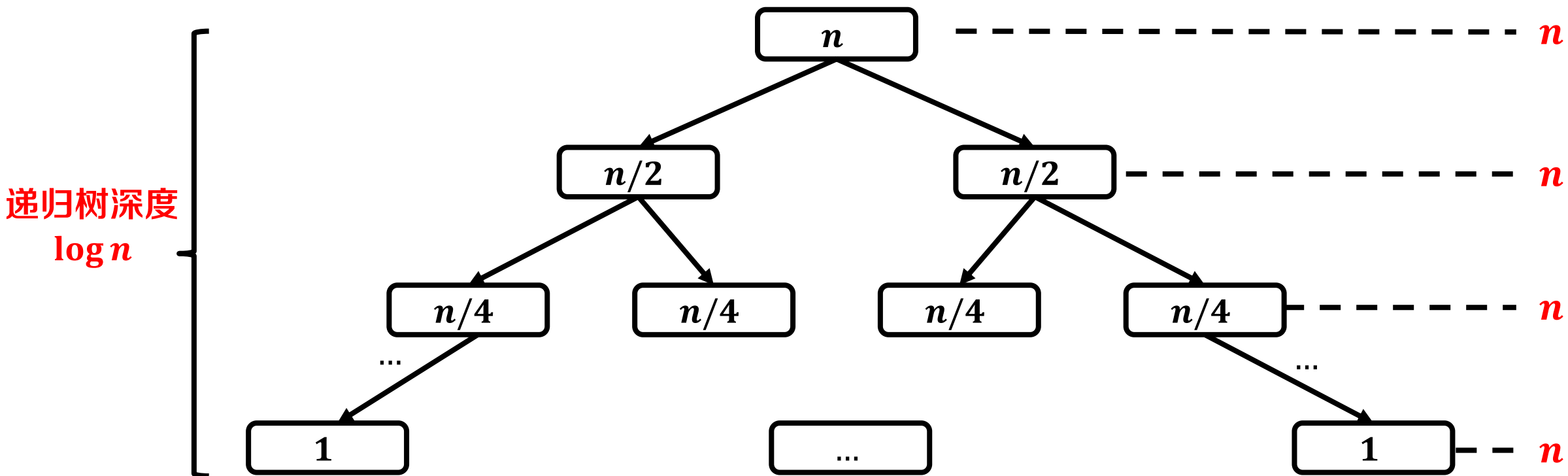
# 时间复杂度分析



- 输入规模为:  $n$

- $$T(n) = \begin{cases} 1, & n = 1 \\ 2 \cdot T(n/2) + O(n), & n > 1 \end{cases}$$

$T(n) = n \log n$





- 本节讲述了三种不同的算法，用于解决最大子数组和问题：

| 算法名称 | 时间复杂度         |
|------|---------------|
| 蛮力枚举 | $O(n^3)$      |
| 优化枚举 | $O(n^2)$      |
| 分而治之 | $O(n \log n)$ |

- 本节讲述了三种不同的算法，用于解决最大子数组和问题：

| 算法名称 | 时间复杂度         |
|------|---------------|
| 蛮力枚举 | $O(n^3)$      |
| 优化枚举 | $O(n^2)$      |
| 分而治之 | $O(n \log n)$ |
| ?    | $O(n)$        |

问题：是否可以设计一个时间复杂度为 $O(n)$ 的算法？

- 本节讲述了三种不同的算法，用于解决最大子数组和问题：

| 算法名称 | 时间复杂度         |
|------|---------------|
| 蛮力枚举 | $O(n^3)$      |
| 优化枚举 | $O(n^2)$      |
| 分而治之 | $O(n \log n)$ |
| 动态规划 | $O(n)$        |

问题：是否可以设计一个时间复杂度为 $O(n)$ 的算法？

答案：动态规划！